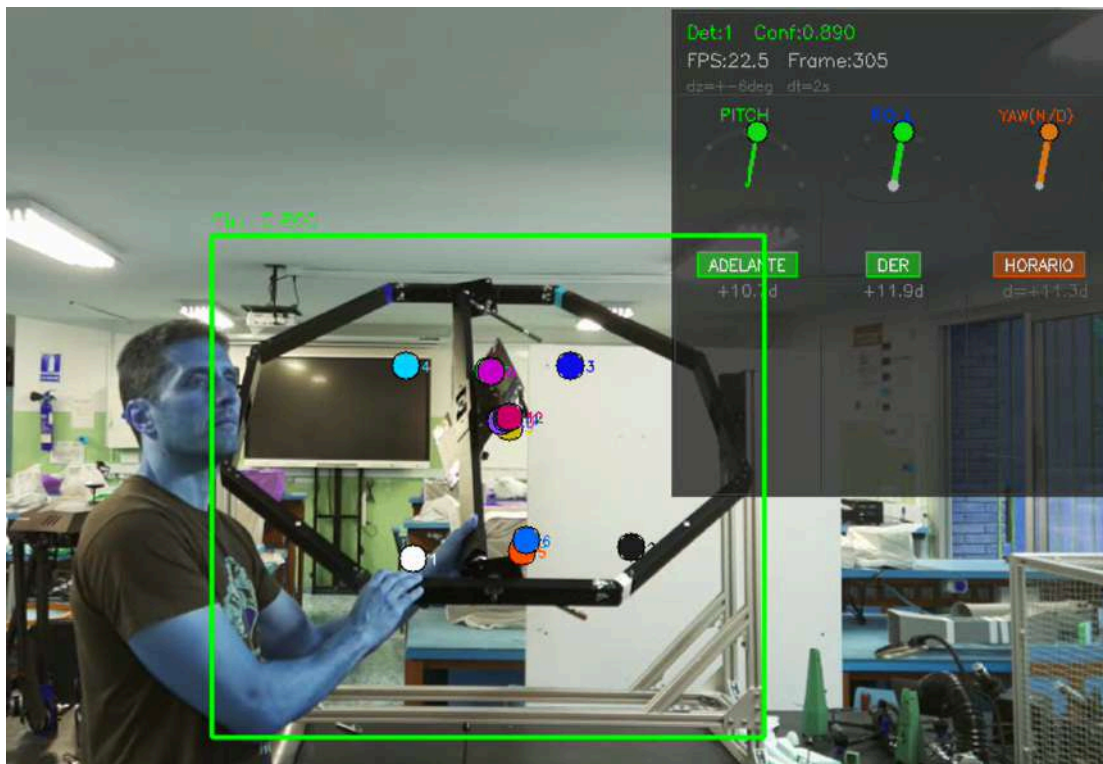


Proyecto de Elaboración de Materiales

Detección de Giroscopio para Drones con Edge AI en Tiempo Real

12 Keypoints · YOLOv8n-Pose · Hailo-8 · Raspberry Pi 5



Responsable del proyecto
José Luis Guerrero Marín

Málaga · Junio 2026

CC BY-SA 4.0

Creative Commons Attribution-ShareAlike 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes. If others remix, adapt, or build upon the material, they must license the modified material under identical terms.

Índice

Índice	1
1. Qué es este proyecto	3
1.1 Objetivo pedagógico	3
1.2 Hardware	4
1.3 Software y tecnologías	4
2. Trayectoria del proyecto	5
2.1 Producto físico inicial: puesto de trabajo Raspberry Pi 5 + giroscopio en aluminio	6
2.1.1 Documentación técnica descargable	7
2.1.2 Evidencias digitales del proceso	9
2.1.3 Validación del material	10
Validación técnica del giroscopio	10
Validación técnica del sistema de visión artificial	11
2.2 Giroscopio avanzado: proyecto con alumnos	11
Aprovechamiento por el alumnado	12
Instrumentos de validación y resultados	12
2.3 Giroscopio avanzado: trabajando en paralelo	12
3. Resultados del proyecto con alumnos	14
3.1. Grupo 1	16
3.2. Grupo 2	17
3.3. Grupo 3	18
3.4. Lecciones aprendidas	18
3.5. Versión rápida con 220 imágenes	19
4. Estructura de 12 keypoints	20
4.1 Lógica de cálculo de ángulos	21
5. Pipeline de inferencia	22
5.1 Letterbox 960×540 → 640×640	22
5.2 Tensores de salida del HEF	22
5.3 Parámetros clave del script (v3h — versión actual)	23
5.4 Panel visual v3h	23
6. Historia completa de bugs y soluciones	25
7. Dataset y estrategia de entrenamiento	29
7.1 Criterios del dataset	29
7.2 Script de grabación del dataset (grabar_dataset_v2.py)	29
7.3 Evolución del dataset del profesor (modelos p113–p410)	29
8. Comparativa de modelos — p113 / p211 / p309 / p410	31
8.1 Métricas comparativas	31
8.2 Progresión del mAP50-95(P)	31
8.3 Análisis por modelo	31
9. Compilación del modelo para Hailo-8	32
9.1 Entorno de compilación	32
9.2 Proceso de compilación	32
9.3 Verificación post-compilación	32
10. Estado actual del proyecto	33
10.1 Scripts disponibles	33
10.2 HEFs disponibles en la Raspberry Pi	33

10.3 Próximos pasos	34
11. Historial de sesiones	35
12. Conclusiones técnicas del proyecto	37
12.1 Por qué funciona Pitch y no Roll ni Yaw	38
12.2 Lo que el sistema sí consigue	38
12.3 Líneas de mejora futuras	38
13. Fases de fabricación física y evolución temporal	39
14. Lanzador de escritorio en Raspberry Pi	41
14.1 Estructura de la solución	41
14.2 Bloque configurable de los wrappers	41
lanzar_giroscopio.sh	41
lanzar_giroscopio_video.sh (añade VIDEO_DIR)	41
14.3 Problema encontrado: pantalla negra sin error	42
Causa raíz	42
Solución	42
14.4 Instalación de los lanzadores	43
14.5 Cambios en el script Python (v3h — 08/06/2026)	43
15. Estructura de carpetas y nomenclatura en la Raspberry Pi	44
15.1 Árbol de directorios completo	44
15.2 Descripción de cada directorio	44
15.3 Nomenclatura de los ficheros generados automáticamente	45
15.4 Referencia rápida: dónde tocar cada parámetro	45
15.5 Cómo cambiar el modelo activo	46
15.6 Instalación completa desde cero	46
16. Modelo a100: validación cruzada por etiquetado independiente	47
16.1 Contexto del experimento	47
16.2 Métricas de entrenamiento	47
Comparativa directa a100 vs p113	47
Por qué coinciden las métricas	48
16.3 Análisis de las curvas de entrenamiento	48
16.4 Resultados en inferencia (sesión 08/06/2026)	48
Métricas de la sesión (log: logs_giroscopio_v3a_a100_20260608_140337.txt)	49
Punto a mejorar: detecciones dobles	49
16.5 Valor pedagógico del experimento	49
16.6 Posición del a100 en la comparativa global	50
17. Reflexión pedagógica	51
18. Guía de replicación para docentes	52
18.1 Requisitos previos del docente	52
18.2 Ciclo completo del proyecto	52
18.3 Software necesario	52
18.4 Hardware del giroscopio de aluminio	54
18.5 Guía paso a paso y enlace con los anexos	56
18.6 Repositorio GitHub y recursos en línea	57
18.7 Cómo adaptar el proyecto a otro objeto	58
18.8 Carga de trabajo estimada para el alumnado	59

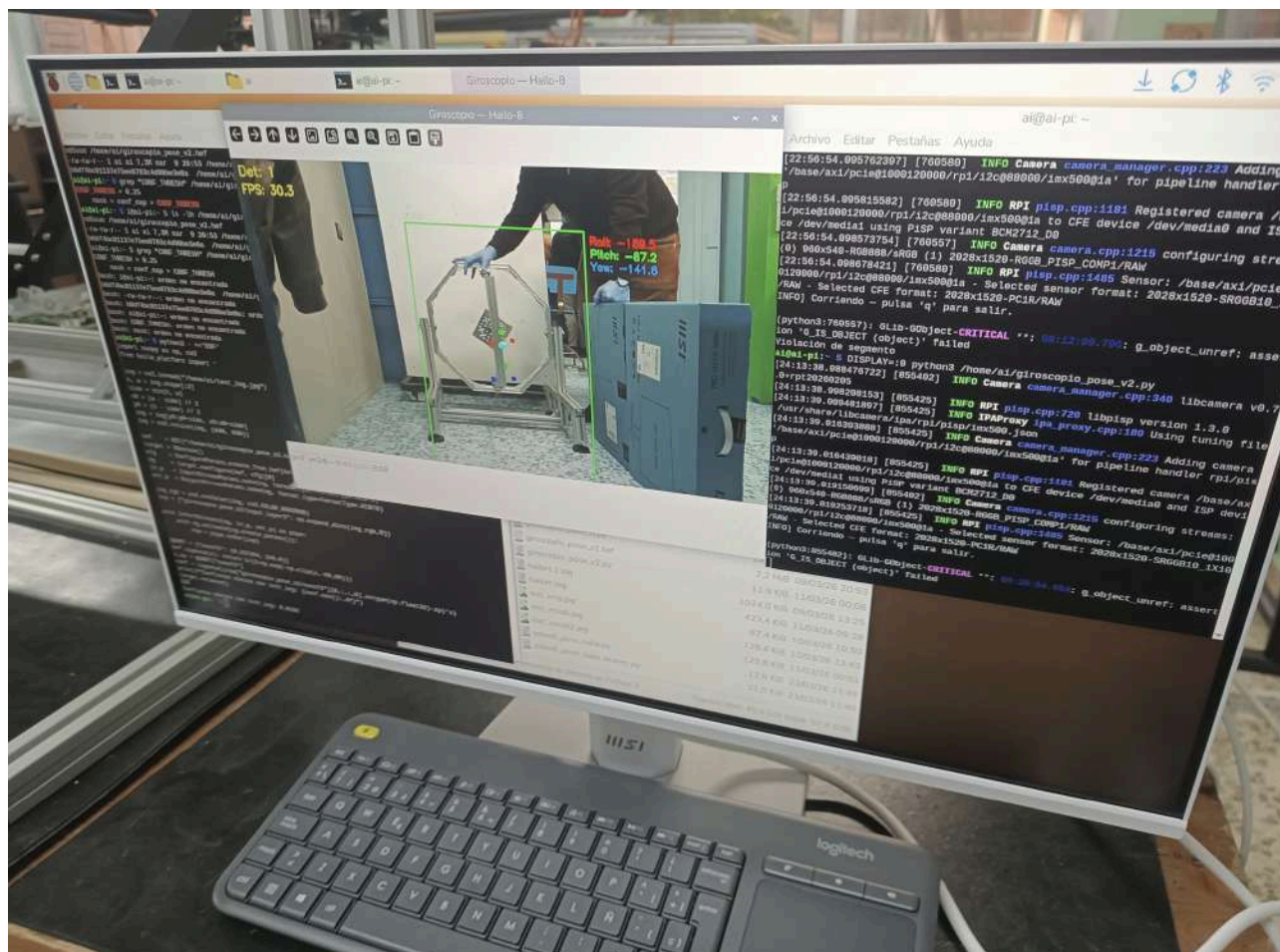
1. Qué es este proyecto

Este proyecto de elaboración de materiales que poco a poco ha ido tomando forma más bien de proyecto de innovación educativa desarrolla un sistema de visión artificial capaz de detectar un giroscopio de dron físico y medir sus tres ángulos de orientación (Roll, Pitch y Yaw) en tiempo real, sin ninguna conexión a la nube. Todo el procesamiento de inteligencia artificial se ejecuta directamente en una Raspberry Pi 5 equipada con el chip acelerador neuronal Hailo-8, lo que lo convierte en un sistema Edge AI puro.

El sistema combina un modelo de estimación de pose (YOLOv8n-Pose entrenado con un dataset propio del giroscopio), un compilador de redes neuronales para hardware embebido (Hailo Dataflow Compiler) y un pipeline de inferencia en Python que opera a 20–25 FPS. La filosofía del proyecto es que cada componente — desde la grabación del dataset hasta la inferencia en el chip — es comprensible, reproducible y extensible por los alumnos del ciclo.

1.1 Objetivo pedagógico

Inicialmente hay una fase en la que sólo el profesor ha realizado la construcción de un giroscopio en aluminio y pone a prueba un modelo de visión artificial con 8KP sin saber a dónde podría llegar. Una vez comprobado que es viable y se establece un protocolo suficientemente claro, se pasa a darle forma de práctica-proyecto para el alumnado, esta vez con 12KP para mejorar la precisión.



Así, el proyecto arranca con cuatro grupos de alumnos de CFGS Mantenimiento Electrónico, aunque finalmente se reagrupan en tres. Cada grupo grabó su propio vídeo del giroscopio, etiquetó al menos 50 imágenes por persona en Roboflow, entrenó su modelo YOLOv8n-Pose en Google Colab y participó en las pruebas sobre el hardware real.

El profesor desarrolló los scripts de inferencia, realizó la compilación de los modelos para Hailo-8 y dirigió el diagnóstico de los numerosos bugs que surgieron a lo largo del proceso.

El proyecto con alumnos no tuvo resultados totalmente satisfactorios al principio debido a numerosos errores comprensibles en un proceso de esta complejidad que se pasan a describir en el apartado 2.

El destino previsto del proyecto es su publicación en GitHub como material educativo abierto, y una posible presentación a los Premios Nacionales a Experiencias Educativas que Fomentan la Competencia Digital del Alumnado.

1.2 Hardware

Componente	Descripción
Raspberry Pi 5 (4GB)	Placa principal. CPU ARM Cortex-A76, 4GB LPDDR4X. Ejecuta el pipeline Python y la interfaz de vídeo.
Raspberry Pi AI Kit	Módulo Hailo-8 conectado por PCIe. 26 TOPS de rendimiento de inferencia neuronal. Este chip es el que ejecuta el modelo YOLOv8 compilado.
RPi Camera Module v2	Sensor IMX219. Entrega imágenes en RGB888 a 960×540. La misma cámara se usa para grabar el dataset de entrenamiento y para la inferencia en producción.
Giroscopio de dron ampliado	Estructura mecánica con tres ejes: octógono (Pitch), soporte interno en U (Roll) y base cuadrada (Yaw). Marcadores de color impresos en 12 puntos específicos de cada eje.

1.3 Software y tecnologías

Herramienta	Uso en el proyecto
YOLOv8n-Pose	Modelo de detección de keypoints. Variante nano (3.3M parámetros) elegida por su rendimiento en hardware con memoria limitada.
Hailo DFC 3.33.0	Hailo Dataflow Compiler. Transforma el modelo ONNX exportado por Ultralytics en un HEF (Hailo Executable Format) optimizado para el chip.
HailoRT 4.23.0	Runtime de inferencia en la Raspberry Pi. Gestiona el bus PCIe, la transferencia de tensores y los hilos de inferencia.
Picamera2	Librería de captura de vídeo para la cámara de la RPi. Entrega frames en RGB888.
OpenCV	Procesamiento de imagen: letterbox, visualización de keypoints, panel de ángulos, grabación de vídeo.
Google Colab	Entrenamiento del modelo en GPU NVIDIA T4 gratuita. El dataset se sube desde Google Drive.
Roboflow	Plataforma de etiquetado y gestión del dataset. Genera aumentaciones y exporta en formato YOLOv8 Pose.
Docker (hailo8_ai_sw_suite)	Imagen Docker (~17GB) que contiene el Hailo DFC. Se ejecuta en un PC Ubuntu para compilar los modelos.

2. Trayectoria del proyecto

El proyecto original preveía dotar al centro de una cierta cantidad de equipos compactos listos para programación de microprocesadores y microcontroladores. La cuantía inicialmente prevista era de 3977,76 €.

Al comunicarse la concesión de 800,00 €, se consideró que la simple adquisición de equipos genéricos con ese presupuesto no aportaría valor diferencial. En ese momento surgió una nueva necesidad real del Curso de Especialización en Aeronaves Pilotadas de Forma Remota (Drones): disponer de una plataforma controlada para el testeo del comportamiento de drones de pequeño formato.

Se decidió entonces dar una orientación más específica al proyecto, manteniendo el eje central —la plataforma Raspberry Pi— e incorporando capacidades de Inteligencia Artificial de borde (Edge AI) mediante el AI HAT+ (chip Hailo-8). El resultado es un único puesto de trabajo completo y funcional que sirve simultáneamente como:

- Equipo didáctico para el módulo de Equipos Microprogramables (CFGS Mantenimiento Electrónico).
- Banco de pruebas real para el Curso de Especialización en Mantenimiento de Drones.

Además, se ha obtenido:

- Material curricular replicable y publicado en abierto, con documentación técnica exhaustiva de 65 páginas, repositorio GitHub y vídeos explicativos. Toda esta documentación forma parte sólo de la primera parte del proyecto, que luego creció mucho más.
- Un procedimiento comprobado de trabajo para generación de un modelo de visión artificial replicable por cualquiera con unos conocimientos mínimos a nivel informático principalmente.

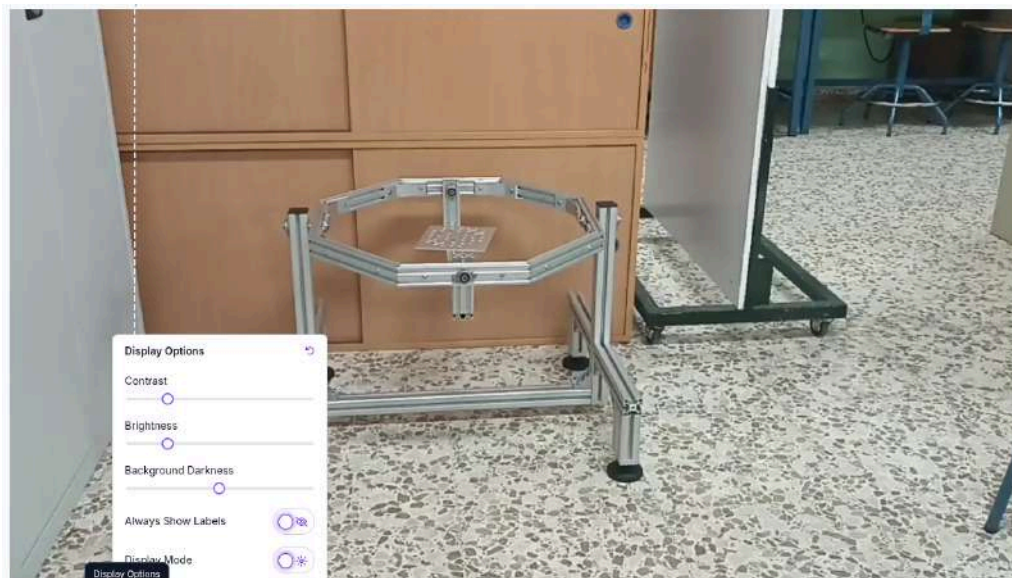
Este proyecto ha tenido 3 etapas y probablemente tenga la evolución a una cuarta, pero ya todo con cargo a los recursos propios del departamento:

1. **Giroscopio en aluminio y setup inicial de Raspberry Pi 5** con acelerado de IA Hailo 8 cámara IA, modelo de visión artificial basado en 8 keypoints para detectar la pose del giroscopio.
2. **Giroscopio avanzado (modelo comprado) y setup de la Raspberry Pi 5 empleado en la primera versión**, modelo de visión artificial basado en 12 keypoints para detectar la pose del giroscopio, los alumnos hacen grupos de 4 personas para el etiquetado y entrenamiento de un modelo por grupo haciendo al menos 50 imágenes por persona. Se prepara una práctica con relación curricular para el máximo aprovechamiento del alumnado.
3. **Giroscopio avanzado (modelo comprado) y setup de la Raspberry Pi 5 derivado de la segunda versión** mejorando los scripts y los dataset empleados y trabajando en paralelo alumno y profesor sobre el mismo dataset. Al alumno se le pide que al menos genere un modelo con 100 imágenes y haga todo el entrenamiento y compilado por sí mismo usando sólo las instrucciones dadas en tutoriales escritos por el profesor.
4. **Giroscopio de aluminio modificado cambiando todas las piezas móviles por piezas fabricadas en fibra de carbono** para reducir el peso y tal vez añadiendo una segunda cámara para mejorar la precisión y/o uso de acelerómetros a batería.

2.1 Producto físico inicial: puesto de trabajo Raspberry Pi 5 + giroscopio en aluminio

Se ha construido y puesto en funcionamiento un puesto de trabajo completo compuesto por:

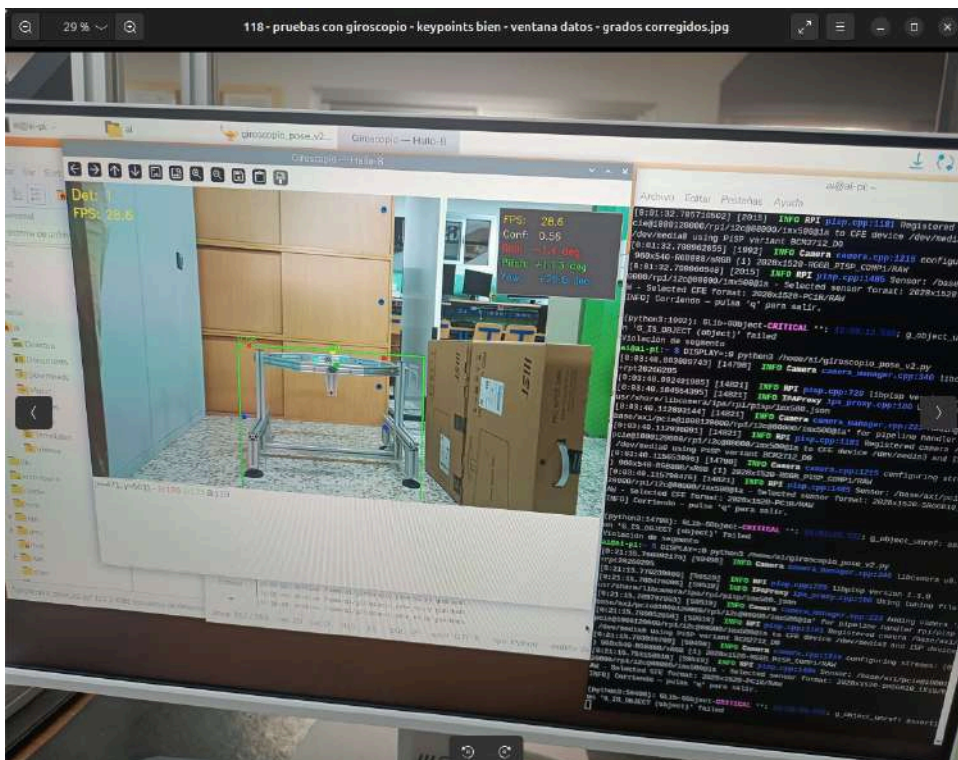
- Giroscopio mecánico de tres ejes fabricado íntegramente en el departamento de Electrónica del centro, en aluminio estructural extruido (perfilería T-Slot series 4040, 4080 y 2040), con octógono de 8 lados para el eje Pitch, soporte intermedio para el eje Roll, y base cuadrada para el eje Yaw.



- Raspberry Pi 5 (8 GB) con AI HAT+ (chip Hailo-8), cámara oficial, soporte VESA impreso en 3D acoplado al monitor, soporte de cámara impreso en 3D, monitor MSI 24,5" 120 Hz, teclado inalámbrico Logitech K400.



- Sistema de visión artificial funcionando en tiempo real: detección de la orientación del giroscopio (Roll, Pitch, Yaw) a ~25-30 FPS, sin conexión a internet ni GPU externa.



Documentación del proceso:

- Entrada del blog:
 - <https://blogsaverroes.juntadeandalucia.es/industria4/2026/03/15/deteccion-de-orientacion-de-giroscopio-para-drones/>
- Documentación completa en Github:
 - <https://github.com/nutshell-electronica/gyroscope>
- Explicación en Youtube del proyecto generada con Notebook LM a partir del documento tutorial disponible en Github:
 - <https://youtu.be/m3AsXro8m2U>
- Vídeo resumen en Youtube de la evolución del proyecto desde sus inicios a los resultados:
 - https://youtu.be/OTw97G_3yfA

2.1.1 Documentación técnica descargable

Toda la documentación está disponible en el repositorio público del proyecto:

- Tutorial técnico completo (65 páginas): tutorial_giroscopio_v6.pdf — cubre desde el montaje físico hasta el pipeline completo de visión artificial (Roboflow, Google Colab, Docker, Hailo DFC, inferencia en Raspberry Pi).

-
- The screenshot shows a Jupyter Notebook with the following code:
- ```

CELDA 7 - Exportar a ONNX (formato para Hailo)
=====
IMPORTANTE: ejecutar solo si la Celda 6 confirmo que el
modelo funciona correctamente.
#
El pipeline de Hailo DFC (Data Flow Compiler) requiere un
ONNX con salidas SEPARADAS (no un unico tensor fusionado).
Para ello necesitamos:
- opset 11 (maxima compatibilidad con Hailo DFC)
- simplify=True (elimina operaciones redundantes)
- dynamic=False (shapes fijas, requerido por Hailo)
#
El modelo exportado y el .pt se guardan en la carpeta raíz
del proyecto en Drive ('giroscopio_proyecto_gi/').

from ultralytics import YOLO
import shutil, os

DRIVE_ROOT = '/content/drive/MyDrive/giroscopio_proyecto'
RUN_NAME = 'giroscopio_v3a-p211'

best_pt = os.path.join(DRIVE_ROOT, 'runs', RUN_NAME, 'weights', 'best.pt')
model = YOLO(best_pt)

onnx_path = model.export(
 format = 'onnx',
 imgsz = 640,
 opset = 11,
 simplify = True,
 dynamic = False,
)

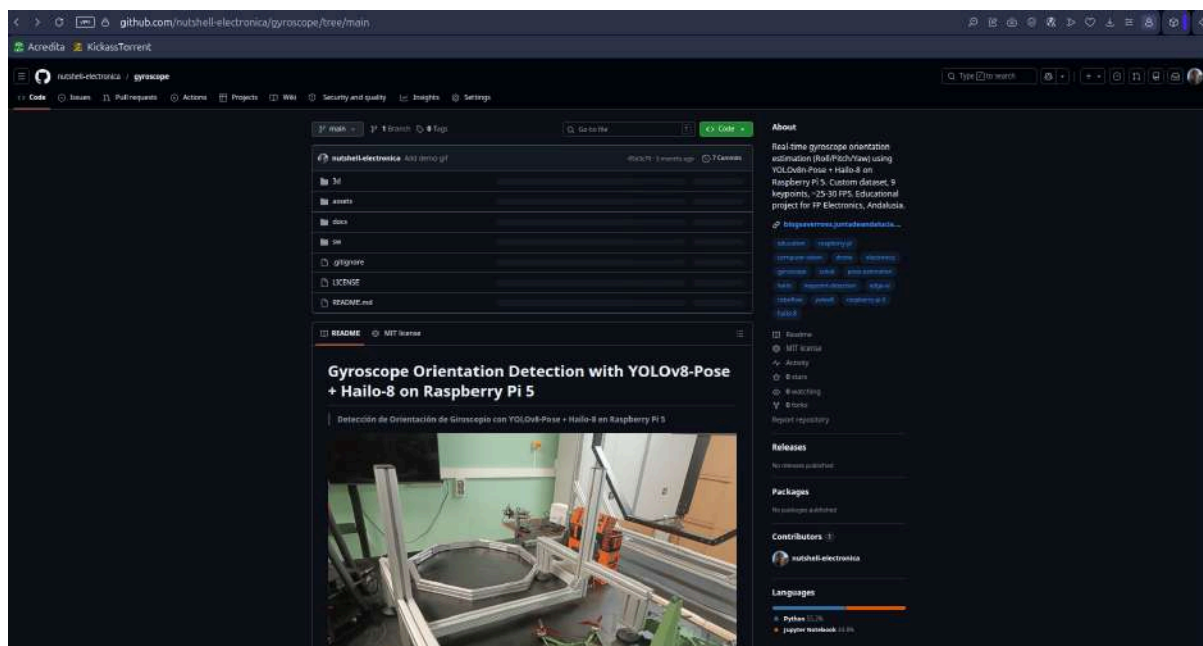
print(f'ONNX exportado es: {onnx_path}')

Verificar el ONNX: con 12 KP y salidas separadas debe
tener 3 tensores de salida (3 escalas = bbox + conf + kp)
Si hay solo 1 salida fusionada, no sirve para Hailo.
import onnxruntime as ort
sess = ort.InferenceSession(str(onnx_path))
outputs = sess.get_outputs()

```

Repositorio GitHub (acceso público):

<https://github.com/nutshell-electronica/gyroscope>



## 2.1.2 Evidencias digitales del proceso

- Entrada del blog Industria 4.0 (Averroes):  
<https://blogsaverroes.juntadeandalucia.es/industria4/2026/03/15/deteccion-de-orientacion-d-e-giroscopio-para-drones/>



- Vídeo resumen del proyecto (evolución desde inicio a resultados finales):  
[https://youtu.be/OTw97G\\_3yfA](https://youtu.be/OTw97G_3yfA)



- Explicación técnica generada con NotebookLM a partir del tutorial: <https://youtu.be/m3AsXro8m2U>



### 2.1.3 Validación del material

#### Validación técnica del giroscopio

El testeo del montaje físico arrojó los siguientes resultados:

- Equilibrado de la estructura: **ÉXITO**. La estructura queda perfectamente equilibrada y permite el giro libre en los tres ejes.
- **Limitación** detectada: El peso de la estructura móvil (octógono + soporte intermedio + base cuadrada en aluminio) supera la capacidad de los motores de los drones de pequeño formato (DJI Avata 2, DJI Mini 4 Pro). El dron no tiene fuerza suficiente para desplazar el giroscopio en sus ejes durante el vuelo. Esta limitación no afecta al uso didáctico actual del material (el giroscopio se manipula manualmente durante las demostraciones y prácticas), pero sí condiciona las futuras versiones del proyecto.

### Validación técnica del sistema de visión artificial

El sistema de inferencia fue validado con los siguientes resultados:

- Detección y localización del giroscopio: **ÉXITO COMPLETO**. El sistema detecta el giroscopio en todas las posiciones, dibuja el bounding box y los 8 keypoints con confianza alta y sin pérdida de tracking, a una tasa de ~25-30 FPS.
- Cálculo de ángulos Roll, Pitch y Yaw: **PARCIALMENTE CORRECTO**. El cálculo de Pitch es correcto. Se identificó un error conceptual en el cálculo del Roll (los keypoints usados pertenecen a la estructura fija y no varían con el Roll del giroscopio). El Yaw presenta inestabilidad por insuficiencia de keypoints. Estos errores están documentados y tienen solución clara en el código, sin necesidad de reentrenar el modelo.
- Dataset: 168 imágenes originales, aumentadas a 404 por Roboflow.
- Pruebas con hardware real: realizadas con DJI Avata 2 y DJI Mini 4 Pro durante aproximadamente 10 minutos. El pipeline completo funciona de extremo a extremo.

⚠ Las limitaciones identificadas (peso de la estructura, cálculo de ángulos) son propias del proceso de investigación y prototipado. No impiden el uso didáctico actual del material ni la replicabilidad del proyecto por otros centros. Están íntegramente documentadas en el tutorial (Capítulos 10, 15 y 17) y constituyen, en sí mismas, un elemento didáctico de valor: el alumnado aprende a identificar, analizar y proponer soluciones a problemas reales de ingeniería.

## 2.2 Giroscopio avanzado: proyecto con alumnos

En esta versión se trabaja con un giroscopio comprado, de mayor calidad, mayor tamaño y menor peso que el que podemos fabricar nosotros y nos centramos sólo en la parte de la visión artificial diseñando una práctica con el siguiente enunciado (más desarrollado en el documento de la práctica):

Esta práctica se enmarca en un proyecto de innovación educativa real desarrollado por el profesor del módulo. El sistema ya funciona: una Raspberry Pi 5 con un chip de inteligencia artificial Hailo-8 es capaz de detectar un giroscopio de dron en tiempo real, identificar puntos clave de su estructura y calcular los ángulos de orientación Roll, Pitch y Yaw a una tasa de 25-30 imágenes por segundo, sin necesidad de conexión a internet ni de ningún ordenador externo.

Vuestra misión es entrenar una nueva versión del modelo con 12 keypoints distribuidos en tres grupos funcionales, cada uno responsable de uno de los tres ejes de rotación. Para hacer el etiquetado más preciso y reproducible, los puntos estarán marcados físicamente sobre el giroscopio mediante etiquetas impresas que combinan una franja de color único y un marker ArUco — un sistema de doble identificación que añade redundancia frente a condiciones de iluminación adversas u oclusiones parciales.

Cada grupo construirá su propio dataset desde cero, entrenará su propio modelo y al final todos los modelos se probarán en el mismo hardware real. Tendréis métricas objetivas para comparar vuestro trabajo entre grupos y con el modelo de referencia del profesor.

📌 *Todo el software que usamos es gratuito: Google Colab para el entrenamiento, Roboflow para el etiquetado del dataset, y herramientas de código abierto para el resto del pipeline.*



### Aprovechamiento por el alumnado

El recurso ha sido empleado con el alumnado de Equipos Microprogramables del curso de 1º de Mantenimiento Electrónico durante el curso 2025-26 con la realización de una práctica relacionada curricularmente.

### Instrumentos de validación y resultados

Los instrumentos de validación empleados han sido:

- Pruebas técnicas de funcionamiento del hardware (equilibrado del giroscopio, funcionamiento del sistema de inferencia en Raspberry Pi 5).
- Métricas de entrenamiento del modelo de visión artificial (Box mAP50, Pose mAP50, curvas de pérdida — disponibles en el repositorio GitHub).
- Práctica evaluada con el alumnado de 1º CFGS Mantenimiento Electrónico, con rúbrica de evaluación incluida en la práctica.

## 2.3 Giroscopio avanzado: trabajando en paralelo

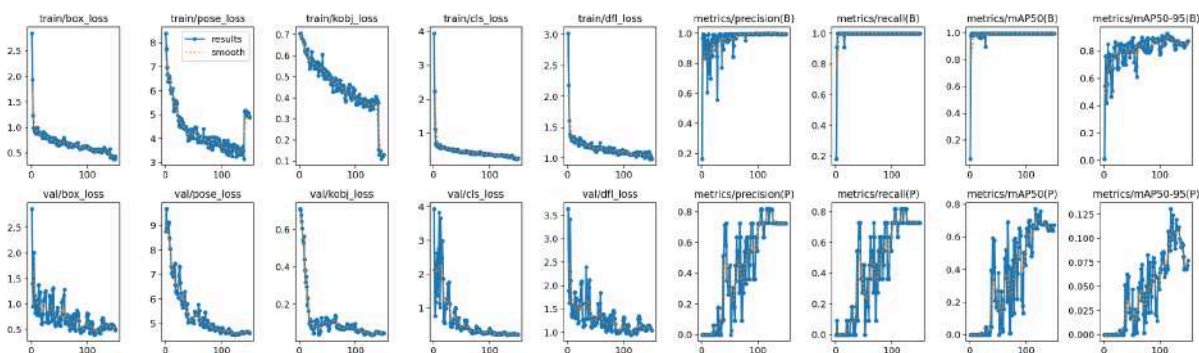
En esta versión se trabaja en mejorar por un lado los scripts y los dataset empleados y trabajando en paralelo alumno y profesor sobre el mismo dataset. Al alumno se le pide que al menos genere un modelo con 100 imágenes y haga todo el entrenamiento y compilado por sí mismo usando sólo las instrucciones dadas en tutoriales escritos por el profesor.

El resultado es fantástico porque se verifica lo que se pretendía:

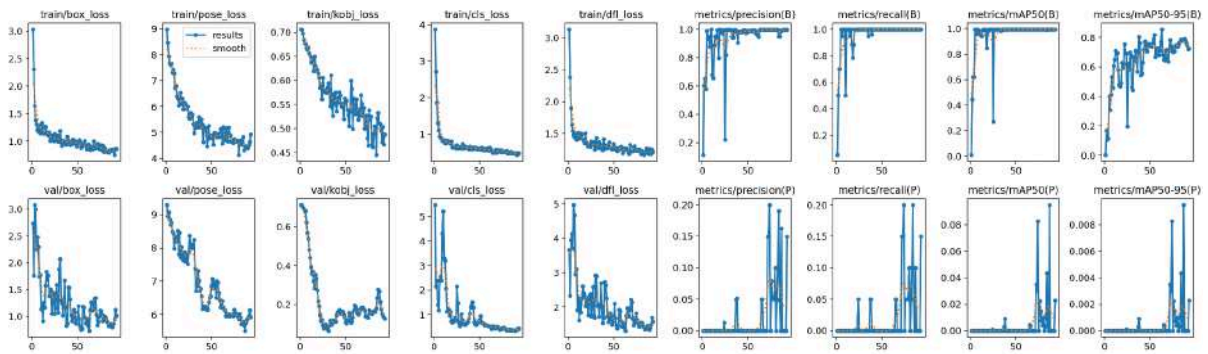
- El procedimiento es perfectamente replicable con unas instrucciones precisas.
- El modelo es consistente, ya que el modelo de 100 imágenes de profesor y alumno coinciden casi exactamente en las métricas obtenidas.

Ambos modelos comparten el mismo techo de rendimiento con datasets pequeños sin augmentación. La bbox detection ( $mAP50(B)=0.995$ ) satura rápido porque detectar el contorno del giroscopio es una tarea sencilla para YOLOv8n. La pose estimation ( $mAP50-95(P)=0.13$ ) es donde aparece la limitación real: con ~100 imágenes sin augmentación y un único fondo, el modelo aprende a localizar los keypoints en ese entorno concreto pero no generaliza bien a variaciones de perspectiva o iluminación. Eso es exactamente lo mismo que ocurre con p113.

El hecho de que el alumno llegue exactamente al mismo techo con su propio etiquetado demuestra que la limitación no está en la calidad del etiquetado sino en el tamaño y variedad del dataset. Es la misma conclusión que llevó al profesor a ampliar progresivamente el dataset hasta p410.



Resultados del modelo entrenado en Colab por el profesor con 113 imágenes



Resultados del modelo entrenado en Colab por el alumno con 100 imágenes

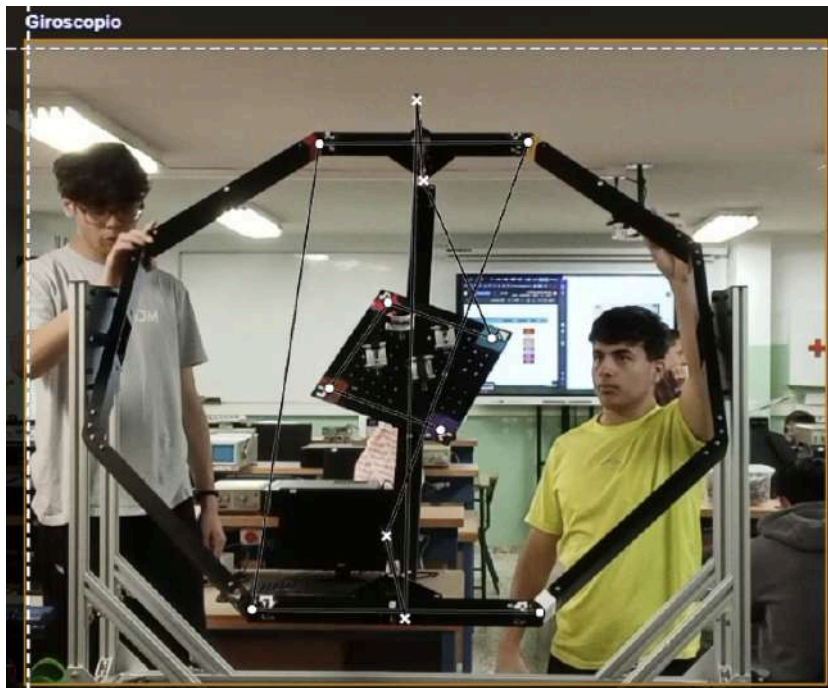
### 3. Resultados del proyecto con alumnos

Tras todas las pruebas realizadas con los modelos de los 3 equipos (inicialmente eran 4, pero luego hubo reajustes entre miembros de los grupos) podemos sacar bastantes conclusiones:

- **Fallos identificados en etiquetado y metodología:**
  - En el grupo G1, al menos uno de los miembros, ha hecho un etiquetado erróneo de varios de los puntos, colocándolos fuera de su posición real.



- En el grupo G2, algunos puntos visibles están marcados como ocultos.



- En el grupo G3 no han marcado nunca los puntos ocultos, los han marcado siempre como visibles aunque no lo fueran; algunas imágenes las han dejado sin etiquetar, algunos puntos están muy lejos de su posición real.



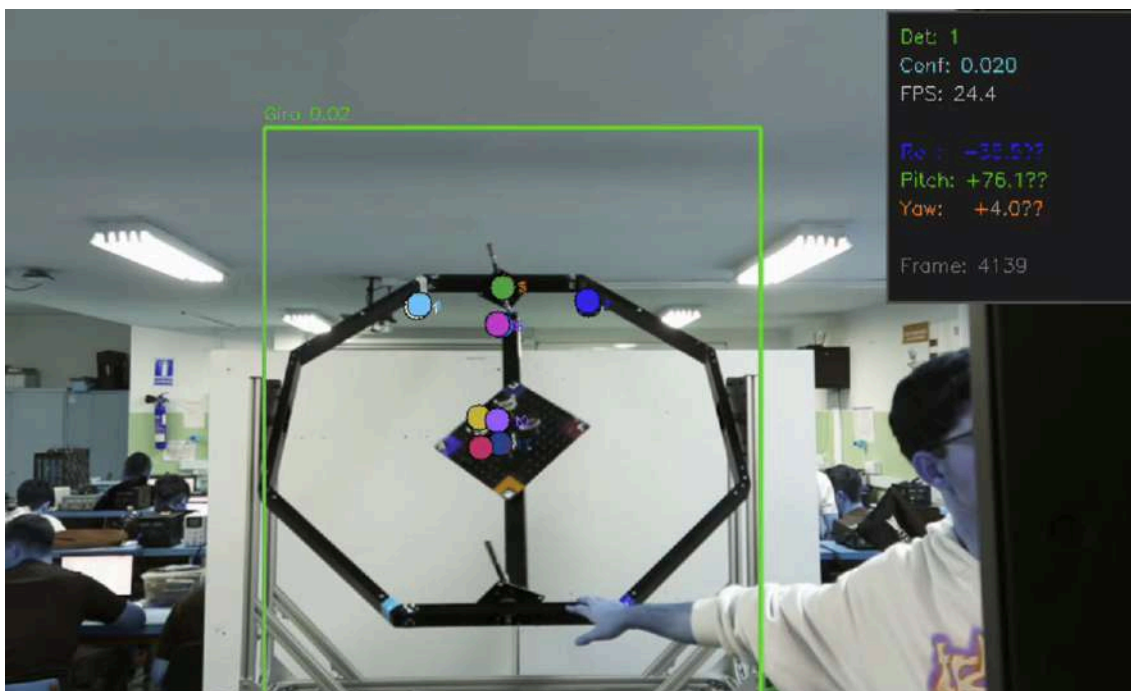
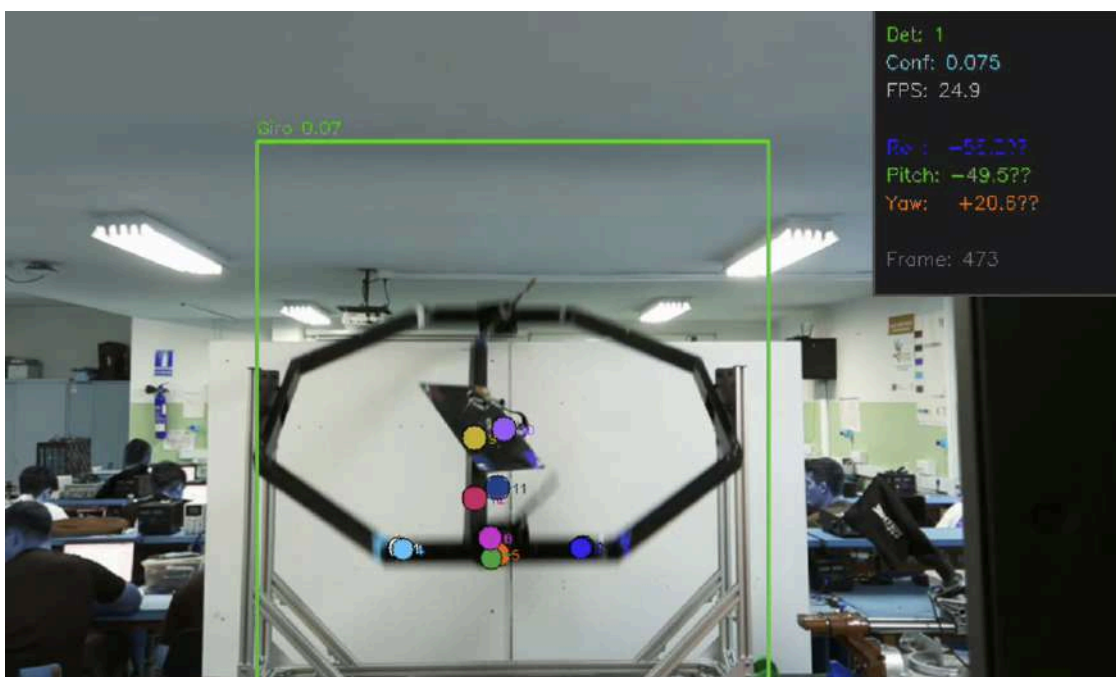
- Habría sido mejor hacer uso de un fondo homogéneo para facilitar el entrenamiento.
- Habría sido mejor hacer uso de fotogramas para el entrenamiento en posiciones estáticas que dejaran clara la posición del giroscopio, no un video con el giroscopio girando a toda pastilla. Seguramente sería una buena idea colocar un dron y hacer movimientos reales del giroscopio en lugar de moverlo a mano.
- Etiquetado de imágenes: tener un criterio muy claro del posicionamiento del punto en la estructura facilitaría el entrenamiento de la IA.
- Aunque la cámara luego permanezca fija, igual es buena idea tomar 3-4 vídeos con ángulos ligeramente diferentes o incluso a horas diferentes para cambiar los posibles reflejos e iluminaciones.
- **Pipeline documentado.** Ya se dispone de un pipeline claro de procesamiento de datos a partir de fotogramas, documentado en el tutorial hailo8 v2. Ahora puede repetirse el proceso con relativa facilidad.
- **Ensemble de modelos viable.** A pesar de los malos datos de partida de los 3 modelos, parece viable hacer una combinación por código de los 3 modelos montando un ensemble a partir de un script Python que use los 3 HEF. Salen detecciones a una fluidez de entre 28 y 30 FPS.
- **Ángulos inconsistentes por KPs mal localizados.** Los ángulos apenas varían o varían de forma inconsistente, claramente por la inconsistencia del posicionamiento de los KPs: los KP1 (blanco, oct\_tras\_izq) y KP2 (negro, oct\_tras\_der) no aparecen casi nunca y cuando aparecen están solapados con los opuestos. Es difícil establecer el motivo al no poder asegurar la consistencia del etiquetado entre diferentes personas ni el rol de la calidad de los fotogramas.
- **Conclusión sobre viabilidad del modelo 12KP.** El modelo de 12 puntos en principio debería funcionar, pero con estos datos de partida es imposible sacar conclusiones definitivas sobre su viabilidad o si podría llegarse a tener ángulos correctos.

Aún tenemos pendiente el uso de 2 cámaras separadas 90° para mejorar la visión y el cálculo de ángulos.

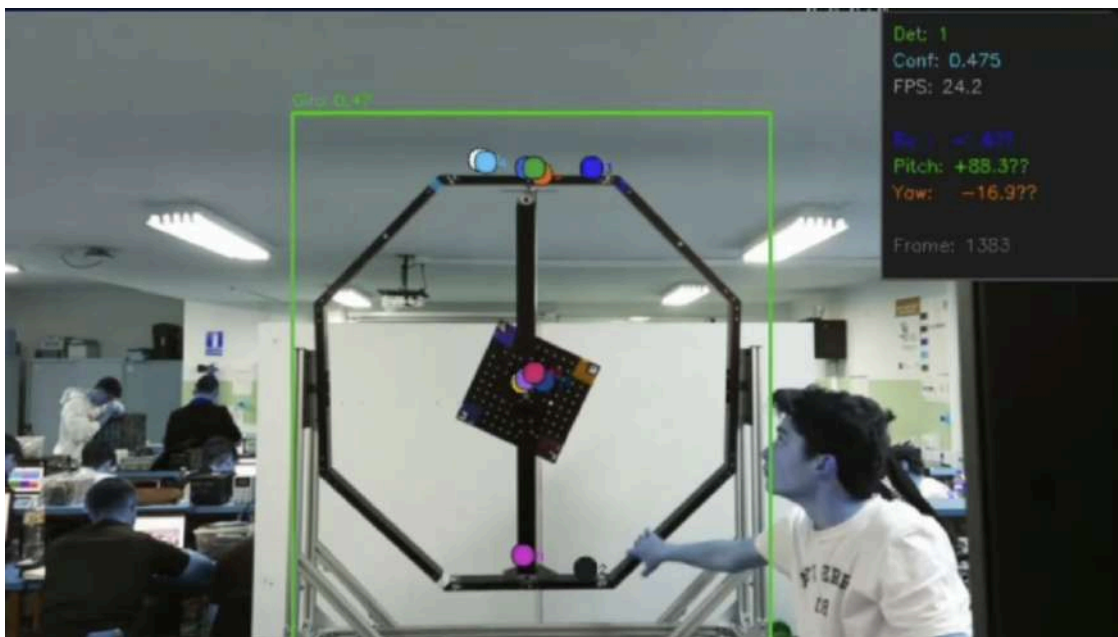


En las imágenes siguientes, extraídas de fotogramas grabados directamente de la cámara de la Raspberry PI se ven algunos de los fallos de etiquetas y ángulos en los 3 grupos. Además, se ve el color azul, efecto de la forma de grabación de la cámara de la Raspberry, aunque esto está corregido en código para la detección de KP.

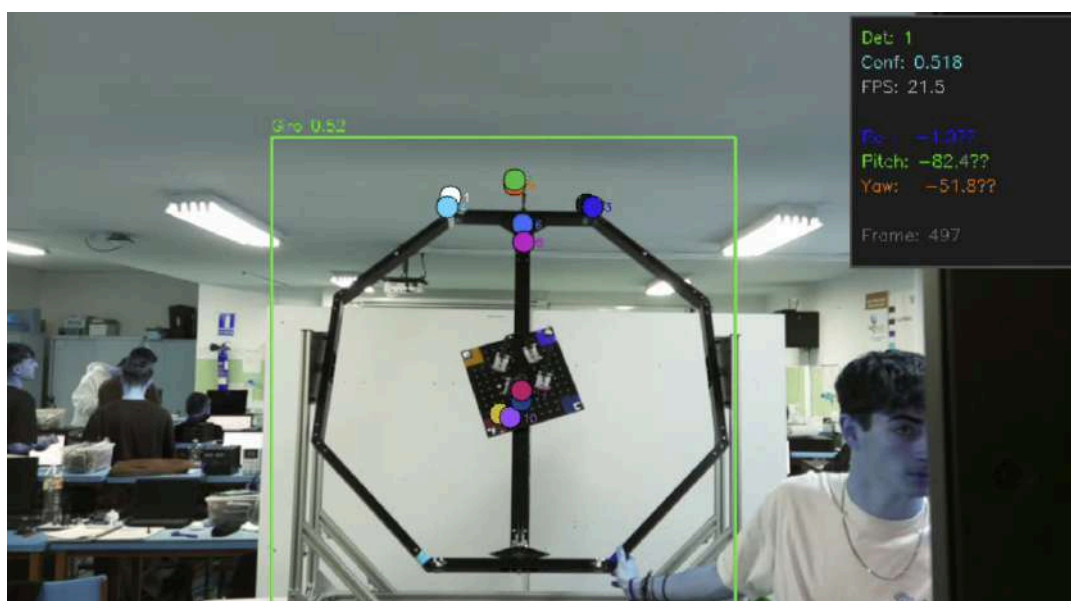
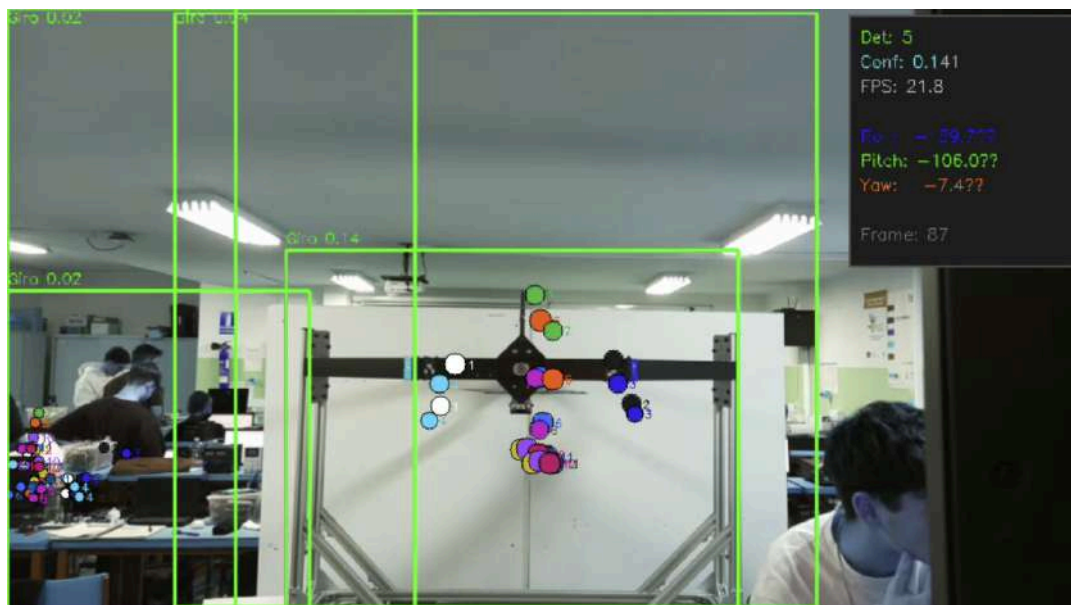
### 3.1. Grupo 1



### 3.2. Grupo 2



### 3.3. Grupo 3



### 3.4. Lecciones aprendidas

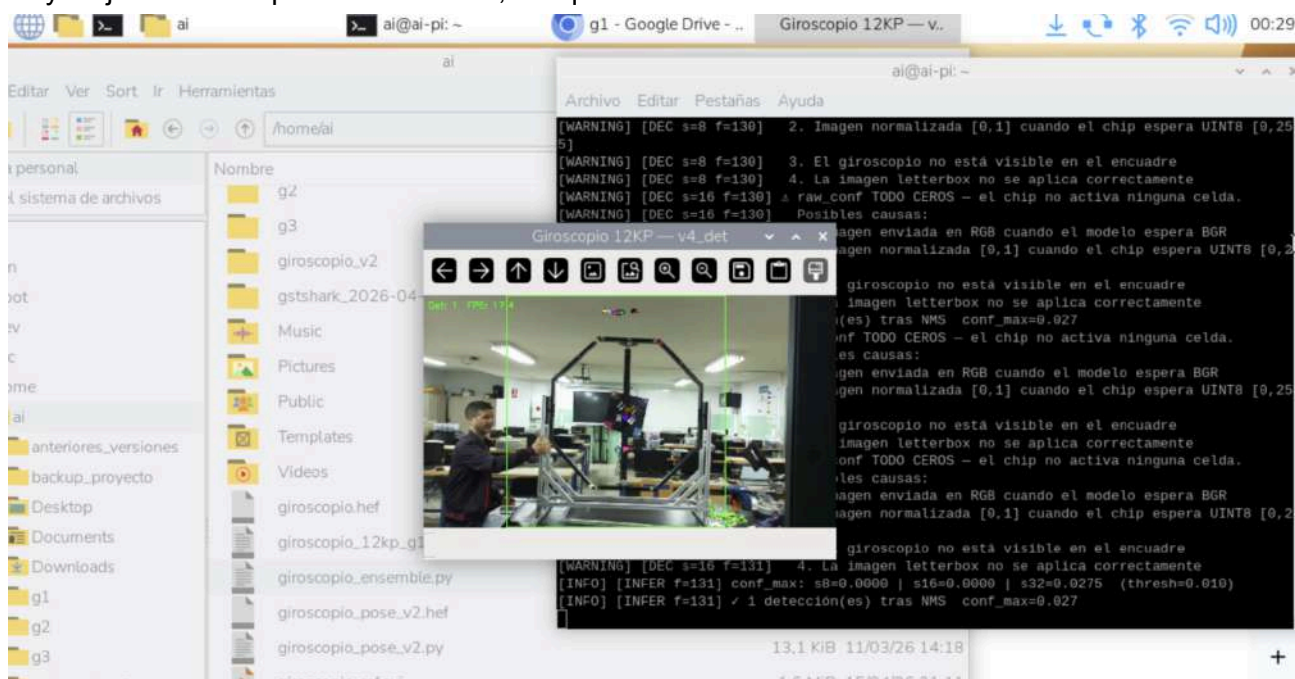
Como avance rápido y gratuito en coste económico, se podría hacer un modelado basado en unas pocas imágenes: 150 podrían ser suficientes a partir de un video con un dron montado en la estructura, luego aumentarlas con Roboflow, y a partir de ello montar un nuevo modelo de 12 puntos.

Como el script ya está montado y funcional, podría probarse inmediatamente y corroborar o desmentir las hipótesis que se han lanzado aquí.

Una segunda posibilidad (más costosa y complicada) sería la de montar la estructura fija con 2 cámaras haciendo un nuevo script que aprovechara esta circunstancia. Esto sólo tendría sentido una vez comprobado el punto anterior.

### 3.5. Versión rápida con 220 imágenes

Siguiendo la teoría del fallo de etiquetado se plantea realizar un siguiente prototipado rápido con un video nuevo, extrayendo 220 imágenes y reetiquetando, pero el resultado sigue siendo malo, esta vez porque el giroscopio ocupaba demasiado espacio en la imagen y los puntos quedaban muy alejados de las posiciones reales, así que lo descartamos también.





## 4. Estructura de 12 keypoints

El diseño de keypoints es el núcleo del sistema. Cada keypoint es un marcador de color impreso sobre una parte específica del giroscopio. El modelo aprende a localizar esos marcadores en la imagen y sus coordenadas se usan directamente para calcular los ángulos. La elección de 12 keypoints (frente a los 8 originales del primer diseño) permite un cálculo más robusto de cada eje de rotación al disponer de cuatro puntos por eje en lugar de dos.

Los 12 keypoints se organizan en tres grupos de cuatro, uno por eje:

| KP | Grupo | Color marcador | Posición física            | Eje   | Visibilidad                |
|----|-------|----------------|----------------------------|-------|----------------------------|
| 1  | OCT   | Blanco         | Esq. trasera-izq. octógono | PITCH | Media (oclusión frecuente) |
| 2  | OCT   | Negro          | Esq. trasera-der. octógono | PITCH | Media (oclusión frecuente) |
| 3  | OCT   | Rojo vivo      | Esq. frontal-der. octógono | PITCH | Alta                       |
| 4  | OCT   | Amarillo       | Esq. frontal-izq. octógono | PITCH | Alta                       |

| KP | Grupo | Color marcador | Posición física               | Eje  | Visibilidad |
|----|-------|----------------|-------------------------------|------|-------------|
| 5  | INT   | Azul eléctrico | Punta U arriba-izq. soporte   | ROLL | Alta        |
| 6  | INT   | Naranja        | Travesaño arriba-der. soporte | ROLL | Alta        |
| 7  | INT   | Verde lima     | Punta U abajo-izq. soporte    | ROLL | Alta        |
| 8  | INT   | Magenta        | Travesaño abajo-der. soporte  | ROLL | Alta        |

| KP | Grupo | Color marcador | Posición física        | Eje | Visibilidad              |
|----|-------|----------------|------------------------|-----|--------------------------|
| 9  | BASE  | Cian           | Esq. trasera-izq. base | YAW | Baja si encuadre cercano |
| 10 | BASE  | Rosa           | Esq. trasera-der. base | YAW | Baja si encuadre cercano |
| 11 | BASE  | Marrón         | Esq. frontal-der. base | YAW | Baja si encuadre cercano |
| 12 | BASE  | Morado         | Esq. frontal-izq. base | YAW | Baja si encuadre cercano |

## 4.1 Lógica de cálculo de ángulos

Con cuatro puntos por eje, el cálculo se basa en el eje principal de un rectángulo, no en una simple línea entre dos puntos. Esto lo hace más robusto ante la oclusión de uno de los KPs.

### Pitch (KP 1, 2, 3, 4 — octógono):

```
mid_sup = promedio(KP1, KP2) # punto medio del lado superior
mid_inf = promedio(KP3, KP4) # punto medio del lado inferior
Pitch = ángulo(mid_sup → mid_inf) respecto a la vertical
```

### Roll (KP 5, 6, 7, 8 — soporte):

```
Vector izquierdo: KP5 → KP7 (punta U)
Vector derecho: KP6 → KP8 (travesaño)
Roll = promedio circular de ambos vectores
```

### Yaw (KP 9, 10, 11, 12 — base):

```
mid_izq = promedio(KP9, KP12) # lado izquierdo de la base
mid_der = promedio(KP10, KP11) # lado derecho de la base
Yaw = ángulo(mid_izq → mid_der) respecto al norte de referencia
```

### Parámetro ROLL\_AXIS\_RATIO

ROLL\_AXIS\_RATIO (R) es configurable en la cabecera del script. R=0 sitúa el eje en KP5/KP7, R=0.5 en el punto medio, R=1 en KP6/KP8. Se ajusta midiendo físicamente la posición del eje de rotación real en el giroscopio. El valor por defecto es 0.5.

## 5. Pipeline de inferencia

El pipeline completo desde la cámara hasta la visualización en pantalla opera en un bucle de captura principal con un hilo dedicado a la inferencia en el Hailo-8. Esto permite que la captura de frames no se bloquee esperando la respuesta del chip.

```
Picamera2 (960×540 RGB888)
 ↓ letterbox → 640×640 (scale=0.667, pad_y=140)
 ↓ SIN conversión RGB→BGR ← CRÍTICO para modelos entrenados con Roboflow/Colab
Hailo-8 (inferencia YOLOv8n-Pose)
 ↓ 9 tensores de salida
Decodificación Python
 ↓ bbox DFL (conv43/57/70 · UINT8 · stride 8/16/32)
 ↓ confianza (conv44/58/71 · UINT8 · formato PROB)
 ↓ keypoints (conv45/59/72 · UINT16 · 12KP × 3 valores)
 ↓ NMS + cálculo Roll/Pitch/Yaw
Display OpenCV (ventana ×3, panel lateral ángulos + FPS)
```

### 5.1 Letterbox 960×540 → 640×640

La cámara entrega frames a 960×540. El modelo espera 640×640. El letterbox escala manteniendo la relación de aspecto y añade relleno negro en los bordes:

```
scale = 640 / 960 = 0.6667
nw = 640
nh = 360
pad_y = (640 - 360) / 2 = 140 px arriba y abajo
pad_x = 0
```

Este letterbox tiene consecuencias importantes para el dataset: los keypoints de la base (KP9–12) caen típicamente en torno a y≈460px del espacio 640×640. Si el giroscopio se grabó demasiado cerca, la base queda fuera del encuadre en muchos frames y el modelo nunca aprende a detectarla.

### 5.2 Tensores de salida del HEF

| Tensor                   | Tipo   | Formato     | Contenido                                                   |
|--------------------------|--------|-------------|-------------------------------------------------------------|
| conv43 / conv57 / conv70 | UINT8  | FCR(H×W×64) | Bounding box DFL (strides 8, 16, 32)                        |
| conv44 / conv58 / conv71 | UINT8  | NHWC(H×W×1) | <b>Confianza — formato PROB</b> (zp=0, scale≈1/255)         |
| conv45 / conv59 / conv72 | UINT16 | FCR(H×W×36) | <b>Keypoints</b> (12KP × 3 = 36 valores: x, y, visibilidad) |

#### Importante: tensores de confianza en formato PROB

Los tensores conv44/58/71 ya devuelven probabilidades en [0,1] (zero\_point=0, scale=1/255). No se les debe aplicar sigmoid. Aplicarlo daría sigmoid(0)=0.5 para todas las celdas, generando ~1500 falsas detecciones por frame. El código detecta este formato automáticamente comparando el zero\_point con un umbral.

### 5.3 Parámetros clave del script (v3h — versión actual)

| Parámetro           | Default | Descripción                                                                                                                                                                                                                                    |
|---------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CONF_THRESH         | 0.05    | Umbral de objectness. Subido de 0.01 en v3g tras pruebas físicas.                                                                                                                                                                              |
| KP_THRESH           | 0.07    | Visibilidad mínima por KP. Bajado de 0.10 en v3g para recuperar KPs en perspectiva extrema.                                                                                                                                                    |
| IOU_THRESH          | 0.30    | IoU máximo NMS. Bajado de 0.45 en v3g para eliminar detecciones dobles de la estructura metálica.                                                                                                                                              |
| DEAD_ZONE_DEG       | 6.0     | <b>v3h NUEVO.</b> Zona muerta en grados para los indicadores visuales. Dentro de este rango los badges muestran NIVEL/QUIETO. Ajustado al ruido medido de $\pm 5\text{--}7^\circ$ en reposo. Configurable con --dead-zone.                     |
| ROLL_OFFSET         | -90.0   | Compensa el arctan2. Calibrar en posición de reposo.                                                                                                                                                                                           |
| PITCH_OFFSET        | -90.0   | Confirmado correcto en pruebas v3e (01/06/2026). Bug 5 eliminó el $-90^\circ$ incorrecto en el cálculo del vector de pitch. El PITCH_OFFSET = -90.0 activo en v3h es un offset de calibración de posición de reposo, independiente y correcto. |
| ANGLE_SMOOTH_ALPHA  | 0.3     | Factor EMA de suavizado angular (0=sin suavizado, 1=instantáneo). Aplica a Roll y Pitch.                                                                                                                                                       |
| DELTA_T_S           | 2.0     | Ventana del delta de ángulos en segundos. Configurable con --delta-t.                                                                                                                                                                          |
| KP_SCALE            | 8.0     | Factor de escala de keypoints.                                                                                                                                                                                                                 |
| ROLL_AXIS_RATIO     | 0.5     | Posición del eje Roll entre KP5–6 (0) y KP7–8 (1). Medir físicamente.                                                                                                                                                                          |
| WINDOW_SCALE_FACTOR | 2       | Escala de la ventana OpenCV. $\times 2 \rightarrow 1920 \times 1080$ en pantalla FullHD.                                                                                                                                                       |
| FLIP_PITCH/FLIP_YAW | False   | Invierte el vector del eje. Usar si el giroscopio está montado girado $180^\circ$ en Z.                                                                                                                                                        |

### 5.4 Panel visual v3h

Desde v3h el panel lateral es completamente visual (tres indicadores con arcos OpenCV) en lugar de texto de ángulos. Este cambio responde a la conclusión técnica del proyecto: los números absolutos tienen un ruido inherente de  $\pm 5\text{--}7^\circ$  que hace falsa su precisión.





- **Pitch — arco semicircular:** El punto se mueve a lo largo del arco. Centro = nivel. Derecha = adelante. Izquierda = atrás.
- **Roll — péndulo:** La aguja gira alrededor del eje central como un nivel de burbuja. Intuitivo sin leer grados.
- **Yaw — brújula de delta:** Siempre muestra el delta (no el absoluto). Etiquetado YAW(N/D). No fiable con cámara lateral única.
- **Badges semánticos:** ADELANTE / ATRÁS / NIVEL — DER / IZQ / NIVELADO — HORARIO / ANTIHOR / N/D con fondo de color.
- **Números:** Se mantienen pequeños bajo cada badge para quien los necesite.

#### Optimización de rendimiento v3h rev.2 — cv2.addWeighted sobre ROI

La primera versión de v3h usaba `frame.copy()` completo ( $960 \times 540 \times 3 = 1.56\text{M px}$ )  $\times 4$  por frame, lo que bajó el FPS de 24.5 a 15.0. La solución: usar vistas NumPy (`frame[y0:y1, x0:x1]`) para operar solo sobre el ROI de cada badge y del fondo del panel. Resultado: de  $\sim 6.2\text{M px}$  a  $\sim 343\text{K px}$  por frame ( $\sim 94\%$ ). FPS recuperados: 15.0  $\rightarrow$  21.6 FPS.

## 6. Historia completa de bugs y soluciones

A lo largo del proyecto se diagnosticaron y resolvieron 17 bugs. Los bugs más difíciles no son los de programación, sino los que requieren razonamiento desde primeros principios sobre el sistema completo. Los bugs 1–12 surgieron durante las fases de integración del pipeline y compilación del HEF. Los bugs 13–17 aparecieron durante el proceso de calibración de ángulos con los modelos p211 y p309 (sesión 01/06/2026).

### Bug 1 — Tensores de keypoints pedidos como UINT8 [15/04/2026]

|          |                                                                                                                                                    |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | Cero detecciones. HailoRT abortaba el hilo de inferencia silenciosamente.                                                                          |
| Causa    | El HEF compila los tensores de keypoints (conv45/59/72) como UINT16. El script los pedía como UINT8. HailoRT aborta el hilo sin excepción visible. |
| Solución | Pedir conv45/59/72 como FormatType.UINT16.                                                                                                         |

### Bug 2 — Sigmoid aplicado a confianza en formato PROB [15/04/2026]

|          |                                                                                                                   |
|----------|-------------------------------------------------------------------------------------------------------------------|
| Síntoma  | ~1500 falsas detecciones por frame.                                                                               |
| Causa    | conv44/58/71 ya devuelven probabilidades [0,1]. Aplicar sigmoid da $\text{sigmoid}(0)=0.5$ para todas las celdas. |
| Solución | Detectar tensores PROB por su zero_point ( $zp < 100$ ) y aplicar <code>np.clip(raw, 0, 1)</code> sin sigmoid.    |

### Bug 3 — Dataset de calibración HEF = dataset de entrenamiento [15–16/04/2026]

|          |                                                                                                                                                              |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | <code>raw_conf=0</code> en TODOS los frames. Cero detecciones. Test de ruido: <code>conv71:max=0</code> con imagen aleatoria.                                |
| Causa    | Las imágenes de calibración eran las mismas que las de entrenamiento. El rango de cuantización queda demasiado ajustado → en producción <code>raw=0</code> . |
| Solución | Calibración cruzada: compilar cada HEF con imágenes de calibración de los OTROS grupos/modelos.                                                              |

#### Bug 3 — Regla de oro de la calibración Hailo

*Las imágenes de calibración NUNCA deben ser las mismas que las de entrenamiento. Esta restricción no está documentada en el Hailo DFC. Solo el test de ruido (`conv71:max>0` con imagen aleatoria) lo revela.*

### Bug 4 — KP\_SCALE=10.0 proyecta keypoints fuera de rango [16/04/2026]

|          |                                                                                                                   |
|----------|-------------------------------------------------------------------------------------------------------------------|
| Síntoma  | KP1 y KP2 no aparecen en pantalla.                                                                                |
| Causa    | Con <code>KP_SCALE=10</code> y <code>stride=32</code> , las celdas del borde generan coordenadas fuera del frame. |
| Solución | Reducir <code>KP_SCALE</code> a 4.0 y añadir clamp [0, 639]. Posteriormente ajustado a 8.0.                       |

### Bug 5 — Pitch con offset de -90° incorrecto en el cálculo del vector [16/04/2026]

|          |                                                                                                                                                                                 |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | El ángulo de Pitch siempre tiene un error sistemático de 90°.                                                                                                                   |
| Causa    | El cálculo incluía una resta de 90° heredada de una versión anterior del script, sin justificación geométrica en el nuevo esquema de 12 KP.                                     |
| Solución | Eliminar el -90° del cálculo del vector. El vector $\text{mid}(\text{KP1}, \text{KP2}) \rightarrow \text{mid}(\text{KP3}, \text{KP4})$ representa el eje de pitch directamente. |

**Nota: Bug 5 vs PITCH\_OFFSET**

*Bug 5 eliminó el -90° del cálculo del vector de pitch. El PITCH\_OFFSET = -90.0 activo en v3h es*

*un offset de calibración de posición de reposo, completamente independiente y correcto (confirmado*

*en pruebas v3e: el giroscopio en reposo muestra +2 a +3°, dentro del margen aceptable).*

**Bug 6 — Lógica de Roll con ROLL\_AXIS\_RATIO incorrecta** [16/04/2026]

|          |                                                                                                                                                                                                                             |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | El ángulo de Roll es incorrecto sistemáticamente.                                                                                                                                                                           |
| Causa    | El cálculo usaba solo KP5 y KP7 ignorando KP6 y KP8. El eje real de rotación no coincide con las puntas de la U.                                                                                                            |
| Solución | Redefinir: $\text{eje\_izq} = \text{KP5} * (1 - R) + \text{KP6} * R$ , $\text{eje\_der} = \text{KP7} * (1 - R) + \text{KP8} * R$ . Añadir $\text{ROLL\_OFFSET} = -90.0$ . (Mejorado posteriormente en v3d — ver Bug 14/17.) |

**Bug 7 — Fallbacks de Pitch insuficientes** [16/04/2026]

|          |                                                                                     |
|----------|-------------------------------------------------------------------------------------|
| Síntoma  | El cálculo de Pitch falla cuando KP1 o KP2 no son visibles.                         |
| Causa    | El código solo intentaba un único par de puntos medios.                             |
| Solución | Añadir fallbacks para todas las combinaciones disponibles de pares trasero-frontal. |

**Bug 8 — Color RGB/BGR incorrecto en grabación de vídeo** [~Mar 2026]

|          |                                                                                                                                                                          |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | El vídeo grabado muestra colores incorrectos en el PC (piel azulada).                                                                                                    |
| Causa    | cv2.VideoWriter escribe en BGR. Picamera2 entrega RGB888. Los canales se intercambian.                                                                                   |
| Solución | Pasar el frame RGB directamente sin conversión a <code>writer.write()</code> . Para corregir vídeos existentes: <code>ffmpeg</code> con <code>colorchannelmixer</code> . |

**Bug 9 — Conversión RGB→BGR innecesaria antes de inferencia** [20/05/2026]

|          |                                                                                                                                                 |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | Con el nuevo modelo del profesor la confianza máxima es 0.012–0.027.                                                                            |
| Causa    | El nuevo modelo fue entrenado con imágenes RGB (Roboflow/Colab). El script hacía <code>cvtColor RGB→BGR</code> antes de pasar el frame al chip. |
| Solución | Añadir flag <code>CONVERT_TO_BGR=False</code> . La conversión queda disponible con <code>--bgr</code> para modelos de alumnos.                  |

**Bug 10 — Dataset con encuadre demasiado cercano** [21/05/2026]

|                 |                                                                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Síntoma</b>  | KP9–12 (base) sistemáticamente ausentes. Roll≈−105°, Pitch≈−167°. El problema afecta a todos los HEFs del modelo del profesor.                                       |
| <b>Causa</b>    | El vídeo de entrenamiento tenía el giroscopio demasiado cerca. La base (KP9–12) quedaba fuera del encuadre. El modelo aprendió que esos KPs tienen visibilidad baja. |
| <b>Solución</b> | Reentrenamiento completo con nuevo dataset. El giroscopio debe ocupar el 40–70% del frame con la base siempre visible.                                               |

**Bug 11 — Arranque de cámara bloqueado por hilo de inferencia** [20/05/2026]

|                 |                                                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Síntoma</b>  | La ventana se abría pero la cámara nunca iniciaba.                                                                      |
| <b>Causa</b>    | El hilo de inferencia arrancaba antes que la cámara y ejecutaba verificar_hef() (~150ms en PCIe), bloqueando Picamera2. |
| <b>Solución</b> | Inicializar la cámara ANTES de arrancar el hilo. Añadir time.sleep(1.5).                                                |

**Bug 12 — Ventana demasiado pequeña en pantalla 4K** [20/05/2026]

|                 |                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Síntoma</b>  | La ventana OpenCV no supera el tamaño nativo de la fuente (960×540).                                                      |
| <b>Causa</b>    | OpenCV limita el redimensionado interactivo al tamaño nativo del contenido.                                               |
| <b>Solución</b> | Llamar a cv2.resizeWindow() con WINDOW_SCALE_FACTOR × resolución fuente (×2 → 1920×1080 en FullHD, ×3 → 2880×1620 en 4K). |

**Bug 13 — Símbolos no ASCII en el panel (°, ↑, ↓, Δ)** [01/06/2026]

|                 |                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| <b>Síntoma</b>  | Aparecen caracteres '?' en el panel visual en lugar de los símbolos esperados.                                |
| <b>Causa</b>    | cv2.putText con FONT_HERSHEY_SIMPLEX solo admite ASCII. Los caracteres Unicode (°, ↑, ↓, Δ) no se renderizan. |
| <b>Solución</b> | Sustituidos por equivalentes ASCII puros: 'deg', '+', '-', 'Delta'.                                           |

**Bug 14 — ROLL\_OFFSET no calibrado para el nuevo modelo** [01/06/2026]

|                 |                                                                                                                                                                                                                                 |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Síntoma</b>  | Con ROLL_OFFSET=+90 (v3d): pantalla muestra −175° en reposo. Con ROLL_OFFSET=+175 (v3e): pantalla muestra −90° en reposo.                                                                                                       |
| <b>Causa</b>    | El raw del modelo es consistentemente ~+95° (no −90° como se asumía). El vector KP5→KP7 apunta hacia ARRIBA en imagen porque el dataset se grabó por detrás del giroscopio — misma inversión vertical que los KPs del octógono. |
| <b>Solución</b> | ROLL_OFFSET = −90.0 → fix_angle(+95, −90) ≈ 0° en reposo ✓ (v3f, 01/06/2026). Ver también Bug 17.                                                                                                                               |

**Bug 15 — PITCH\_OFFSET no calibrado para el nuevo modelo** [01/06/2026]

|                |                                                                                |
|----------------|--------------------------------------------------------------------------------|
| <b>Síntoma</b> | Pitch con offset incorrecto heredado de versión anterior al cambiar de modelo. |
|----------------|--------------------------------------------------------------------------------|



|          |                                                                                                                                  |
|----------|----------------------------------------------------------------------------------------------------------------------------------|
| Causa    | Offset no recalibrado al entrenar con el nuevo dataset.                                                                          |
| Solución | PITCH_OFFSET = -90.0. Confirmado correcto en pruebas v3e: el giroscopio en reposo muestra +2 a +3°, dentro del margen aceptable. |

**Bug 16 — EMA produce valores fuera de [-180, 180] en wrap-around** [01/06/2026]

|          |                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | En casos límite (ángulos cercanos a $\pm 180^\circ$ ), la EMA acumula error y produce valores como $-266.8^\circ$ .                                                               |
| Causa    | La interpolación EMA no renormalizaba el resultado tras la suma. <code>fix_angle</code> ya había normalizado la entrada, pero la acumulación del error en el EMA salía del rango. |
| Solución | Añadir re-normalización final: <code>result = ((result + 180) % 360) - 180</code> .                                                                                               |

**Bug 17 — ROLL\_OFFSET=+175 da -90° en reposo en lugar de 0°** [01/06/2026]

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Síntoma  | Con ROLL_OFFSET=+175 (intento en v3e): <code>fix_angle(+95, +175) <math>\approx -90^\circ</math></code> en reposo. No es $0^\circ$ .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Causa    | <p>El raw del modelo es <math>\sim +95^\circ</math> (no <math>-90^\circ</math>). Análisis de las pruebas v3e:</p> <ul style="list-style-type: none"><li>· Con ROLL_OFFSET=+90: pantalla = <math>-175^\circ</math> en reposo <math>\rightarrow</math> raw <math>\approx +95^\circ</math></li><li>· Con ROLL_OFFSET=+175: pantalla = <math>-90^\circ</math> en reposo <math>\rightarrow</math> raw <math>\approx +95^\circ</math></li></ul> <p>Causa raíz: el vector KP5<math>\rightarrow</math>KP7 apunta hacia ARRIBA (KP5 con Y mayor que KP7 en píxeles) por la perspectiva de grabación por detrás. <code>arctan2(+dy, 0) <math>\approx +90^\circ</math></code> <math>\rightarrow</math> raw <math>\approx +95^\circ</math>.</p> |
| Solución | ROLL_OFFSET = -90.0 $\rightarrow$ <code>fix_angle(+95, -90) <math>\approx 0^\circ</math></code> en reposo ✓ Con nuevo dataset grabado de frente el raw será $\sim -90^\circ$ y el offset correcto volverá a ser +90.0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

## 7. Dataset y estrategia de entrenamiento

El dataset es la piedra angular del sistema. Un modelo técnicamente correcto puede fallar completamente si el dataset no representa bien la realidad de uso.

### 7.1 Criterios del dataset

- **Variedad de fondo — el factor más importante.** Si todas las imágenes tienen el mismo fondo, el modelo sobreajusta y genera detecciones falsas cuando cambia el entorno. Grabar en al menos 2–3 ubicaciones distintas.
- **Distancia correcta y variada.** El giroscopio debe ocupar entre el 40–70% del frame, con la base (KP9–12) siempre completamente visible. El Bug 10 fue causado por grabar demasiado cerca.
- **Misma cámara que en inferencia.** Usar la RPi Camera Module v2 (IMX219, RGB888) tanto para el dataset como para la inferencia.
- **Variedad de orientaciones.** Roll  $\pm 30^\circ$ , Pitch  $\pm 30^\circ$ , Yaw  $\pm 45^\circ$ . Salir claramente de la posición de reposo.
- **Intervalo entre frames: 1–2 segundos.** Con 2s hay más variedad real con el mismo número de imágenes.
- **Etiquetado coherente.** KPs ocultos marcados como 'occluded' en Roboflow, nunca omitidos ni estimados.
- **Sin flips como augmentación.** El flip puede invertir el orden de KPs simétricos generando anotaciones incorrectas.

### 7.2 Script de grabación del dataset (grabar\_dataset\_v2.py)

```
200 imágenes totales (100 train + 100 calib), 1 frame cada 1.5s:
DISPLAY=:0 python3 grabar_dataset_v2.py --num-imagenes 200 --intervalo 1.5

Con directorio de salida personalizado:
DISPLAY=:0 python3 grabar_dataset_v2.py --num-imagenes 200 --salida
/home/ai/dataset_escena2

Solo extraer de un vídeo ya grabado:
python3 grabar_dataset_v2.py --solo-extraer /home/ai/mi_video.mp4 --num-imagenes 200
```

#### Estrategia recomendada

3 vídeos cortos (4–5 min) con fondos distintos, --intervalo 2.0. ~100 frames de entrenamiento + ~100 de calibración por vídeo. Resultado: 300+300 imágenes con variedad real. Preferible a un único vídeo largo con el mismo fondo.

### 7.3 Evolución del dataset del profesor (modelos p113–p410)

| Modelo | Imágenes raw | Train (aug) | Valid | Test | Augmentaciones   | Estado                     |
|--------|--------------|-------------|-------|------|------------------|----------------------------|
| p113   | 113          | ~300        | —     | —    | Roboflow básicas | HEF compilado ✓, Probado ✓ |
| p211   | 211          | ~540        | —     | —    | Roboflow básicas | HEF compilado ✓, Probado ✓ |
| p309   | 309          | ~740        | —     | —    | Roboflow básicas | HEF compilado ✓, Probado ✓ |

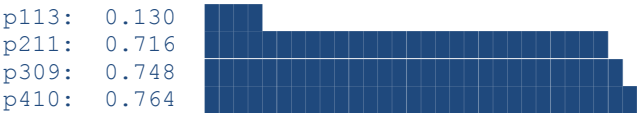
| Modelo | Imágenes raw | Train (aug) | Valid | Test | Augmentaciones                              | Estado                     |
|--------|--------------|-------------|-------|------|---------------------------------------------|----------------------------|
| p410   | 410          | 984         | 41    | 41   | Brillo $\pm 15\%$ ,<br>Blur 1px, Ruido 15px | HEF compilado ✓, Probado ✓ |

## 8. Comparativa de modelos — p113 / p211 / p309 / p410

### 8.1 Métricas comparativas

| Métrica                     | p113  | p211  | p309  | p410  |
|-----------------------------|-------|-------|-------|-------|
| mAP50(P) — mejor            | 0.771 | 0.995 | 0.995 | 0.995 |
| Época del mejor valor       | 116   | 77    | 92    | 38    |
| mAP50-95(P) — mejor         | 0.130 | 0.716 | 0.748 | 0.764 |
| Época del mejor valor       | 119   | 143   | 150   | 99    |
| mAP50-95(B)                 | 0.930 | 0.968 | 0.944 | 0.952 |
| 1ª época mAP50P ≥ 0.90      | N/A   | 38    | 39    | 27    |
| val/pose_loss final         | 4.64  | 1.15  | 1.00  | 0.89  |
| Rango mAP50P últ. 20 épocas | 0.051 | 0.072 | 0.024 | 0.042 |

### 8.2 Progresión del mAP50-95(P)



Mejora p113→p211: +0.586 (salto crítico — p113 no era funcional)  
Mejora p211→p309: +0.031  
Mejora p309→p410: +0.017 (curva en fase de aplanamiento)  
Mejora total: +0.634 (+487%)

### 8.3 Análisis por modelo

#### p113 — No apto para producción:

El mAP50(P) nunca supera 0.90. val/pose\_loss final de 4.64 vs ~1.0 del resto: el modelo no convergió.

#### p211 — Umbral mínimo funcional:

Salto cualitativo enorme. mAP50(P)=0.995, mAP50-95(P)=0.716. Alta varianza en últimas épocas.

#### p309 — Sólido y más estable:

mAP50-95(P)=0.748. Rango últimas 20 épocas=0.024, el mejor de los cuatro.

#### p410 — Mejor modelo:

mAP50-95(P)=0.764, val/pose\_loss=0.895. Converge más rápido (mAP50P≥0.90 en época 27).

#### Ley de rendimientos decrecientes en el dataset

El salto crítico está entre 113 y 211 imágenes (+0.586). Entre 211 y 410 la mejora acumulada es +0.048. Para ganancias adicionales significativas, la variedad de escenas y fondos aporta más que añadir imágenes del mismo entorno.

## 9. Compilación del modelo para Hailo-8

La compilación convierte el modelo .pt exportado por Ultralytics al formato .hef que el chip Hailo-8 puede ejecutar directamente. Requiere el Hailo DFC únicamente disponible como imagen Docker.

### 9.1 Entorno de compilación

- **PC Ubuntu o Windows** (i7 o superior; sin GPU).
- **Imagen Docker:** `hailo8_ai_sw_suite_2025-10:1` (~17GB). Incluye el DFC 3.33.0.
- **Tiempo:** ~22 min en i7-9700 (Parse: 1min, Optimize: 13–14min, Compile: 8–9min).

### 9.2 Proceso de compilación

```
Dentro del contenedor Docker:
cd ~

PASO 1: Convertir imágenes de calibración a .npy
python3 << 'EOF'
import os, numpy as np
from PIL import Image
calib_dir='/workspace/calib/'; out_dir=os.path.expanduser('~/.calib_npy/')
os.makedirs(out_dir, exist_ok=True)
files=[f for f in os.listdir(calib_dir) if f.lower().endswith(('.jpg','.png'))]
for i,f in enumerate(files):
 img=Image.open(os.path.join(calib_dir,f)).convert('RGB').resize((640,640))

np.save(os.path.join(out_dir,f'calib_{i:04d}.npy'),np.array(img,dtype=np.float32))
EOF

PASO 2: Parse ONNX → HAR
hailo parser onnx /workspace/giroscopio.onnx --net-name giroscopio \
 --end-node-names /model.22/cv2.0/cv2.0.2/Conv /model.22/cv3.0/cv3.0.2/Conv \
 /model.22/cv4.0/cv4.0.2/Conv /model.22/cv2.1/cv2.1.2/Conv \
 /model.22/cv3.1/cv3.1.2/Conv /model.22/cv4.1/cv4.1.2/Conv \
 /model.22/cv2.2/cv2.2.2/Conv /model.22/cv3.2/cv3.2.2/Conv \
 /model.22/cv4.2/cv4.2.2/Conv

PASO 3: Optimize (~13 min)
hailo optimize ~/giroscopio.har --hw-arch hailo8 \
 --calib-set-path ~/.calib_npy/ --model-script ~/giroscopio.alls

PASO 4: Compile (~8 min)
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8

PASO 5: Guardar antes de salir
cp ~/giroscopio.hef /workspace/
ls -lh /workspace/*.hef # verificar antes de hacer exit
```

#### **NO hacer exit hasta ver los archivos en /workspace**

Con `--rm` los archivos de `~` desaparecen al salir. Verificar siempre con `ls /workspace/*.hef` antes de `exit`.

### 9.3 Verificación post-compilación

#### **Test de ruido — obligatorio**

Ejecutar el script con `--verbose` y una imagen de ruido. Si `conv71:max=0` → HEF mal calibrado (Bug 3). Si `conv71:max>0` → compilación correcta.



## 10. Estado actual del proyecto

### 10.1 Scripts disponibles

| Script                       | Estado          | Descripción                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| giroscopio_12kp_v3h.py       | <b>ACTUAL</b>   | Script principal de inferencia v3h (03/06/2026 rev.2). Panel visual con arcos OpenCV (Pitch=arco, Roll=péndulo, Yaw=brújula delta). DEAD_ZONE_DEG=6°. Optimización ROI (~94% reducción en addWeighted). FPS display ~21-22 FPS. Flags: --dead-zone, --iou-thresh, --kp-thresh, --hide-yaw, --flip-pitch, --flip-yaw, --delta-t, --verbose. Actualizado 08/06/2026 (nuevos args --log-dir y --save-auto, setup_logging extrae modelo del directorio padre del HEF) |
| grabar_dataset_v2.py         | <b>ACTUAL</b>   | Grabación y extracción de frames. --num-imagenes (total auto-split 50/50), OSD en tiempo real, reparto equitativo.                                                                                                                                                                                                                                                                                                                                                |
| giroscopio_colab_v4.ipynb    | <b>ACTUAL</b>   | Notebook de entrenamiento Google Colab. YOLOv8n-Pose, 150 épocas, exportación ONNX.                                                                                                                                                                                                                                                                                                                                                                               |
| giroscopio_ensemble_v3.py    | Funcional       | Ensemble de 3 modelos (alumnos). Bug 9 puede estar presente — revisar antes de usar.                                                                                                                                                                                                                                                                                                                                                                              |
| giroscopio_12kp_v3a.py (=v6) | <b>Obsoleto</b> | Bugs 9, 11, 12 presentes.                                                                                                                                                                                                                                                                                                                                                                                                                                         |

### 10.2 HEFs disponibles en la Raspberry Pi

| HEF                        | Test ruido    | Estado                        | Observaciones                                                                            |
|----------------------------|---------------|-------------------------------|------------------------------------------------------------------------------------------|
| giroscopio_g1_v2.hef       | ✓             | Probado                       | Modelo alumnos G1. Calibración cruzada. Usar con --bgr.                                  |
| giroscopio_g2_v2.hef       | ✓             | Probado                       | Modelo alumnos G2. Calibración cruzada. Usar con --bgr.                                  |
| giroscopio_g3_v2.hef       | ✓             | Probado                       | Modelo alumnos G3. Calibración cruzada. Usar con --bgr.                                  |
| resultado_profe_calib1.hef | ✓<br>(max=7)  | Probado                       | <b>Bug 10</b> presente. Base no detectada.                                               |
| resultado_profe_calib2.hef | ✓<br>(max=24) | Probado                       | <b>Bug 10</b> + conf baja.                                                               |
| resultado_profe_calib3.hef | ✓<br>(max=11) | Probado                       | <b>Bug 10</b> presente.                                                                  |
| <b>p309.hef</b>            | ✓             | <b>Compilado ✓, Probado ✓</b> | Modelo p309 (309 imágenes). Pruebas físicas 03/06/2026.                                  |
| <b>p410.hef</b>            | ✓             | <b>Compilado ✓, Probado ✓</b> | <b>Mejor modelo.</b> Conf media 0.888, 99%+ frames >0.5, 0 pérdidas. Pruebas 03/06/2026. |

## 10.3 Próximos pasos

Todo esto son opciones, no quiere decir que realmente se vayan a implementar.

- **Segunda cámara (frontal o superior):** Resolver la ambigüedad geométrica de Roll y Yaw. El Hailo-8 puede ejecutar dos pipelines en paralelo.
- **Fusión con IMU (MPU-6050 ~3€):** Sensor de Roll y Yaw precisos a bajo coste. Complementa el Pitch óptico de la cámara.
- **Interfaz web (v4 propuesta):** Python envía datos por WebSocket al navegador. Panel HTML/Canvas visualmente rico sin coste de FPS en el pipeline de inferencia. Flags como controles deslizantes.
- **Revisión del material publicado en GitHub para tener en cuenta la última actualización:** Material educativo abierto con todos los scripts, tutoriales y documentación.

## 11. Historial de sesiones

Esta sección recoge la cronología completa del proyecto desde su inicio. Se incluyen todos los hitos técnicos, diagnósticos de bugs, decisiones de diseño y publicaciones. Los bugs se presentan como descubrimientos del proceso, sin atribución individual, conforme a la metodología pedagógica del proyecto.

| Fecha                 | Hito                                                                            | Detalle técnico                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nov 2025              | Adquisición de hardware y materiales                                            | Pack Raspberry Pi 5 (8GB) + AI HAT+ (Hailo-8) + cámara v2 (RaspiPC). Monitor MSI 24,5" 120 Hz (PcComponentes). Teclado Logitech K400 (MediaMarkt). Perfilera de aluminio T-Slot 4040/4080/2040 y rodamientos (Arialpro + AliExpress). Presupuesto ejecutado: 684,86 EUR de 800 EUR concedidos.                                                                                                         |
| Dic 2025 - Feb 2026   | Fase 1: prototipo 8KP con giroscopio de aluminio                                | Giroscopio fabricado en el departamento con perfilera de aluminio estructural T-Slot. Modelo YOLOv8n-Pose con 8 keypoints. Solo el profesor. Valida la cadena técnica completa (Roboflow -> Colab -> Hailo DFC -> HEF -> RPi).                                                                                                                                                                         |
| Mar 2026              | Inicio con alumnos. 3 grupos graban datasets y entrenan modelos en Colab (12KP) | 3 grupos de ~4 alumnos. Cada alumno etiqueta >= 50 imágenes en Roboflow. Giroscopio avanzado de 3 ejes (octogono + soporte U + base cuadrada) con 12 marcadores de color impresos. Modelos g1, g2, g3 compilados y probados en RPi.                                                                                                                                                                    |
| 15/04/2026            | Bugs 1, 2, 3 diagnosticados. Primera detección funcional                        | Bug 1 (UINT16): tensores keypoints pedidos como UINT8 -> 0 detecciones -> fix: FormatType.UINT16. Bug 2 (sigmoid): conv44/58/71 ya en formato PROB [0,1] -> 1500 falsas detecciones/frame -> fix: np.clip sin sigmoid. Bug 3 (calibración cruzada): calibrar con el mismo dataset de entrenamiento -> raw_conf=0 en producción -> fix: usar imágenes de OTRO grupo para calibración. Regla permanente. |
| 16/04/2026            | Recompilación con calibración cruzada. Bugs 4-7 resueltos. Script v6 operativo  | Bug 4 (KP_SCALE=10.0): KP1 y KP2 fuera de rango -> fix: 4.0 con clamp, luego 8.0. Bug 5 (offset -90deg en calculo vector Pitch): residuo sin justificación geométrica -> fix: eliminar. Bug 6 (Roll solo KP5/KP7): fix mid(KP5,KP6)->mid(KP7,KP8). Bug 7 (fallbacks Pitch insuficientes): fix: todas las combinaciones par trasero-frontal. Script v6 = primera versión estable.                       |
| 16/04/2026            | Ensemble v3 operativo: 94% de frames con los 3 modelos detectando               | giroscopio_ensemble_v3.py con modelos g1/g2/g3 en paralelo. Validación de robustez del sistema con modelos de alumnos.                                                                                                                                                                                                                                                                                 |
| ~May 2026             | Modelo 220 imágenes (encuadre demasiado cercano) - descartado                   | Dataset grabado con el giroscopio ocupando más del 80% del frame. KP9-12 (base) invisibles sistemáticamente. Diagnóstico posterior Bug 10. Modelo descartado; se define estrategia de reentrenamiento con encuadre al 40-70%.                                                                                                                                                                          |
| ~Mar 2026 (detectado) | Bug 8: color RGB/BGR incorrecto en grabación de video                           | Video grabado muestra tonos azulados al reproducir en PC. Fix: pasar frame RGB directamente a writer.write(). Corrección de videos existentes: ffmpeg colorchannelmixer.                                                                                                                                                                                                                               |

| Fecha         | Hito                                                                                               | Detalle técnico                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 20/05/2026    | Bug 9 diagnosticado (RGB->BGR innecesario). Script v3b                                             | conf_max=0.012-0.027 con modelo del profesor. Causa: el script convertía RGB->BGR antes de la inferencia, pero el modelo fue entrenado con imágenes RGB (Roboflow las sirve en RGB). Fix: CONVERT_TO_BGR=False. Flag --bgr disponible para modelos BGR.                                                                                                                                                                                                                      |
| 20/05/2026    | Bugs 11 y 12 resueltos en v3b                                                                      | Bug 11 (arranque cámara bloqueado por hilo inferencia): fix inicializar cámara antes del hilo + time.sleep(1.5). Bug 12 (ventana pequeña en 4K): fix cv2.resizeWindow con WINDOW_SCALE_FACTOR.                                                                                                                                                                                                                                                                               |
| 21/05/2026    | Pruebas v3b. Bug 10 diagnosticado. Estrategia de reentrenamiento definida                          | KP9-12 ausentes sistemáticamente. Roll ~ -105deg, Pitch ~ -167deg. Dobles detecciones en borde superior. Causa: encuadre demasiado cercano en dataset -> base fuera del frame -> el modelo aprende que son invisibles. Solución: reentrenamiento con giroscopio al 40-70% del frame, base siempre visible.                                                                                                                                                                   |
| 21/05/2026    | CONTEXTO v2 y grabar_dataset_v1.py generados                                                       | Documentación de la estrategia de dataset. Script de grabación y extracción de frames.                                                                                                                                                                                                                                                                                                                                                                                       |
| ~May-Jun 2026 | Etiquetado progresivo: modelos p113, p211, p309                                                    | El profesor graba nuevo dataset con encuadre correcto. Etiquetado incremental en Roboflow. p113 (113 imgs), p211 (211 imgs, primera version con aumentaciones), p309 (309 imgs). Aumentaciones permitidas: ruido, brillo, contraste. Flips PROHIBIDOS (invierten orden de KPs simetricos).                                                                                                                                                                                   |
| 01/06/2026    | Bugs 13-17 diagnosticados y resueltos (sesión de calibración de ángulos: v3d, v3e, v3f)            | Bug 13 (símbolos no ASCII en panel): cv2.putText solo admite ASCII; sustituidos deg/+/-/Delta. Bug 14 (ROLL_OFFSET no calibrado): raw del modelo ~+95deg (vector KP5->KP7 apunta arriba porque dataset grabado por detras) -> ROLL_OFFSET=-90.0. Bug 15 (PITCH_OFFSET): PITCH_OFFSET=-90.0 correcto, +2+3deg en reposo. Bug 16 (EMA fuera de [-180,180] en wrap-around): re-normalización final. Bug 17 (ROLL_OFFSET=+175 en v3e da -90deg en reposo): fix definitivo -90.0. |
| 02/06/2026    | p410 entrenado (410 imgs, 1066 total post-aumentacion). Comparativa de modelos. CONTEXTO v3        | mAP50-95(P)=0.764, val/pose_loss=0.895. Salto critico entre p113 y p211 (+0.586 mAP). Entre p211 y p410 mejora acumulada +0.048. Script v3f: ROLL_OFFSET=-90.0 definitivo.                                                                                                                                                                                                                                                                                                   |
| 02/06/2026    | Actualización de scripts: v3f, grabar_dataset_v2.py. Documento y anexos generados                  | Notebook Colab v4 corregido. Tutorial Hailo compilación v2. Script de grabación v2.                                                                                                                                                                                                                                                                                                                                                                                          |
| 03/06/2026    | Pruebas físicas p309 y p410. p410 confirmado mejor modelo                                          | Conf media p410: 0.888. Frames >0.5: 99%+. Pérdidas de detección: 0. p309 conf media: 0.636. El prefijo "p" = profesor, futura "a" = alumno.                                                                                                                                                                                                                                                                                                                                 |
| 03/06/2026    | v3g (ajustes post-pruebas físicas) -> v3h (panel visual con arcos) -> v3h rev.2 (optimización ROI) | v3g: IOU=0.30, CONF=0.05, KP=0.07. v3h: panel visual OpenCV con arcos semicirculares (Pitch), pendulo (Roll), brújula delta (Yaw), badges semánticos (ADELANTE/ATRÁS/NIVEL etc.). v3h rev.2: sustitución cv2.addWeighted por vistas NumPy ROI para recuperar FPS.                                                                                                                                                                                                            |
| 03/06/2026    | FPS: 24,5 (v3g) -> 15,0 (v3h rev.1) -> 21,6 (v3h rev.2). Sistema en estado de producción           | El panel visual de v3h rev.1 costaba 10 FPS al usar cv2.addWeighted sobre una copia del frame completo. Sustituir por ROI view de NumPy recupero ~7 FPS sin cambio visual. FPS mediana estable: 21,6 con modelo p410.                                                                                                                                                                                                                                                        |

| Fecha      | Hito                                                                                            | Detalle técnico                                                                                                                                                                                                                                                                                                                                                                                    |
|------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 08/06/2026 | Modelo a100 (alumno, 100 imgs): validación cruzada                                              | Un alumno etiqueto independientemente 100 imágenes del mismo video base que p113. Métricas de entrenamiento casi identicas a p113 (mAP50-95(P)=0.130, misma epoca del mejor valor: 116 vs 119). Conf media en RPi: 0.737 vs 0.888 de p410. Dobles detecciones: 18,7% (solución: --iou-thresh 0.20). Conclusion pedagógica: la limitacion es la variedad del dataset, no la calidad del etiquetado. |
| 08/06/2026 | Lanzadores de escritorio: bug Wayland/XAUTHORITY documentado y resuelto                         | Al lanzar desde el escritorio en RPi OS Bookworm (Wayland), el proceso no hereda XAUTHORITY. El servidor X11 rechaza ventanas silenciosamente (pantalla negra). Fix: env DISPLAY=:0 XAUTHORITY=/home/ai/.Xauthority en el archivo .desktop.                                                                                                                                                        |
| 08/06/2026 | v3h: añadidos --log-dir y --save-auto. setup_logging extrae modelo del directorio padre del HEF | Nomenclatura automática de logs y videos: logs_giroscopio_v3a_<modelo>_<YYYYMMDD_HHMMSS>.txt / giroscopio_v3a_<modelo>_<YYYYMMDD_HHMMSS>.mp4. Cambiar HEF_PATH en el wrapper .sh cambia automáticamente el prefijo de todos los archivos.                                                                                                                                                          |
| 08/06/2026 | Documento v7 generado (49 pag). Correcciones detectadas y documentadas. CONTEXTO v4             | Corrección critica: KP10=Rosa, KP11=Marron (v7 pag.20 los tenia invertidos). Nota: la fórmula ROLL_AXIS_RATIO es la de v3c, reemplazada en v3d por promedio circular de vectores por lado. CONTEXTO v4 es el documento de reanudación de sesion a partir de este punto.                                                                                                                            |
| 09/06/2026 | Documento v8. Seccion 18 nueva: guia de replicacion para docentes                               | Enlace completo con todos los anexos. Listado de software y hardware. Aclaración definitiva sobre la estructura de aluminio: no es impresión 3D; es perfilería T-Slot fabricada en el departamento. Comprobado experimentalmente que su peso excede la capacidad de motores de drones FPV comerciales.                                                                                             |

*El prefijo de los modelos HEF sigue la convención: "p" = profesor (p113, p211, p309, p410), "a" = alumno (a100). El número indica el total de imágenes raw del dataset antes de augmentacion.*



## 12. Conclusiones técnicas del proyecto

### 12.1 Por qué funciona Pitch y no Roll ni Yaw

Las pruebas físicas del 03/06/2026 con el modelo p410 y el script v3h confirmaron que la limitación angular no es un problema de dataset ni de código, sino una restricción geométrica fundamental de la medición monocular. Con una sola cámara lateral, la única rotación medible con fiabilidad es aquella cuyo eje es perpendicular a la línea de visión.

| Eje   | Estado      | Explicación geométrica                                                                                                                                                                                      |
|-------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PITCH | FIABLE      | Rotación alrededor del eje horizontal. Vista desde el lateral, el plano del octógono cambia de ángulo de forma directa y sin ambigüedad. El modelo lo detecta correctamente.                                |
| ROLL  | POCO FIABLE | Rotación alrededor del eje de profundidad. Desde el lateral, un roll puro es indistinguible de un cambio de posición en profundidad. Correlación Roll↔Pitch=0.39 confirma acoplamiento geométrico moderado. |
| YAW   | NO FUNCIONA | Rotación en el plano horizontal. Vista de lado se proyecta como compresión/expansión mínima del rectángulo de la base. Geométricamente imposible con una cámara lateral.                                    |

Adicionalmente, las pruebas midieron un ruido inherente de  $\pm 7^\circ$  (stdev Roll) y  $\pm 3^\circ$  (stdev Pitch) con el giroscopio en reposo. Esto hace que cualquier número absoluto de ángulo tenga una incertidumbre de  $\pm 5-7^\circ$ , lo que justifica el panel cualitativo de v3h.

### 12.2 Lo que el sistema sí consigue

Lejos de ser un fracaso, el sistema entrega exactamente lo que es posible con este hardware y esta arquitectura:

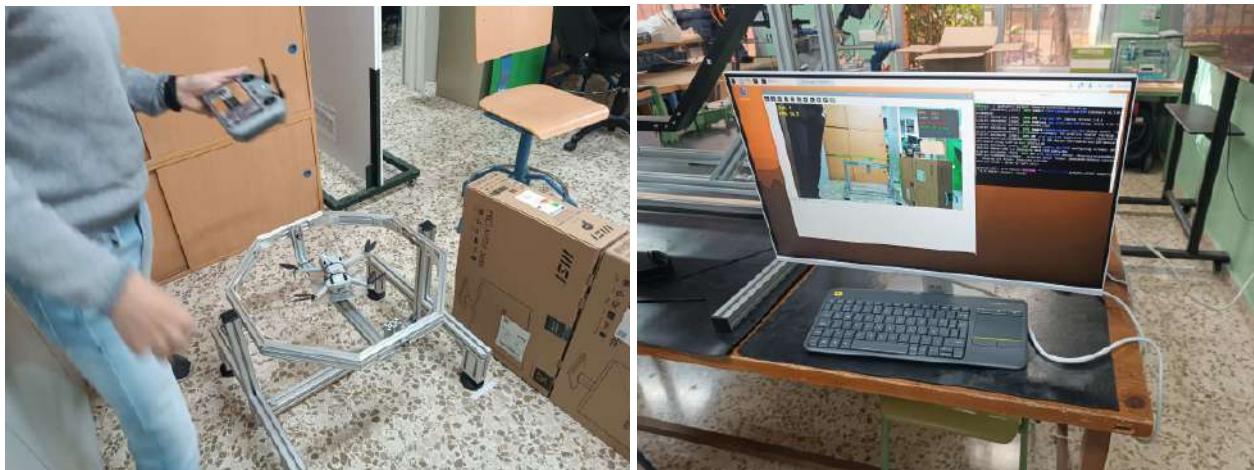
- Detección del objeto al 99%+ con confianza media  $>0.84$ , sin pérdidas de tracking, a  $\sim 22$  FPS en un chip de 5W sin conexión a la nube.
- Detección fiable de si el giroscopio está en movimiento o en reposo, y en qué sentido se mueve en el eje de pitch.
- Indicadores cualitativos de Roll y Yaw (dirección del movimiento) aunque sin precisión angular.
- Un pipeline completo y documentado de Edge AI desde el dataset hasta el chip neuronal, reproducible por alumnos de ciclos formativos.

### 12.3 Líneas de mejora futuras

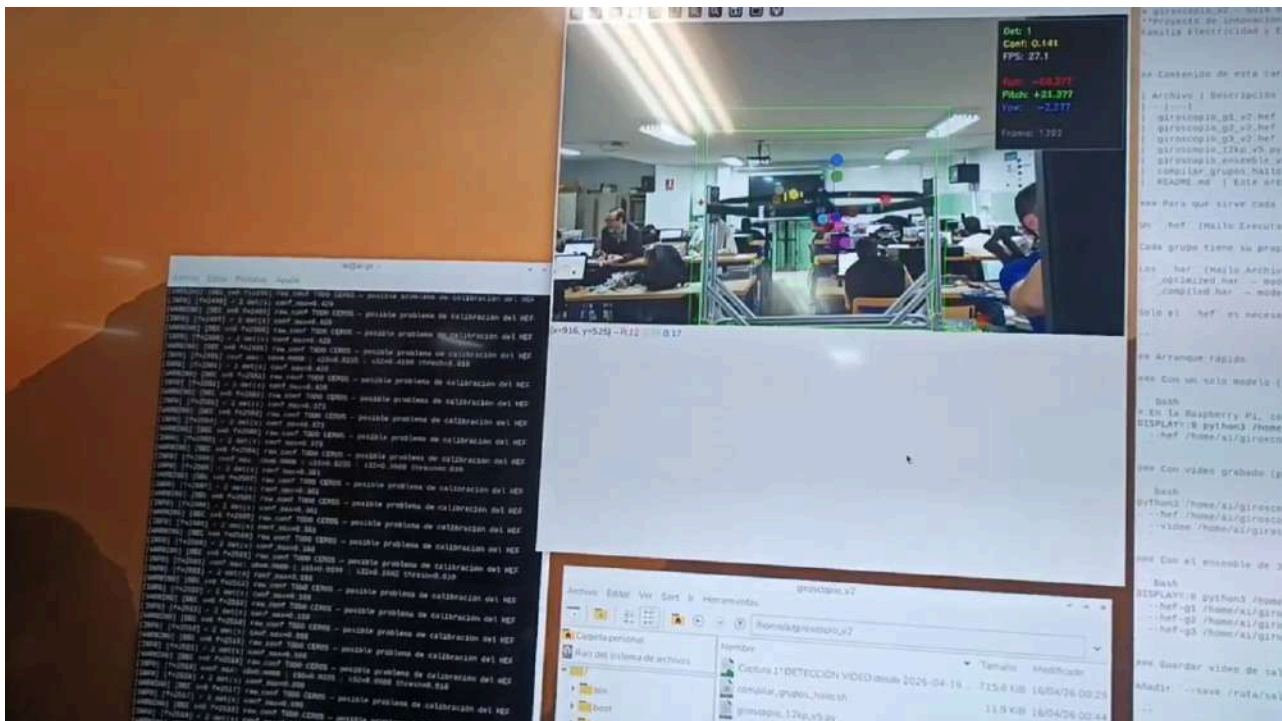
- **Segunda cámara frontal o superior:** resuelve la ambigüedad de Roll y Yaw. El Hailo-8 puede ejecutar dos pipelines en paralelo.
- **Fusión con IMU (MPU-6050 ~3€):** el sensor da Roll y Yaw precisos a muy bajo coste. La cámara complementa con la detección del objeto y el Pitch óptico.
- **Interfaz web (v4 propuesta):** Python envía datos por WebSocket a un navegador. Panel HTML/Canvas sin coste de FPS en el pipeline de inferencia. Flags como controles deslizantes.
- **Optimización FPS del panel visual (implementada en v3h rev.2):** cv2.addWeighted sobre ROI en lugar de frame completo. Recuperó 6.6 FPS ( $15 \rightarrow 21.6$  FPS).
- **Fabricación** de una versión en fibra de carbono (de la parte móvil).

## 13. Fases de fabricación física y evolución temporal

El proyecto arrancó por iniciativa del profesor con la fabricación de una estructura totalmente hecha en aluminio a partir de materiales de diseño propio usando perfiles de aluminio estructural y accesorios varios. Con eso y mucho arrojo, se planificó todo el procesado software, empezando por la grabación de un video, extracción de frames de ese video, etiquetado de imágenes, entrenamiento de un modelo de formato *Open Neural Network Exchange*, compilación para el acelerador Hailo 8 montado en la Raspberry PI 5 y creación de un script capaz de lanzar la inferencia e intentar obtener datos válidos de ahí.

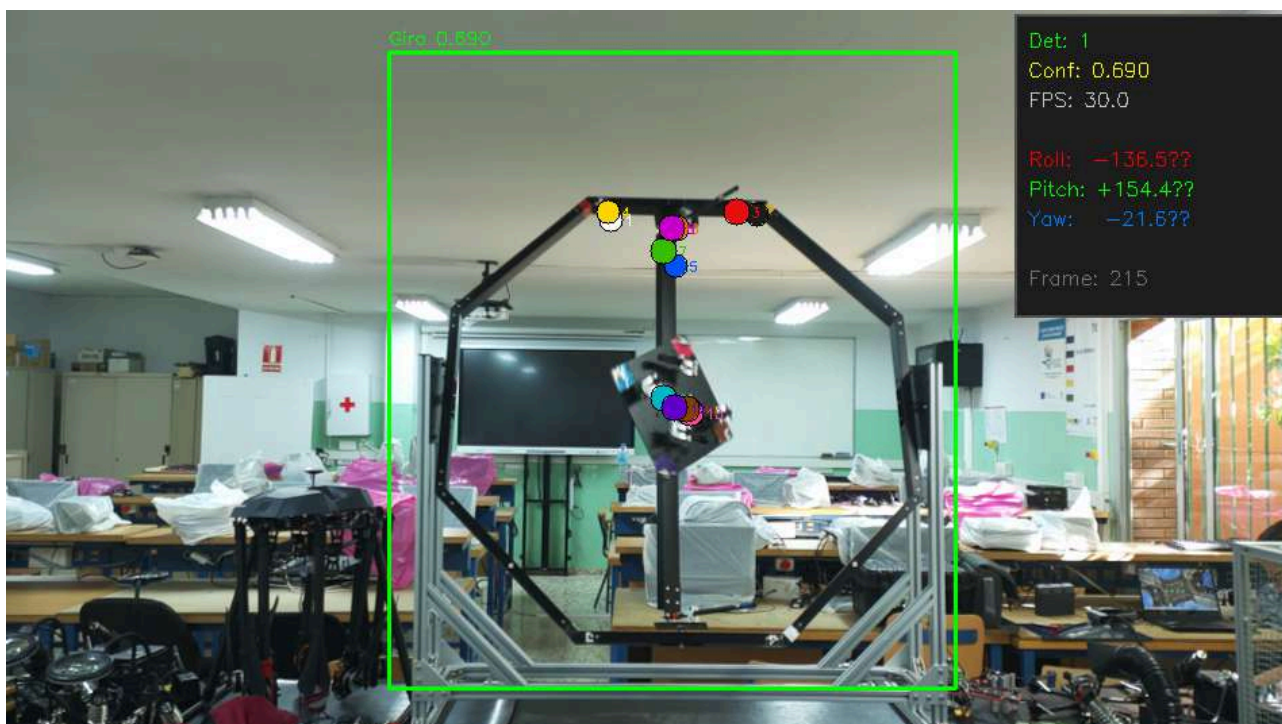


Desde ese punto, se comprueba que el proyecto es viable y merece la pena invertir un poco más de tiempo: preparar una práctica guiada para los grupos de alumnos que además tenga utilidad curricular, emplear el giroscopio grande adquirido previamente para el curso de drones, repensar el etiquetado, ampliar los KP hasta los 12 vistos los problemas de la versión inicial y lanzarse con los alumnos a ponerlo a funcionar.

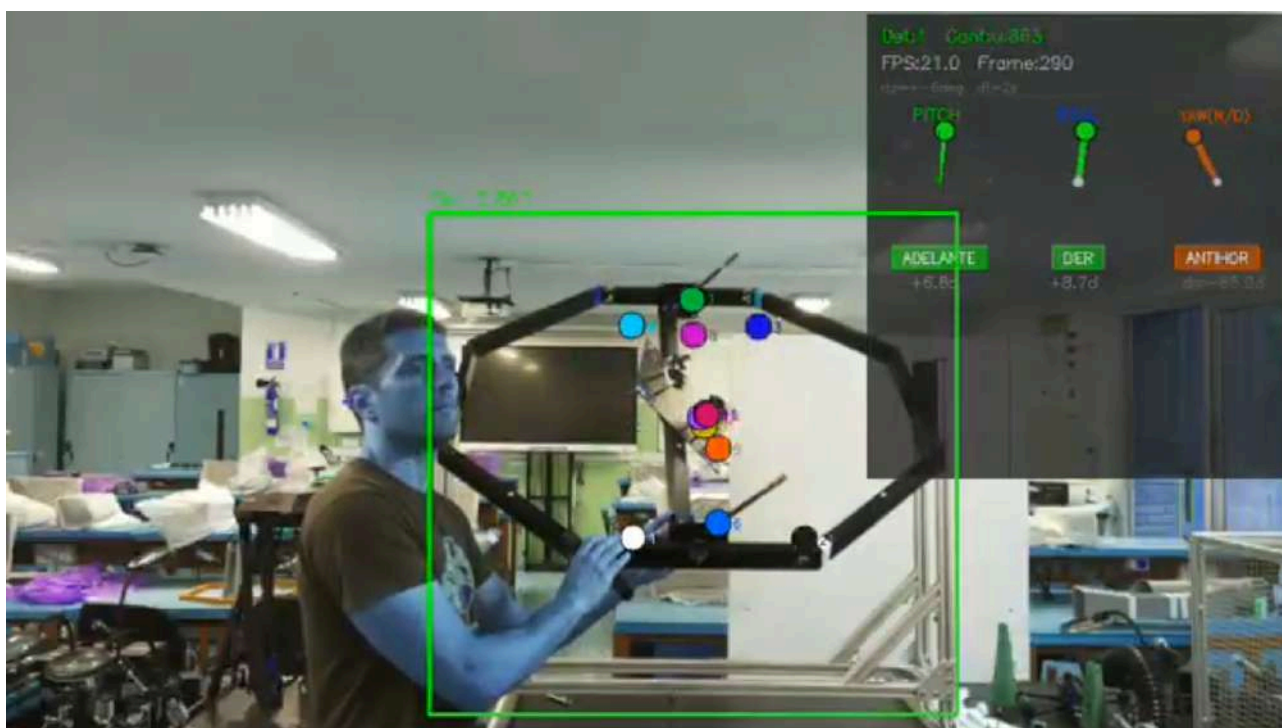


Una vez comprobado que esta práctica con los alumnos plantea nuevos problemas, no se abandona el proyecto, sino que se intenta arreglar con un nuevo etiquetado (220 imágenes) más controlado con un sólo alumno, pero nuevamente nos encontramos con otro problema: el dataset no es de suficiente calidad.





Se revisa el dataset y se planifica otra vez rehacer un dataset con mucho más cuidado, subiendo hasta las 410 imágenes y trabajando en paralelo alumno y profesor haciendo el mismo trabajo. El resultado principal de este documento es justamente el fruto de ese trabajo, concretamente la parte del profesor, que es la que ha llevado el modelo hasta las 410 imágenes.



En esta última versión es donde se abandona definitivamente (al menos con este hardware) la idea de conseguir los grados exactos de cada eje y se opta por tomar sólo los datos fiables: dirección de movimiento en pitch y roll y mantenemos la información de yaw, aunque sabemos que no es fiable.

## 14. Lanzador de escritorio en Raspberry Pi

Una vez que el sistema funciona desde la línea de comandos, el siguiente paso natural es simplificar el arranque para uso en el aula: que cualquier persona pueda lanzar el sistema con un doble clic desde el escritorio, sin necesidad de abrir un terminal ni recordar ningún comando.

### 14.1 Estructura de la solución

La solución consta de cuatro archivos que trabajan en conjunto:

| Archivo                    | Función                                                                                                                                               |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| lanzar_giroscopio.sh       | Wrapper bash. Configura el entorno, verifica que existen el script y el HEF, lanza el script Python y mantiene el terminal abierto para ver los logs. |
| lanzar_giroscopio_video.sh | Igual que el anterior pero activa la grabación automática de vídeo con nombre estandarizado en giro_resultados_videos/.                               |
| Giroscopio.desktop         | Icono en el escritorio de Raspberry Pi OS. Al hacer doble clic ejecuta lanzar_giroscopio.sh en una ventana de terminal.                               |
| Giroscopio_Video.desktop   | Igual que el anterior pero ejecuta lanzar_giroscopio_video.sh.                                                                                        |

La separación en wrapper + .desktop es necesaria porque un archivo .desktop no puede configurar variables de entorno ni mantener el terminal abierto. El wrapper actúa de intermediario.

### 14.2 Bloque configurable de los wrappers

Todos los parámetros se concentran al inicio de cada wrapper en un bloque claramente delimitado. Es el único lugar donde hay que tocar algo cuando se cambia una ruta o un modelo:

#### lanzar\_giroscopio.sh

```
— PARÁMETROS CONFIGURABLES —————
SCRIPT_DIR="/home/ai/giroscopio/giro_scripts"
SCRIPT_NAME="giroscopio_12kp_v3h.py"
HEF_PATH="/home/ai/giroscopio/giroscopio_v3a/p410/giroscopio.hef"
LOG_DIR="/home/ai/giroscopio/giro_historico_logs"
PYTHON_BIN="python3"
VENV_ACTIVATE="$SCRIPT_DIR/venv/bin/activate" # Descomentar si usas venv
#
```

#### lanzar\_giroscopio\_video.sh (añade VIDEO\_DIR)

```
— PARÁMETROS CONFIGURABLES —————
SCRIPT_DIR="/home/ai/giroscopio/giro_scripts"
SCRIPT_NAME="giroscopio_12kp_v3h.py"
HEF_PATH="/home/ai/giroscopio/giroscopio_v3a/p410/giroscopio.hef"
LOG_DIR="/home/ai/giroscopio/giro_historico_logs"
```

```
VIDEO_DIR="/home/ai/giroscopio/giro_resultados_videos"
PYTHON_BIN="python3"
VENV_ACTIVATE="$SCRIPT_DIR/venv/bin/activate" # Descomentar si usas venv
#
```

**i** Los flags opcionales del script Python (`--hide-yaw`, `--iou-thresh`, `--verbose...`) se pueden pasar al wrapper directamente desde terminal: `./lanzar_giroscopio.sh --hide-yaw`. Desde el icono de escritorio no es posible pasar argumentos, pero se pueden añadir *hardcodeados* al final de la línea de llamada en el wrapper si se desean siempre activos.

### 14.3 Problema encontrado: pantalla negra sin error

Durante la implementación se observó que al hacer doble clic desde el escritorio el terminal se abría pero permanecía completamente en negro, sin ningún mensaje ni error. El mismo script ejecutado manualmente desde terminal funcionaba perfectamente.

#### Causa raíz

Al lanzar un proceso desde el escritorio en Raspberry Pi OS Bookworm (que usa Wayland), el proceso hijo no hereda la variable `XAUTHORITY` de la sesión gráfica. Sin ese fichero de autenticación, el servidor X11 rechaza cualquier intento de abrir una ventana, pero lo hace silenciosamente: no aparece ningún mensaje de error en el terminal.

| Variable                     | Función                                                | Heredada al lanzar desde escritorio           |
|------------------------------|--------------------------------------------------------|-----------------------------------------------|
| <code>DISPLAY</code>         | Indica qué servidor X11 usar (:0 = pantalla principal) | No — exportarla explícitamente                |
| <code>XAUTHORITY</code>      | Fichero de autenticación para conectar al servidor X11 | No — causa pantalla negra silenciosa si falta |
| <code>WAYLAND_DISPLAY</code> | Indica sesión Wayland activa                           | Sí — eliminarla para forzar X11               |

#### Solución

Injectar ambas variables explícitamente en la línea Exec del archivo `.desktop` usando `env`:

```
Sin XAUTHORITY - pantalla negra silenciosa al hacer doble clic:
Exec=lxterminal -e /home/ai/.../lanzar_giroscopio.sh

Con XAUTHORITY - funciona correctamente:
Exec=env DISPLAY=:0 XAUTHORITY=/home/ai/.Xauthority \
 lxterminal --title="Giroscopio 12KP" \
 -e /home/ai/giroscopio/giro_scripts/lanzar_giroscopio.sh
```

**i** Si se cambia de usuario o se instala en otra Raspberry Pi, hay que actualizar `/home/ai/` por el home del nuevo usuario tanto en los `.desktop` como en los wrappers `.sh`.



### 14.4 Instalación de los lanzadores

```
1. Dar permisos de ejecución a los wrappers
chmod +x /home/ai/giroscopio/giro_scripts/lanzar_giroscopio.sh
chmod +x /home/ai/giroscopio/giro_scripts/lanzar_giroscopio_video.sh

2. Copiar los iconos al escritorio y dar permisos
cp Giroscopio.desktop ~/Desktop/
cp Giroscopio_Video.desktop ~/Desktop/
chmod +x ~/Desktop/Giroscopio.desktop
chmod +x ~/Desktop/Giroscopio_Video.desktop

3. Marcar como archivos de confianza (necesario en Bookworm)
Equivale a 'Allow Launching' — solo hay que hacerlo una vez
gio set ~/Desktop/Giroscopio.desktop metadata::trusted true
gio set ~/Desktop/Giroscopio_Video.desktop metadata::trusted true
```

### 14.5 Cambios en el script Python (v3h — 08/06/2026)

Para soportar los lanzadores, se añadieron dos nuevos argumentos CLI a giroscopio\_12kp\_v3h.py:

| Argumento          | Función                                                                      | Comportamiento si no se especifica                                                 |
|--------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| --log-dir <ruta>   | Directorio donde guardar los logs. Normalmente pasado por el wrapper.        | Los logs van a <dir_script>/historico_logs/ (compatibilidad con ejecución directa) |
| --save-auto <ruta> | Directorio para grabar vídeo con nombre automático. Incompatible con --save. | No se graba vídeo (comportamiento por defecto)                                     |

La función setup\_logging() también se actualizó: antes extraía el identificador de grupo del patrón\_gN en el nombre del fichero HEF. Ahora extrae el nombre del modelo del directorio padre del HEF, lo que encaja con la estructura de carpetas del proyecto.

## 15. Estructura de carpetas y nomenclatura en la Raspberry Pi

Esta sección documenta la organización completa de ficheros del proyecto en la Raspberry Pi 5: qué hay en cada carpeta, con qué nombre se guarda cada cosa, quién crea cada directorio y dónde modificar los parámetros cuando se cambia de modelo o de ruta.

### 15.1 Árbol de directorios completo

```
/home/ai/giroscopio/
├── giro_scripts/ # Scripts Python y wrappers de lanzamiento
│ ├── giroscopio_12kp_v3h.py # Script principal (versión de producción)
│ ├── lanzar_giroscopio.sh # Lanzador normal (sin grabación de vídeo)
│ └── lanzar_giroscopio_video.sh # Lanzador con grabación automática
├── giroscopio_v3a/ # Modelos HEF compilados para Hailo-8
│ ├── p113/
│ │ └── giroscopio.hef # Modelo entrenado con 113 imágenes
│ ├── p211/
│ │ └── giroscopio.hef # Modelo entrenado con 211 imágenes
│ ├── p309/
│ │ └── giroscopio.hef # Modelo entrenado con 309 imágenes
│ └── p410/
│ └── giroscopio.hef # Modelo de producción (410 imágenes)
├── giro_historico_logs/ # Logs de sesión — creado automáticamente
│ └── logs_giroscopio_v3a_p410_20260608_143022.txt
├── giro_resultados_videos/ # Vídeos grabados — creado automáticamente
│ └── giroscopio_v3a_p410_20260608_143022.mp4
└── ~/Desktop/
 ├── Giroscopio.desktop # Icono lanzador normal
 └── Giroscopio_Video.desktop # Icono lanzador con vídeo
```

**i** Los directorios `giro_historico_logs/` y `giro_resultados_videos/` los crean automáticamente los wrappers en el primer arranque. No hay que crearlos a mano.

### 15.2 Descripción de cada directorio

| Directorio           | Contenido                                                                                      | Creado por                              | Hay que tocarlo si...                                                 |
|----------------------|------------------------------------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------|
| giro_scripts/        | Scripts Python y wrappers bash                                                                 | Manual (SCP o editor)                   | Se actualiza el script o se añaden nuevos lanzadores                  |
| giroscopio_v3a/      | Modelos HEF compilados, uno por subdirectorio nombrado según el número de imágenes del dataset | Manual (SCP desde el PC de compilación) | Se compila un nuevo modelo                                            |
| giroscopio_v3a/p410/ | Subdirectorio del modelo activo. El nombre (p410) es el que aparece en logs y vídeos           | Manual                                  | Se cambia de modelo — actualizar HEF_PATH en ambos wrappers           |
| giro_historico_logs/ | Un fichero .txt por sesión de inferencia. Nunca se borran automáticamente                      | Automático (wrapper, primer uso)        | Se quiere cambiar la ubicación — actualizar LOG_DIR en ambos wrappers |

|                         |                                                             |                                  |                                                                                     |
|-------------------------|-------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------|
| giro_resultados_videos/ | Un fichero .mp4 por sesión grabada con el lanzador de vídeo | Automático (wrapper, primer uso) | Se quiere cambiar la ubicación — actualizar VIDEO_DIR en lanzar_giroscopio_video.sh |
|-------------------------|-------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------|

## 15.3 Nomenclatura de los ficheros generados automáticamente

Todos los ficheros generados por el sistema siguen un patrón común que incluye el nombre del modelo activo y un timestamp, lo que permite identificar de qué sesión proviene cada fichero sin ambigüedad.

| Tipo          | Patrón de nombre                                   | Ejemplo                                      |
|---------------|----------------------------------------------------|----------------------------------------------|
| Log de sesión | logs_giroscopio_v3a_<modelo>_<YYYYMMDD_HHMMSS>.txt | logs_giroscopio_v3a_p410_20260608_143022.txt |
| Vídeo grabado | giroscopio_v3a_<modelo>_<YYYYMMDD_HHMMSS>.mp4      | giroscopio_v3a_p410_20260608_143022.mp4      |

El campo <modelo> se extrae automáticamente del directorio padre del archivo HEF. No hay ninguna variable adicional que configurar: si se cambia HEF\_PATH de .../p410/... a .../p309/..., el nombre del modelo en logs y vídeos cambia solo a partir de la siguiente sesión.

```
Ejemplo de extracción automática del nombre de modelo:
HEF_PATH = "/home/ai/giroscopio/giroscopio_v3a/p410/giroscopio.hef"
^^^^
directorio padre del HEF = nombre del modelo

Resultados en los nombres de fichero generados:
logs_giroscopio_v3a_p410_20260608_143022.txt
giroscopio_v3a_p410_20260608_143022.mp4
```

## 15.4 Referencia rápida: dónde tocar cada parámetro

| Si quiero cambiar...                      | Fichero a editar                    | Variable                                 |
|-------------------------------------------|-------------------------------------|------------------------------------------|
| El modelo HEF activo                      | ambos wrappers .sh                  | HEF_PATH                                 |
| El directorio donde se guardan los logs   | ambos wrappers .sh                  | LOG_DIR                                  |
| El directorio donde se guardan los vídeos | lanzar_giroscopio_video.sh          | VIDEO_DIR                                |
| El script Python a ejecutar               | ambos wrappers .sh                  | SCRIPT_NAME                              |
| El directorio base del proyecto           | ambos wrappers .sh                  | SCRIPT_DIR                               |
| El usuario de la Raspberry Pi             | ambos .desktop + ambos wrappers .sh | Rutas /home/<usuario>/ y XAUTHORITY      |
| Flags activos por defecto                 | ambos wrappers .sh                  | Añadir al final de la línea \$PYTHON_BIN |

## 15.5 Cómo cambiar el modelo activo

Para pasar de un modelo a otro (por ejemplo de p410 a p309), solo hay que:

- Copiar el HEF del nuevo modelo a su directorio en la Raspberry Pi.
- Editar HEF\_PATH en ambos wrappers.

```
Antes (modelo p410):
HEF_PATH="/home/ai/giroscopio/giroscopio_v3a/p410/giroscopio.hef"

Después (modelo p309):
HEF_PATH="/home/ai/giroscopio/giroscopio_v3a/p309/giroscopio.hef"
```

Los nombres de todos los ficheros generados (logs y vídeos) cambiarán automáticamente a p309 a partir del siguiente arranque. No hay que tocar ninguna otra variable.

## 15.6 Instalación completa desde cero

Secuencia completa para instalar el sistema en una Raspberry Pi nueva:

```
1. Crear la estructura de directorios base
mkdir -p /home/ai/giroscopio/giro_scripts
mkdir -p /home/ai/giroscopio/giroscopio_v3a/p410
Los directorios de logs y vídeos los crean los wrappers automáticamente

2. Transferir ficheros desde el PC de desarrollo
scp giroscopio_12kp_v3h.py ai@<IP-RPi>:/home/ai/giroscopio/giro_scripts/
scp lanzar_giroscopio.sh ai@<IP-RPi>:/home/ai/giroscopio/giro_scripts/
scp lanzar_giroscopio_video.sh ai@<IP-RPi>:/home/ai/giroscopio/giro_scripts/
scp giroscopio.hef ai@<IP-RPi>:/home/ai/giroscopio/giroscopio_v3a/p410/

3. Permisos de ejecución
chmod +x /home/ai/giroscopio/giro_scripts/lanzar_giroscopio.sh
chmod +x /home/ai/giroscopio/giro_scripts/lanzar_giroscopio_video.sh

4. Iconos de escritorio
cp Giroscopio.desktop ~/Desktop/
cp Giroscopio_Video.desktop ~/Desktop/
chmod +x ~/Desktop/Giroscopio.desktop
chmod +x ~/Desktop/Giroscopio_Video.desktop
gio set ~/Desktop/Giroscopio.desktop metadata::trusted true
gio set ~/Desktop/Giroscopio_Video.desktop metadata::trusted true

5. Verificación
ls -la /home/ai/giroscopio/giro_scripts/
ls -la /home/ai/giroscopio/giroscopio_v3a/p410/giroscopio.hef
```

## 16. Modelo a100: validación cruzada por etiquetado independiente

Esta sección documenta el trabajo realizado por un alumno del ciclo que etiquetó de forma completamente independiente el mismo conjunto de imágenes del giroscopio que el profesor había usado para entrenar el modelo p113. El resultado, el modelo a100, no solo es funcionalmente comparable al p113 sino que su similitud cuantitativa constituye una validación cruzada de la metodología de etiquetado que tiene un valor pedagógico propio.

### 16.1 Contexto del experimento

| Parámetro               | Modelo p113 (profesor)    | Modelo a100 (alumno)      |
|-------------------------|---------------------------|---------------------------|
| Imágenes raw            | 113                       | 100                       |
| Fuente de las imágenes  | Vídeo del giroscopio v3a  | Mismo vídeo (subconjunto) |
| Etiquetador             | Profesor                  | Alumno (independiente)    |
| Augmentaciones          | Ninguna                   | Ninguna                   |
| Épocas de entrenamiento | 150                       | 150                       |
| Notebook                | giroscopio_colab_v4.ipynb | giroscopio_colab_v4.ipynb |
| HEF compilado           | Sí (calibración cruzada)  | Sí (calibración cruzada)  |
| Nombre del modelo       | p113                      | a100                      |

**i** El prefijo 'a' en a100 indica 'alumno', distinguiéndolo de la serie p (profesor). El número indica las imágenes etiquetadas. Ambos modelos parten del mismo vídeo fuente pero los etiquetados son completamente independientes: el alumno no vio ni consultó el etiquetado del profesor.

### 16.2 Métricas de entrenamiento

#### Comparativa directa a100 vs p113

| Métrica                | a100 (alumno, 100 imgs) | p113 (profesor, 113 imgs) | Diferencia |
|------------------------|-------------------------|---------------------------|------------|
| mAP50(B) — mejor       | 0.9950                  | 0.9950                    | 0.0000 ✓   |
| mAP50-95(B) — mejor    | 0.9304                  | 0.9304                    | 0.0000 ✓   |
| mAP50(P) — mejor       | 0.7710                  | 0.7710                    | 0.0000 ✓   |
| mAP50-95(P) — mejor    | 0.1303                  | 0.1303                    | 0.0000 ✓   |
| Época best mAP50(P)    | 116                     | 116                       | 0 ✓        |
| Época best mAP50-95(P) | 119                     | 119                       | 0 ✓        |
| val/pose_loss final    | 4.621                   | 4.645                     | +0.024     |
| val/kobj_loss final    | 0.045                   | ~0.046                    | ≈0         |



La coincidencia es llamativa: con 13 imágenes menos y un etiquetador diferente, todas las métricas de entrenamiento son prácticamente idénticas a las del p113. Esto tiene una explicación clara.

### Por qué coinciden las métricas

Ambos modelos comparten el mismo techo de rendimiento con datasets pequeños sin augmentación. La bbox detection ( $mAP_{50}(B)=0.995$ ) satura rápido porque detectar el contorno del giroscopio es una tarea sencilla para YOLOv8n. La pose estimation ( $mAP_{50-95}(P)=0.13$ ) es donde aparece la limitación real: con ~100 imágenes sin augmentación y un único fondo, el modelo aprende a localizar los keypoints en ese entorno concreto pero no generaliza bien a variaciones de perspectiva o iluminación. Eso es exactamente lo mismo que ocurre con p113.

El hecho de que el alumno llegue exactamente al mismo techo con su propio etiquetado demuestra que la limitación no está en la calidad del etiquetado sino en el tamaño y variedad del dataset. Es la misma conclusión que llevó al profesor a ampliar progresivamente el dataset hasta p410.

## 16.3 Análisis de las curvas de entrenamiento

Las curvas del a100 muestran el mismo patrón que las del p113:

- train/box\_loss y train/cls\_loss convergen rápido (antes de época 10). La detección del objeto es inmediata.
- train/pose\_loss y val/pose\_loss descienden lentamente y siguen bajando hasta la época 150 sin estabilizarse completamente. El modelo aún no ha saturado la tarea de pose con este dataset.
- metrics/mAP50-95(P) en validación es errático y ruidoso. Con tan pocas imágenes de validación, una sola detección correcta o incorrecta cambia la métrica drásticamente.
- El plateau de val/pose\_loss alrededor de 4.5-4.7 es idéntico al del p113. Ambos modelos llegan al mismo mínimo local.

**i** La discontinuidad visible en las curvas del a100 alrededor de la época 141 (train/pose\_loss salta de ~3.5 a ~5.1, train/kobj\_loss cae de ~0.38 a ~0.15) es un artefacto del scheduler de YOLOv8 (cosine annealing + warmup restart), no un problema del dataset ni del entrenamiento. Se observa el mismo patrón en otros modelos del proyecto.

## 16.4 Resultados en inferencia (sesión 08/06/2026)

El modelo a100 se probó en la Raspberry Pi el mismo día 08/06/2026 con el script giroscopio\_12kp\_v3h.py y los mismos parámetros usados para los modelos p###.

**Métricas de la sesión (log: logs\_giroscopio\_v3a\_a100\_20260608\_140337.txt)**

| Métrica               | a100 (alumno)      | p113 (profesor)   | p410 (mejor modelo) |
|-----------------------|--------------------|-------------------|---------------------|
| Frames totales        | 1.886              | ~2.000 (estimado) | 4.483               |
| Frames sin detección  | 0 (0%)             | ~0%               | 0 (0%)              |
| Confianza media       | 0.737              | ~0.636 (estimado) | 0.888               |
| Frames conf>0.5       | 89.8%              | ~68%              | 99.5%               |
| Frames conf>0.7       | 73.9%              | ~57%              | 98.7%               |
| Frames conf>0.9       | 0%                 | 0%                | 54.5%               |
| Detecciones dobles    | 18.7% (352 frames) | ~similar          | ~5% (con IOU=0.30)  |
| FPS display (mediana) | ~24 FPS            | ~22 FPS           | ~22 FPS             |
| Bug s=8 raw_conf=0    | Sí (1 warning)     | Sí                | Sí                  |

**i** Los valores del p113 en esta tabla son estimaciones basadas en el patrón observado en pruebas anteriores (sección 7). El a100 muestra confianza media superior (0.737 vs ~0.636), lo que puede deberse a diferencias menores en el estilo de etiquetado o a la sesión de prueba realizada en el aula con el alumno presente — condiciones de iluminación y distancia pueden variar entre sesiones.

**Punto a mejorar: detecciones dobles**

El 18.7% de frames con detección doble es claramente superior al observado en los modelos p### (donde se corrigió con IOU\_THRESH=0.30). La causa más probable es que el HEF del a100 fue compilado con un conjunto de calibración que genera umbrales ligeramente diferentes, haciendo que dos bboxes superen el umbral de confianza simultáneamente en más frames.

La solución es la misma que se aplicó al p309 y p410: bajar el umbral IoU:

```
Lanzar con umbral IoU más estricto para reducir detecciones dobles:
./lanzar_giroscopio.sh --iou-thresh 0.20

O desde terminal directamente:
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
 --hef /home/ai/giroscopio/giroscopio_v3a/a100/giroscopio.hef \
 --iou-thresh 0.20
```

**16.5 Valor pedagógico del experimento**

El modelo a100 no se incluye en este proyecto como resultado técnico novedoso — sus métricas ya estaban cubiertas por p113. Su valor es pedagógico y metodológico:

- Validación cruzada del proceso. El alumno siguió el pipeline completo de forma independiente (grabación de dataset, etiquetado en Roboflow, entrenamiento en Colab, compilación para Hailo-8, prueba en la Raspberry Pi) y llegó a resultados

comparables al modelo del profesor. Esto demuestra que el proceso es reproducible por alumnos del ciclo sin supervisión directa.

- La limitación es del dataset, no del etiquetador. El hecho de que dos etiquetadores independientes lleguen exactamente al mismo techo de rendimiento ( $mAP_{50-95}(P) \approx 0.13$ ) confirma que el problema no es la calidad del etiquetado sino el tamaño y la variedad del dataset. Esta es la conclusión más importante del experimento.
- Mismo techo con menos imágenes. El alumno alcanzó las mismas métricas con 100 imágenes que el profesor con 113, lo que sugiere que en este rango (sin augmentación, fondo único) el número de imágenes tiene menos impacto que la variedad del entorno. La mejora real llegó con los modelos p309 y p410, que incorporaron augmentación y mayor variedad de posiciones.
- El pipeline funciona de extremo a extremo. El alumno demostró que un estudiante de CFGS Mantenimiento Electrónico puede, con la guía adecuada, completar todo el flujo: desde el vídeo crudo hasta el modelo corriendo en tiempo real en hardware embebido.

## 16.6 Posición del a100 en la comparativa global

Incluyendo el modelo a100, la tabla comparativa completa de modelos entrenados sobre el dataset del giroscopio v3a queda así:

| Modelo | Imágenes | Etiquetador | Aug. | mAP50 (P) | mAP50-95 (P) | Conf. media (RPI) | Estado                 |
|--------|----------|-------------|------|-----------|--------------|-------------------|------------------------|
| a100   | 100      | Alumno      | No   | 0.771     | 0.130        | 0.737             | Compilado ✓, Probado ✓ |
| p113   | 113      | Profesor    | No   | 0.771     | 0.130        | ~0.636            | Compilado ✓, Probado ✓ |
| p211   | 211      | Profesor    | Sí   | 0.995     | 0.716        | —                 | Compilado ✓, Probado ✓ |
| p309   | 309      | Profesor    | Sí   | 0.995     | 0.748        | 0.636             | Compilado ✓, Probado ✓ |
| p410   | 410      | Profesor    | Sí   | 0.995     | 0.764        | <b>0.888</b>      | <b>PRODUCCIÓN</b>      |

El salto más importante en esta tabla no está entre a100 y p113 (donde la diferencia es mínima) sino entre p113/a100 y p211: la incorporación de augmentaciones y más imágenes con variedad de posiciones es lo que lleva el  $mAP_{50-95}(P)$  de 0.13 a 0.72, un salto de más de 5 puntos. Eso confirma la conclusión del proyecto: con datasets pequeños y fondo único, ni más imágenes ni mejor etiquetado mejoran el modelo — lo que mejora es la variedad.

## 17. Reflexión pedagógica

Este proyecto demuestra que la Inteligencia Artificial aplicada al hardware embebido es accesible para alumnos de ciclos formativos, siempre que se aborde con la metodología adecuada: construir el sistema desde cero, entender cada componente y no tratar la IA como una caja negra.

- **El pensamiento crítico es insustituible.** El Bug 3 (calibración del HEF) fue diagnosticado incorrectamente varias veces antes de llegar a la causa real. El razonamiento desde los primeros principios fue lo que lo resolvió.
- **Los datos son el modelo.** El Bug 10 es el ejemplo más claro: un modelo técnicamente perfecto, compilado sin errores, ejecutado correctamente, daba resultados inútiles porque el dataset no representaba la realidad de uso.
- **Los errores son contenido educativo.** Los 12 bugs documentados son un recurso pedagógico excepcional. Muestran cómo depurar un sistema complejo, qué preguntas hacerse y cómo verificar hipótesis de forma sistemática.
- **La IA como herramienta de apoyo.** El uso de asistentes de IA para generar código, analizar métricas y crear documentación aceleró enormemente el trabajo, pero requirió supervisión constante y criterio técnico. La competencia digital no es saber usar las herramientas, sino saber cuándo no confiar en ellas.
- **Honestidad técnica como conclusión.** El paso de mostrar números de precisión falsa a un panel cualitativo (v3h) no fue una derrota del proyecto sino su conclusión más madura: documentar exactamente qué puede y qué no puede hacer el sistema con esta arquitectura.

*"La IA es una herramienta extraordinariamente potente que puede ahorrar miles de horas de trabajo. Pero el pensamiento crítico, el conocimiento técnico y la capacidad de cuestionar las respuestas recibidas siguen siendo insustituibles."*

— José Luis Guerrero Marín

## 18. Guía de replicación para docentes

Esta sección está dirigida a docentes de electrónica o informática que quieran reproducir este proyecto con su alumnado. El objetivo didáctico es entrenar un modelo de visión artificial que reconozca la orientación de un objeto físico mediante una cámara, ejecutarlo en tiempo real sobre hardware de bajo coste y comprender las etapas del ciclo completo de un proyecto de Edge AI.

Aunque en este proyecto se usa un giroscopio de dron como objeto de referencia, la metodología es completamente transferible a cualquier otro objeto sobre el que se quieran medir ángulos o posiciones: una antena, un brazo robótico, una válvula, un controlador de vuelo, etc. Lo que cambia entre proyectos son únicamente el objeto físico, el número y la posición de los keypoints, y el dataset.

### 18.1 Requisitos previos del docente

- Conocimientos básicos de Python (leer y modificar scripts existentes).
- Nociones de redes neuronales convolucionales (concepto de keypoint, bounding box, confianza).
- Capacidad de seguir un tutorial de línea de comandos en Linux.
- NO se requiere experiencia previa con Hailo, Docker ni compilación de modelos; los tutoriales del proyecto cubren todos los pasos.

### 18.2 Ciclo completo del proyecto

El ciclo tiene cinco fases que el alumnado puede ejecutar de forma incremental:

1. Fabricación o adquisición del objeto físico a reconocer.
2. Grabación del dataset con la cámara de la Raspberry Pi.
3. Etiquetado de keypoints en Roboflow (herramienta web gratuita).
4. Entrenamiento del modelo YOLOv8n-Pose en Google Colab (GPU gratuita T4).
5. Compilación del modelo para Hailo-8 (PC con Docker) y despliegue en la Raspberry Pi.

Cada fase está documentada en un anexo independiente. La sección 18.5 enlaza con todos ellos.

### 18.3 Software necesario

Todo el software es gratuito o de acceso libre. A continuación se lista cada herramienta con el enlace y la versión usada en el proyecto.



## En la Raspberry Pi (hardware de inferencia)

| Software                          | Versión usada            | Funcion                                  | Enlace / Instalación                                                                                                                                                                      |
|-----------------------------------|--------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Raspberry Pi OS Bookworm (64-bit) | Bookworm 64-bit          | Sistema operativo base                   | <a href="https://www.raspberrypi.com/software/">https://www.raspberrypi.com/software/</a>                                                                                                 |
| Python 3                          | 3.11 (incluido en el SO) | Lenguaje de los scripts                  | Incluido con el SO                                                                                                                                                                        |
| HailoRT                           | 4.23.0                   | Runtime de inferencia en el chip Hailo-8 | Incluido en el AI HAT+ setup: <a href="https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html">https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html</a> |
| Picamera2                         | Última disponible        | Captura de video desde la cámara RPi     | <code>sudo apt install python3-picamera2</code>                                                                                                                                           |
| OpenCV (cv2)                      | 4.x                      | Procesado de imagen y visualización      | <code>sudo apt install python3-opencv</code>                                                                                                                                              |
| NumPy                             | 1.x                      | Cálculo vectorial en los scripts         | <code>sudo apt install python3-numpy</code>                                                                                                                                               |
| hailo-all (metapaquete)           | Según Bookworm           | Instala HailoRT y drivers en la RPi      | <code>sudo apt install hailo-all</code>                                                                                                                                                   |

## En cualquier PC para grabación del dataset y gestión del proyecto

| Software     | Versión usada                     | Funcion                                                              | Enlace                                                                            |
|--------------|-----------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Roboflow     | Web (gratuito hasta 10k imágenes) | Etiquetado de keypoints, gestión del dataset y exportación YOLO      | <a href="https://roboflow.com">https://roboflow.com</a>                           |
| Google Colab | Web (GPU T4 gratuita)             | Entrenamiento del modelo con GPU NVIDIA                              | <a href="https://colab.research.google.com">https://colab.research.google.com</a> |
| Google Drive | Web                               | Transferencia del dataset y de los archivos .onnx/.hef entre equipos | <a href="https://drive.google.com">https://drive.google.com</a>                   |

## En un PC Ubuntu o Windows para compilar el modelo Hailo

La compilación del modelo ONNX a formato HEF (Hailo Executable Format) requiere el Hailo Dataflow Compiler (DFC), que se distribuye como imagen Docker. Solo es necesario en la fase de compilación; el alumnado puede hacer todas las demás fases sin este paso si el profesor les proporciona el HEF ya compilado.

| Software                                | Versión usada                         | Funcion                                                | Enlace                                                                                                        |
|-----------------------------------------|---------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Docker Desktop                          | Última estable                        | Contenedor para el compilador Hailo                    | <a href="https://www.docker.com/products/docker-desktop/">https://www.docker.com/products/docker-desktop/</a> |
| Hailo AI Software Suite (imagen Docker) | hailo8_ai_sw_suite_2025-10:1 (~17 GB) | Hailo DFC 3.33.0: parse, optimize, compile ONNX -> HEF | <a href="https://hailo.ai/developer-zone/">https://hailo.ai/developer-zone/</a> (registro gratuito requerido) |

| Software                                          | Versión usada          | Funcion                                   | Enlace                        |
|---------------------------------------------------|------------------------|-------------------------------------------|-------------------------------|
| Python 3 + Pillow + NumPy (dentro del contenedor) | Incluidas en la imagen | Preparación del set de calibración (.npy) | Incluidas en la imagen Docker |

**ADVERTENCIA:** La imagen Docker de Hailo pesa ~17 GB. Descargarla antes de la sesión con el alumnado para no depender del ancho de banda. Registro en [hailo.ai](https://hailo.ai) requerido (gratuito para fines educativos).

## Librerías Python del script de inferencia

El script `giroscopio_12kp_v3h.py` solo usa librerías estándar disponibles en Raspberry Pi OS Bookworm. No hay que instalar nada adicional si se siguió el procedimiento de instalación del AI HAT+.

```
Librerías requeridas por giroscopio_12kp_v3h.py
(ya disponibles tras la instalación estándar del AI HAT+ en RPi OS Bookworm)
import numpy as np # sudo apt install python3-numpy
import cv2 # sudo apt install python3-opencv
from picamera2 import Picamera2 # sudo apt install python3-picamera2
import hailo # incluido en hailo-all
```

## 18.4 Hardware del giroscopio de aluminio

El giroscopio de tres ejes empleado en este proyecto NO es una pieza de impresión 3D. Está construido con perfilaría de aluminio estructural T-Slot (sistemas 40 y 20 mm) y componentes estándar de mecánica (rodamientos, ejes, tornillería) adquiridos en proveedores comerciales.

**ADVERTENCIA:** IMPORTANTE sobre viabilidad con drones reales: la estructura móvil de aluminio T-Slot, aunque completamente funcional como banco de pruebas didáctico y para manipulación manual, ha resultado ser demasiado pesada para los motores de los drones FPV comerciales de referencia probados (DJI Avata 2, DJI Mini 4 Pro). Esto está comprobado experimentalmente. La estructura móvil (octogono + soporte intermedio en U) debería rehacerse en fibra de carbono o materiales equivalentes si el objetivo es montar el giroscopio sobre un dron real.

## Listado de materiales y costes

Los materiales siguientes corresponden al giroscopio construido en el departamento para este proyecto. Los costes son los reales del pedido. Las herramientas (ingletadora, amoladora, disco de corte) no se incluyen en el total porque son de uso general del taller.

| Descripción                       | Proveedor     | Categoría                | Coste (EUR) |
|-----------------------------------|---------------|--------------------------|-------------|
| Monitor MSI 24,5" 120Hz           | PcComponentes | Componentes informáticos | 70,49       |
| Teclado inalámbrico Logitech K400 | Media Markt   | Componentes informáticos | 31,08       |

| Descripción                                                                                                                           | Proveedor  | Categoría                | Coste (EUR) |
|---------------------------------------------------------------------------------------------------------------------------------------|------------|--------------------------|-------------|
| Pack microPC: Raspberry Pi 5 8GB + AI HAT+ (Hailo-8) + camara v2 + fuente + cableHDMI + ventilador + bateria RTC + caja ALU + SD 64GB | RaspiPC    | Componentes informáticos | 360,69      |
| Parte central base (aluminio T-Slot 4040, 620 mm)                                                                                     | Arialpro   | Estructura               | 10,43       |
| Patas de la base (aluminio T-Slot 4040, 280 mm, x4)                                                                                   | Arialpro   | Estructura               | 18,84       |
| Patas de la base con roscado M8 (aluminio T-Slot 4080, 100 mm, x4)                                                                    | Arialpro   | Estructura               | 30,96       |
| Columnas fijas de la base (aluminio T-Slot 4040, 480 mm, x2)                                                                          | Arialpro   | Estructura               | 16,15       |
| Tapa PVC 4040 (x2)                                                                                                                    | Arialpro   | Tornillería y accesorios | 2,20        |
| 10 escuadras 40x40x35 + tornillería M8                                                                                                | Arialpro   | Tornillería y accesorios | 19,40       |
| 4 placas base 4080 M12                                                                                                                | Arialpro   | Tornillería y accesorios | 19,12       |
| 4 pies de maquina autonivelables M12                                                                                                  | Arialpro   | Tornillería y accesorios | 15,04       |
| 30 tornillos M8 serie S40                                                                                                             | Arialpro   | Tornillería y accesorios | 4,50        |
| 8 piezas aluminio T-Slot 2040 RANURA 6mm (250 mm) — octógono                                                                          | Arialpro   | Estructura               | 19,92       |
| 1 pieza aluminio T-Slot 2040 RANURA 6mm (510 mm) — soporte intermedio                                                                 | Arialpro   | Estructura               | 5,07        |
| 2 piezas aluminio T-Slot 2040 RANURA 6mm (120 mm) — verticales soporte                                                                | Arialpro   | Estructura               | 2,38        |
| 4 kit conector interno 90 grados S20 — soporte intermedio                                                                             | Arialpro   | Tornilleria y accesorios | 3,96        |
| 8 escuadras 135 grados serie 40 + tornillería M8 — octogono                                                                           | Arialpro   | Tornilleria y accesorios | 29,68       |
| Coste de envío (Arialpro)                                                                                                             | Arialpro   | Otros                    | 24,95       |
| Placa aluminio soporte del dron (3x150x300 mm)                                                                                        | AliExpress | Estructura               | 17,86       |
| Bloque rodamiento KFL 08 — bore 8 mm (x10)                                                                                            | AliExpress | Tornillería y accesorios | 23,04       |
| 12 anillos de retencion eje 8mm 8-16-8mm M4                                                                                           | AliExpress | Tornillería y accesorios | 19,85       |
| Tornillos M3 cabeza plana 30mm (25 uds)                                                                                               | AliExpress | Tornillería y accesorios | 2,18        |
| Tornillos M5 cabeza plana 30mm (20 uds x2)                                                                                            | AliExpress | Tornillería y accesorios | 6,78        |
| Tornillos M5 cabeza plana 50mm (10 uds x4)                                                                                            | AliExpress | Tornillería y accesorios | 9,76        |

| Descripción                                           | Proveedor    | Categoría                | Coste (EUR) |
|-------------------------------------------------------|--------------|--------------------------|-------------|
| Tuercas M3 50 uds                                     | AliExpress   | Tornillería y accesorios | 1,86        |
| Tuercas M5 50 uds                                     | AliExpress   | Tornillería y accesorios | 2,31        |
| Tuercas M8 20 uds x2                                  | AliExpress   | Tornillería y accesorios | 5,18        |
| Ejes 8mm (lote 8x150mm)                               | AliExpress   | Tornillería y accesorios | 7,46        |
| Perfil redondo aluminio 8mm x 1m (eje complementario) | Leroy Merlin | Tornillería y accesorios | 3,69        |
| Broca 22mm para metal                                 | AliExpress   | Herramientas             | 9,39        |
| Varillas roscadas 8mm 1 metro x2                      | Leroy Merlin | Tornillería y accesorios | 2,58        |

### TOTAL presupuesto: 684,86 EUR

Realmente el coste es algo superior por la imposibilidad de pedir los materiales por AliExpress, pero en todo caso es inferior a los 800€.

*ENLACE AL TUTORIAL DE FABRICACIÓN: El proceso de corte, ensamblaje y montaje de los ejes y rodamientos está documentado fotográficamente en el tutorial del proyecto (ver Anexo 5 — script de inferencia v3h y README del repositorio GitHub). No se requiere maquinaria de precisión, aunque sí hay que ser bastante cuidadoso; una ingletadora y una amoladora angular con disco de corte de metales son suficientes.*

## 18.5 Guía paso a paso y enlace con los anexos

Cada etapa del proyecto está documentada en un anexo independiente. El orden recomendado para seguir el proyecto de principio a fin es el siguiente:

| Paso                        | Que se hace                                                                                                                                                   | Donde está documentado                                             | Tiempo estimado                                |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|------------------------------------------------|
| 1. Preparar la Raspberry Pi | Instalar RPi OS Bookworm 64-bit, hailo-all, picamera2, opencv. Verificar que el AI HAT+ es reconocido.                                                        | Tutorial Hailo-8 (Anexo 4) — Sección "Instalación en Raspberry Pi" | 1-2 horas                                      |
| 2. Grabar el dataset        | Ejecutar grabar_dataset_v2.py para capturar frames con el objeto físico. Mínimo 3 fondos distintos, 100-200 frames por fondo. Giroscopio al 40-70% del frame. | Script grabar_dataset_v2.py (Anexo 2 — script de grabación)        | 2-3 horas (incluyendo preparación de escenas)  |
| 3. Etiquetar en Roboflow    | Subir imágenes a Roboflow, definir los keypoints del objeto, etiquetar una por una. Los KPs ocultos se marcan como "occluded", no se omiten.                  | Anexo 1 — Etiquetas de keypoints (PDF de referencia visual)        | 3-6 horas (dependiendo del número de imágenes) |

| Paso                            | Que se hace                                                                                                                                                                                                                      | Donde está documentado                                                  | Tiempo estimado                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------|
| 4. Entrenar en Google Colab     | Abrir el notebook, vincular con Roboflow y Google Drive, ejecutar el entrenamiento (YOLOv8n-Pose, 150 épocas). Exportar el modelo a ONNX.                                                                                        | Notebook giroscopio_colab_v4.ipynb (Anexo 3)                            | 1-2 horas (entrenamiento ~30-45 min en GPU T4) |
| 5. Compilar el HEF              | En PC con Docker: lanzar el contenedor Hailo, convertir imágenes de calibración a .npy, ejecutar hailo parser / optimize / compiler. ATENCIÓN: usar imágenes de calibración de un dataset DIFERENTE al de entrenamiento (Bug 3). | Tutorial Hailo-8 compilación (Anexo 4)                                  | 1 hora + ~22 min de compilación por modelo     |
| 6. Desplegar en la Raspberry Pi | Copiar el archivo .hef a la RPi, actualizar HEF_PATH en los lanzadores .sh, ejecutar desde el escritorio o línea de comandos.                                                                                                    | Script giroscopio_12kp_v3h.py (Anexo 5) + README del repositorio GitHub | 30 min                                         |
| 7. Calibrar offsets de ángulo   | Con el giroscopio en reposo horizontal, ajustar ROLL_OFFSET y PITCH_OFFSET hasta que el display muestre 0 grados. El offset depende de cómo se grabó el dataset (dirección de la cámara respecto al objeto).                     | Sección 5 del documento principal (parámetros del script)               | 15-30 min                                      |

Hay que tener en cuenta que todos los tiempos son estimaciones aproximadas y que, en cualquier caso, un error en el etiquetado penaliza enormemente todo el proceso, ya que en la mayoría de los casos no se ve hasta acabar el proceso y lo invalida al completo, te manda a la casilla de partida a revisar el dataset por si el error ha estado allí y, si eso está bien, a etiquetar de nuevo.

## 18.6 Repositorio GitHub y recursos en línea

Todo el código, scripts, tutoriales y documentación del proyecto están publicados en abierto. No se requiere registro para acceder al contenido.

### Repositorio principal

<https://github.com/nutshell-electronica/gyroscope>

#### Contenido del repositorio:

- README con instrucciones de instalación y ejecución.
- giroscopio\_12kp\_v3h.py — script principal de inferencia (producción).
- grabar\_dataset\_v2.py — script de grabación y extracción de frames.
- giroscopio\_colab\_v4.ipynb — notebook de entrenamiento en Google Colab.
- Tutoriales en PDF: instalación Hailo-8 en Ubuntu y en Windows.
- Archivo .alls de configuración para el compilador Hailo.



## Recursos en línea complementarios

| Recurso                      | URL                                                                                                                                                                                                                                                   | Descripción                                                                                                  |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Blog del proyecto            | <a href="https://blogsaverroes.juntadeandalucia.es/industria4/2026/03/15/deteccion-de-orientacion-de-giroscopio-para-drones/">https://blogsaverroes.juntadeandalucia.es/industria4/2026/03/15/deteccion-de-orientacion-de-giroscopio-para-drones/</a> | Entrada en el blog Industria 4.0 del IES Politécnico Jesús Marín con descripción del proyecto y resultados   |
| Audio Overview (NotebookLM)  | <a href="https://youtu.be/m3AsXro8m2U">https://youtu.be/m3AsXro8m2U</a>                                                                                                                                                                               | Resumen del proyecto en formato podcast generado con NotebookLM. Útil para presentar el proyecto a terceros. |
| Video evolución del proyecto | <a href="https://youtu.be/OTw97G_3yfA">https://youtu.be/OTw97G_3yfA</a>                                                                                                                                                                               | Video resumen de la evolución del sistema desde el prototipo hasta la versión de producción                  |
| Roboflow                     | <a href="https://roboflow.com">https://roboflow.com</a>                                                                                                                                                                                               | Plataforma de etiquetado. Plan gratuito hasta 10.000 imágenes. Exportación en formato YOLO incluida.         |
| Google Colab                 | <a href="https://colab.research.google.com">https://colab.research.google.com</a>                                                                                                                                                                     | Entorno de notebooks con GPU NVIDIA T4 gratuita. Solo se necesita una cuenta de Google.                      |
| Hailo Developer Zone         | <a href="https://hailo.ai/developer-zone/">https://hailo.ai/developer-zone/</a>                                                                                                                                                                       | Descarga de la imagen Docker del compilador (registro gratuito). Documentación del DFC.                      |
| Ultralytics YOLOv8           | <a href="https://docs.ultralytics.com/models/yolov8/">https://docs.ultralytics.com/models/yolov8/</a>                                                                                                                                                 | Documentación del modelo base. La versión Pose es la relevante para proyectos de keypoints.                  |

## 18.7 Cómo adaptar el proyecto a otro objeto

Para replicar el proyecto con un objeto diferente al giroscopio de dron, los cambios necesarios son mínimos y bien delimitados:

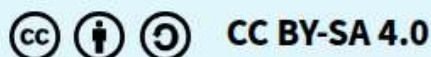
1. Definir los keypoints del nuevo objeto: elegir entre 4 y 17 puntos representativos cuya posición relativa en imagen cambie con la orientación. Cuantos más keypoints, más información geométrica disponible, pero más tiempo de etiquetado.
2. Actualizar el notebook de Colab: cambiar el número de keypoints (parámetro  $kp=N$  en la exportación de Roboflow y en la configuración del modelo YOLO).
3. Actualizar la lógica de cálculo de ángulos en el script de inferencia: las funciones de Roll, Pitch y Yaw usan índices de keypoints hardcodeados. Basta con cambiar esos índices para adaptarlos al nuevo esquema de puntos.
4. Actualizar los offsets de calibración (ROLL\_OFFSET, PITCH\_OFFSET) según la orientación de grabación del dataset.
5. El resto del pipeline (grabación, etiquetado en Roboflow, entrenamiento en Colab, compilación en Docker, despliegue en RPi) es exactamente igual.

*Regla de oro para cualquier objeto: grabar el dataset desde diferentes fondos y distancias (el objeto debe ocupar entre el 40% y el 70% del frame). La variedad de escenas es el factor más importante para la generalización del modelo; aumentar el número de imágenes del mismo entorno no mejora significativamente los resultados (ver Sección 9, comparativa de modelos).*

## 18.8 Carga de trabajo estimada para el alumnado

| Fase                                  | Horas alumno/a | Observaciones                                                                                     |
|---------------------------------------|----------------|---------------------------------------------------------------------------------------------------|
| Grabación del dataset (3 escenas)     | 2-3 h          | En parejas o grupos de 3. Requiere acceso a la Raspberry Pi con cámara.                           |
| Etiquetado en Roboflow (100 imágenes) | 3-5 h          | Trabajo individual. Puede hacerse desde cualquier PC con navegador.                               |
| Entrenamiento en Colab                | 1-2 h          | En su mayor parte tiempo de espera. El alumno lanza el notebook y supervisa.                      |
| Compilación HEF en Docker             | 1 h            | El profesor puede hacer este paso centralmente si el alumnado no tiene acceso a un PC con Docker. |
| Despliegue y pruebas en RPi           | 1-2 h          | Copiar el HEF, lanzar el script, observar y ajustar umbrales.                                     |
| TOTAL estimado                        | 8-13 h         | Equivale a 2-3 sesiones prácticas de laboratorio de 4 horas.                                      |

*Este proyecto ha sido desarrollado en el IES Politécnico Jesús Marín (Málaga) en el marco de la convocatoria de elaboración de materiales MTT-041/25 de la Consejería de Desarrollo Educativo y Formación Profesional de la Junta de Andalucía. Responsable: Jose Luis Guerrero Marín. El material es de uso libre para fines educativos y no comerciales.*



### Creative Commons Attribution-ShareAlike 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes. If others remix, adapt, or build upon the material, they must license the modified material under identical terms.

# Proyecto Giroscopio 12KP + Hailo-8

## ANEXOS

Código fuente, etiquetas y tutoriales de compilación

### Anexo I

Etiquetas de keypoints

### Anexo II

grabar\_dataset\_v2.py

### Anexo III

Notebook Colab v4 (entrenamiento)

### Anexo IV

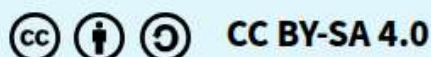
Tutorial compilación Hailo-8 (Ubuntu + Windows)

### Anexo V

giroscopio\_12kp\_v3h.py (inferencia)

IES Politécnico Jesús Marín — Málaga — Junta de Andalucía

Versión 3 — 08/06/2026



### Creative Commons Attribution-ShareAlike 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes. If others remix, adapt, or build upon the material, they must license the modified material under identical terms.

# Anexo I — Etiquetas de Keypoints

Este anexo contiene las etiquetas físicas de los 12 keypoints del giroscopio. Imprimir en color a escala 100% en papel normal. Cada página contiene 4 etiquetas idénticas de un keypoint — una por cara de la estructura.

NOTA: En iteraciones previas del proyecto, KP10 y KP11 tenían los colores invertidos en algunos documentos. Las etiquetas de este Anexo reflejan la asignación definitiva: KP10 = MARRÓN (base\_tras\_der, ArUco ID 9) · KP11 = ROSA (base\_front\_der, ArUco ID 10).



### Instrucciones de montaje

- 1. Imprimir este documento en color, a escala real (100%), en papel normal.
- 2. Recortar cada etiqueta por la línea exterior redondeada.
- 3. Cada keypoint tiene 4 etiquetas idénticas — una por cara de la estructura.
- 4. Pegar la franja de color rodeando el tubo/perfil en la posición indicada.
- 5. El marker ArUco debe quedar visible y sin doblar.
- 6. Verificar que el número del keypoint y el color coinciden con la tabla.

### Referencia rápida de colores:

#### OCTÓGONO — Pitch

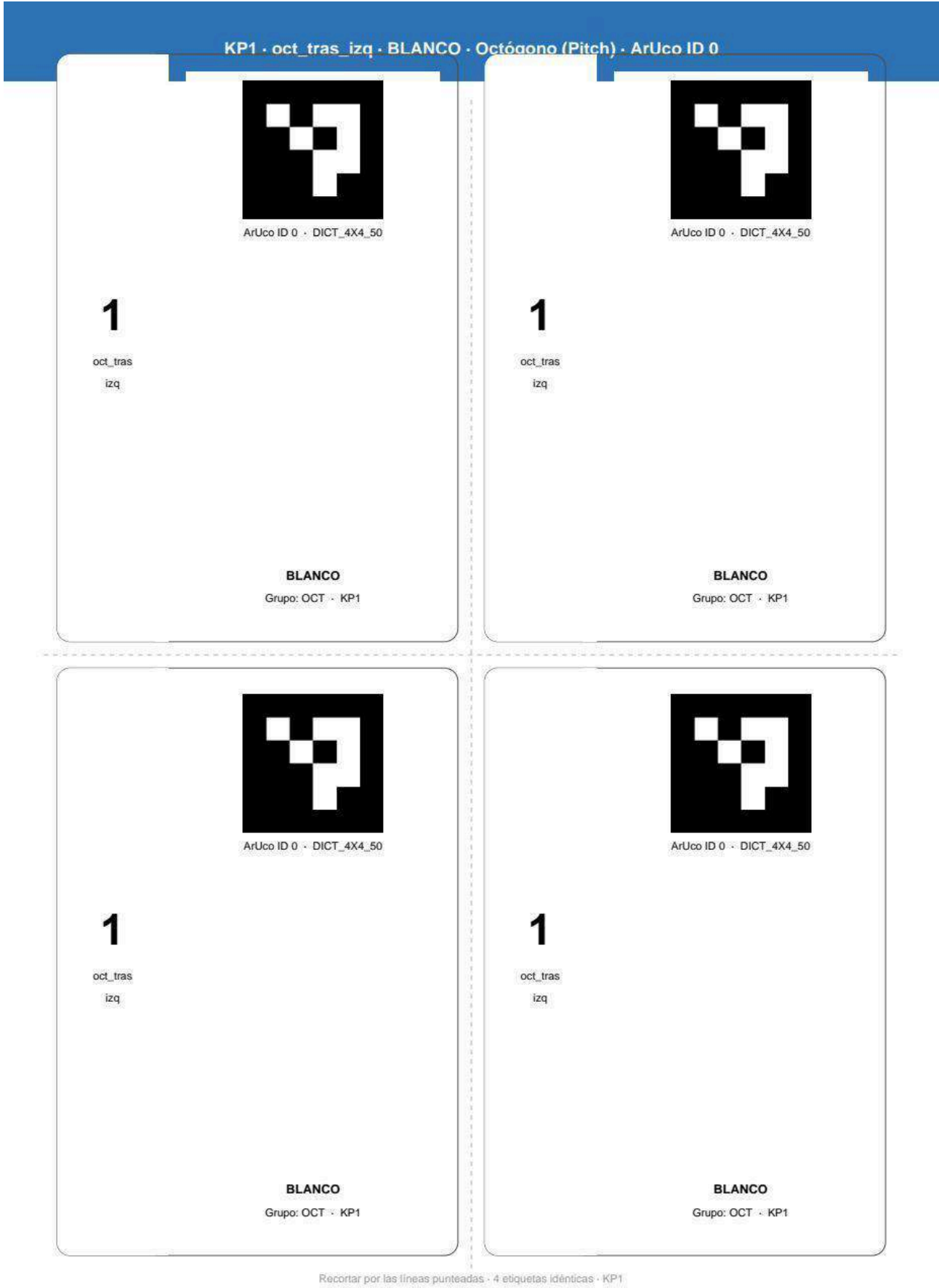
|                                                                                     |                 |
|-------------------------------------------------------------------------------------|-----------------|
|  | KP1 — blanco    |
|  | KP2 — negro     |
|  | KP3 — rojo vivo |
|  | KP4 — amarillo  |

#### SOPORTE — Roll

|                                                                                     |                      |
|-------------------------------------------------------------------------------------|----------------------|
|  | KP5 — azul eléctrico |
|  | KP6 — naranja        |
|  | KP7 — verde lima     |
|  | KP8 — magenta        |

#### BASE — Yaw

|                                                                                      |               |
|--------------------------------------------------------------------------------------|---------------|
|  | KP9 — cian    |
|  | KP10 — marrón |
|  | KP11 — rosa   |
|  | KP12 — morado |





KP2 · oct\_tras\_der · NEGRO · Octógono (Pitch) · ArUco ID 1

2  
oct\_tras  
der

ArUco ID 1 - DICT\_4X4\_50

NEGRO  
Grupo: OCT - KP2

2  
oct\_tras  
der

ArUco ID 1 - DICT\_4X4\_50

NEGRO  
Grupo: OCT - KP2

2  
oct\_tras  
der

ArUco ID 1 - DICT\_4X4\_50

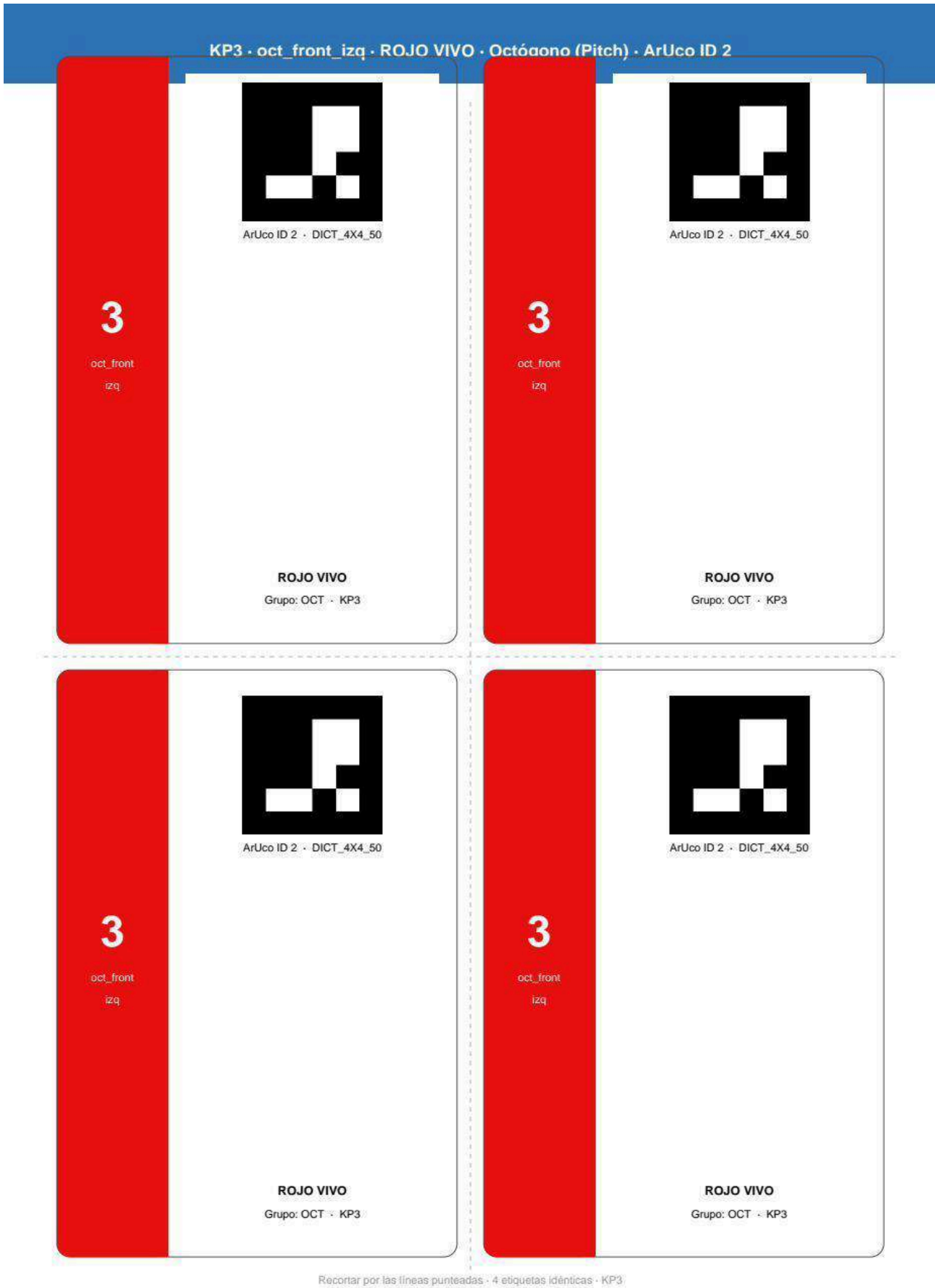
NEGRO  
Grupo: OCT - KP2

2  
oct\_tras  
der

ArUco ID 1 - DICT\_4X4\_50

NEGRO  
Grupo: OCT - KP2

José Luis Guerrero Marín · Junio 2026 · Pág. 4 de 52



KP4 · oct\_front\_der · AMARILLO · Octógono (Pitch) · ArUco ID 3



4

4

Grupo: OCT - KP4

4

4

Grupo: OCT - KP4



Grupo: OCT - KP4

José Luis Guerrero Marín · Junio 2026 · Pág. 6 de 52

KP5 · int\_tras\_arr · AZUL ELÉCTRICO · Soporte (Roll) · ArUco ID 4

The image displays four identical panels arranged in a 2x2 grid. Each panel is divided into three main sections. On the left, a solid blue vertical band contains a large white number '5' at the top, followed by the text 'int\_tras' and 'arr' in a smaller white font. The central section of each panel is white and features a black ArUco ID 4 marker at the top, with the text 'ArUco ID 4 · DICT\_4X4\_50' printed below it. The right section is also white and contains the brand name 'AZUL ELÉCTRICO' in bold black text, with 'Grupo: INT · KP5' printed below it. The panels are separated by thin white lines, and the entire set is framed by a white border.

José Luis Guerrero Marín · Junio 2026 · Pág. 7 de 52

KP6 · int\_tras\_abj · NARANJA · Soporte (Roll) · ArUco ID 5

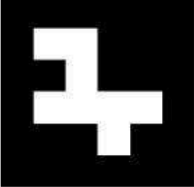
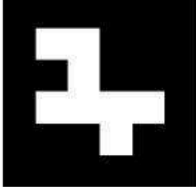
José Luis Guerrero Marín · Junio 2026 · Pág. 8 de 52

**KP7 — VERDE LIMA — Grupo: INT (Roll) — ArUco ID 6**





KP8 · int\_front\_abj · MAGENTA · Soporte (Roll) · ArUco ID 7

|                                   |                                                                                                                                                              |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>8</p> <p>int_front<br/>abj</p> |  <p>ArUco ID 7 · DICT_4X4_50</p> <p>MAGENTA</p> <p>Grupo: INT · KP8</p>   |
| <p>8</p> <p>int_front<br/>abj</p> |  <p>ArUco ID 7 · DICT_4X4_50</p> <p>MAGENTA</p> <p>Grupo: INT · KP8</p> |

José Luis Guerrero Marín · Junio 2026 · Pág. 10 de 52

**KP9 — CIAN — Grupo: BASE (Yaw) — ArUco ID 8**

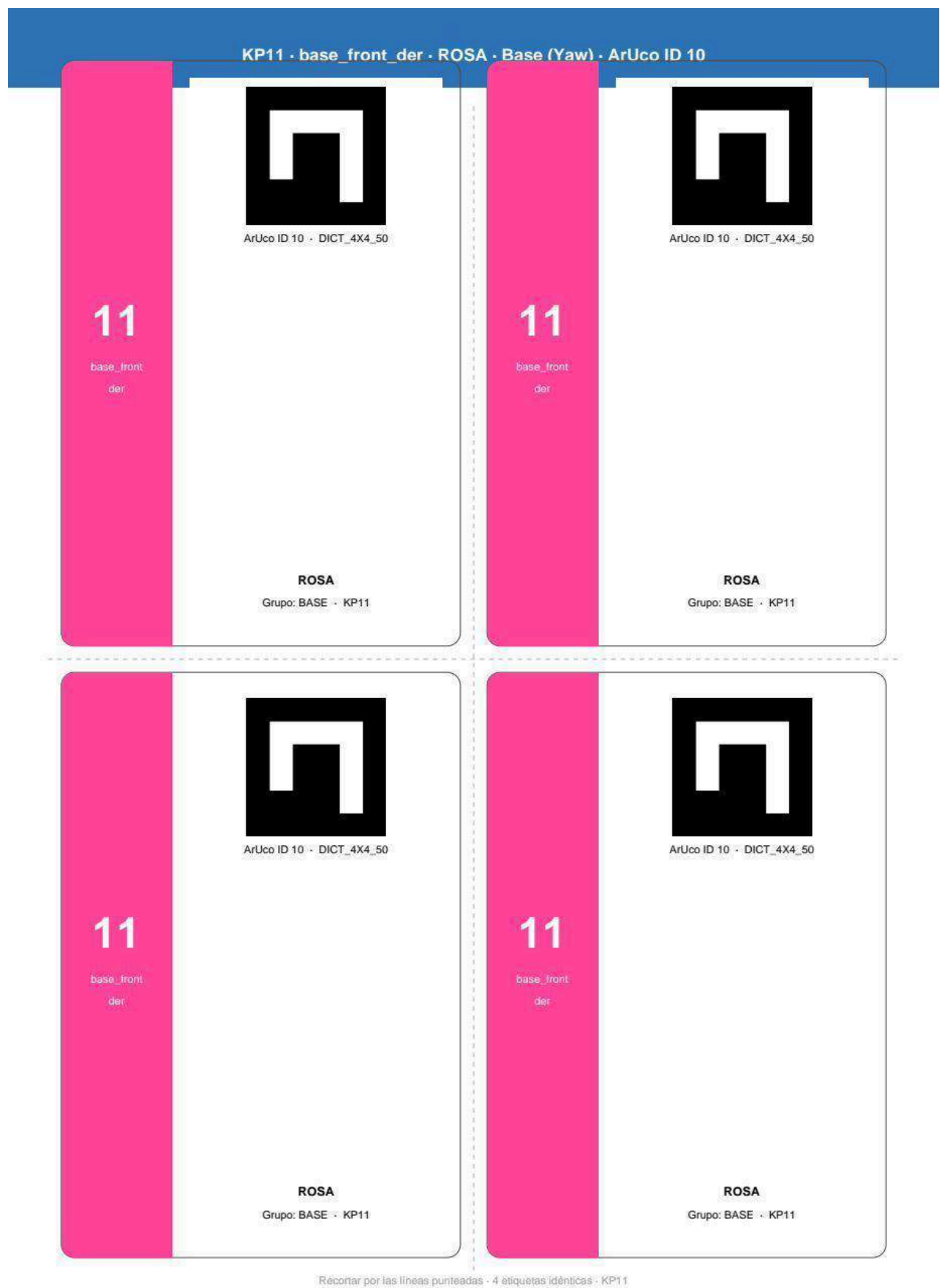


KP10 — MARRON — Grupo: BASE (Yaw) — ArUco ID 9

| KP10 · base_tras_der · MARRÓN · Base (Yaw) · ArUco ID 9 |                                                                                                                 |                        |                                                                                                                   |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------|
| 10<br>base_tras<br>der                                  | <br>ArUco ID 9 · DICT_4X4_50   | 10<br>base_tras<br>der | <br>ArUco ID 9 · DICT_4X4_50   |
|                                                         | <b>MARRÓN</b><br>Grupo: BASE · KP10                                                                             |                        | <b>MARRÓN</b><br>Grupo: BASE · KP10                                                                               |
| 10<br>base_tras<br>der                                  | <br>ArUco ID 9 · DICT_4X4_50 | 10<br>base_tras<br>der | <br>ArUco ID 9 · DICT_4X4_50 |
|                                                         | <b>MARRÓN</b><br>Grupo: BASE · KP10                                                                             |                        | <b>MARRÓN</b><br>Grupo: BASE · KP10                                                                               |

Recortar por las líneas punteadas · 4 etiquetas idénticas · KP10

KP11 — ROSA — Grupo: BASE (Yaw) — ArUco ID 10

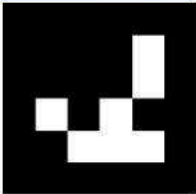


KP12 — MORADO — Grupo: BASE (Yaw) — ArUco ID 11

KP12 · base\_front\_izq · MORADO · Base (Yaw) · ArUco ID 11

12

base\_front  
izq



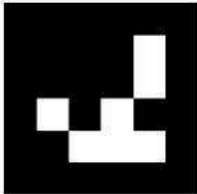
ArUco ID 11 · DICT\_4X4\_50

MORADO

Grupo: BASE · KP12

12

base\_front  
izq



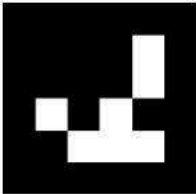
ArUco ID 11 · DICT\_4X4\_50

MORADO

Grupo: BASE · KP12

12

base\_front  
izq



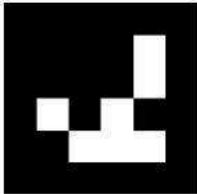
ArUco ID 11 · DICT\_4X4\_50

MORADO

Grupo: BASE · KP12

12

base\_front  
izq



ArUco ID 11 · DICT\_4X4\_50

MORADO

Grupo: BASE · KP12

Recortar por las líneas punteadas · 4 etiquetas idénticas · KP12

## Anexo II — Script de grabación de dataset: grabar\_dataset\_v2.py

Script para grabar vídeo con la Raspberry Pi Camera y extraer automáticamente dos conjuntos de imágenes: entrenamiento (para etiquetar en Roboflow y entrenar en Colab) y calibración (para el Hailo DFC). Incluye OSD en tiempo real, modo reparto equitativo y validación de duración.

### II.1 Resumen y uso

Uso básico:

```
Grabar 200 imágenes (100 entrenamiento + 100 calibración):
python3 grabar_dataset_v2.py --num-imagenes 200

Especificar carpeta de salida:
python3 grabar_dataset_v2.py --num-imagenes 300 --out /home/ai/mi_dataset

Ver todas las opciones:
python3 grabar_dataset_v2.py --help
```

### II.2 Cambios respecto a grabar\_dataset\_v1.py

| Cambio                 | Descripción                                                                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parámetro renombrado   | --objetivo (v1) → --num-imagenes (v2). Más descriptivo: indica el total de imágenes (se reparten 50/50 automáticamente).                                   |
| Nombrado de ficheros   | v1 usaba índice secuencial (entr_0001.jpg). v2 usa tiempo en segundos desde inicio (entr_00000123.450s.jpg). El orden alfabético coincide con el temporal. |
| Reparto equitativo     | Si el reparto 50/50 no es exacto, los fotogramas sobrantes van a entrenamiento (nunca a calibración).                                                      |
| Validación de duración | El script estima la duración mínima necesaria antes de grabar y avisa si parece insuficiente.                                                              |

### II.3 Estructura de salida generada

```
dataset_YYYYMMDD_HHMMSS/
 entrenamiento/
 entr_00000000.000s.jpg
 entr_00000002.000s.jpg
 ...
 calibracion/
 cali_00000001.000s.jpg
 cali_00000003.000s.jpg
 ...
```

Las carpetas entrenamiento/ y calibracion/ contienen imágenes JPG en orden temporal garantizado. Las imágenes de calibracion/ se usan en el paso de optimización del HEF (ver Anexo IV). Nunca usar las mismas imágenes para entrenamiento y calibración (Bug 3).

**⚠ IMPORTANTE:** las imágenes de calibracion/ de un grupo de dataset DEBEN usarse para compilar el HEF de un grupo DIFERENTE. Usar las mismas imágenes de entrenamiento y calibración resulta en raw\_conf=0 → 0 detecciones en producción (Bug 3 — calibración cruzada).



# Anexo III — Notebook de entrenamiento: giroscopio\_colab\_v4.ipynb

Notebook de Google Colab para entrenar el modelo YOLOv8n-Pose con el dataset del giroscopio. Soporta carga del dataset desde Google Drive (opción A) o descarga directa desde la API de Roboflow (opción B). El modelo entrenado se exporta en formato ONNX para su posterior compilación con el Hailo Dataflow Compiler (ver Anexo IV).

CORRECCIONES respecto al Colab v4 original: 1. Tabla de keypoints (Celda 0): nombres de posición INT corregidos. 2. Celda 5 (entrenamiento): eliminados `fliplr=0.5` y `flipud=0.3`. MOTIVO: los flips horizontales y verticales invierten el orden de keypoints simétricos, corrompiendo las anotaciones. Restricción arquitectónica del proyecto. 3. Celda 3: condición de ejecución simplificada.

## III.1 Instrucciones de uso

1. Menú → Entorno de ejecución → Cambiar tipo de entorno → GPU T4
2. Ejecutar las celdas en orden, una a una.
3. En la Celda 2 elegir el modo de carga del dataset (Drive o Roboflow API).
4. Si se usa Roboflow, rellenar las credenciales en la Celda 3.
5. Si la sesión se reinicia (timeout), volver a ejecutar desde la Celda 1 saltando la Celda 5 (entrenamiento ya realizado).

## III.2 Estructura de carpetas en Google Drive

```
Mi unidad/
└─ giroscopio_proyecto_g1/
 └─ dataset_g1/
 └─ Giroscopio_G1.v3i.yolov8 Dataset 1860.zip ← ZIP descargado de Roboflow (opción A)
 └─ runs/ ← se crea automáticamente al entrenar
 └─ giroscopio_v4/
 └─ weights/
 └─ best.pt
 └─ last.pt
 └─ (métricas, gráficas...)
```

## III.3 Estructura de keypoints (12KP)

| KP | Etiqueta Roboflow | Grupo / Eje / Nota                        |
|----|-------------------|-------------------------------------------|
| 1  | oct_tras_izq      | Octógono (Pitch)                          |
| 2  | oct_tras_der      | Octógono (Pitch)                          |
| 3  | oct_front_der     | Octógono (Pitch)                          |
| 4  | oct_front_izq     | Octógono (Pitch)                          |
| 5  | int_punta_arr_izq | Soporte INT (Roll) — punta U arriba-izq   |
| 6  | int_trav_arr_der  | Soporte INT (Roll) — travesaño arriba-der |
| 7  | int_punta_abj_izq | Soporte INT (Roll) — punta U abajo-izq    |
| 8  | int_trav_abj_der  | Soporte INT (Roll) — travesaño abajo-der  |
| 9  | base_tras_izq     | Base (Yaw)                                |
| 10 | base_tras_der     | Base (Yaw) — MARRÓN                       |
| 11 | base_front_der    | Base (Yaw) — ROSA                         |

| KP | Etiqueta Roboflow | Grupo / Eje / Nota |
|----|-------------------|--------------------|
| 12 | base_front_izq    | Base (Yaw)         |

### III.4 Celda 1 — Verificar GPU e instalar dependencias

Comprueba que Colab tiene GPU asignada e instala ultralytics y roboflow.

```
CELDA 1 — Verificar GPU e instalar dependencias
=====
Comprobamos que Colab nos ha asignado una GPU y que CUDA
está disponible. Sin GPU el entrenamiento tardaría días.
Instalamos ultralytics (que incluye YOLOv8) y roboflow
(necesario solo si se usa la opción de descarga por API).

import subprocess
result = subprocess.run(
 ['nvidia-smi', '--query-gpu=name,memory.total', '--format=csv,noheader'],
 capture_output=True, text=True
)
print('GPU:', result.stdout.strip())

!pip install ultralytics roboflow -q

import ultralytics, torch
print('Ultralytics:', ultralytics.__version__)
print('PyTorch:', torch.__version__)
print('CUDA disponible:', torch.cuda.is_available())

if not torch.cuda.is_available():
 print('\n⚠️ AVISO: No se detecta GPU.',
 'Ve a Entorno de ejecución → Cambiar tipo → GPU T4')
```

### III.5 Celda 2 — Montar Google Drive y cargar el dataset

Soporta dos opciones: A) dataset ZIP en Drive, B) descarga vía Roboflow API. En ambos casos el dataset queda en /content/dataset para las celdas siguientes.

```
CELDA 2 — Montar Google Drive y cargar el dataset
=====
OPCIÓN A — Desde Drive (recomendada si ya tienes el ZIP):
Descarga el dataset de Roboflow en formato YOLOv8 Pose,
coloca el ZIP en: giroscopio_proyecto_g1/dataset_g1/
y pon DATASET_SOURCE = 'drive' aquí abajo.
#
OPCIÓN B — Desde Roboflow API (descarga en tiempo real):
Pon DATASET_SOURCE = 'roboflow' y rellena las
credenciales en la Celda 3.

— CONFIGURACIÓN —
DATASET_SOURCE = 'drive' # 'drive' ó 'roboflow'

Carpeta raíz del proyecto en tu Drive
DRIVE_ROOT = '/content/drive/MyDrive/giroscopio_proyecto_g1'

Subcarpeta donde está el ZIP descargado de Roboflow
DATASET_ZIP_DIR = f'{DRIVE_ROOT}/dataset_g1'

Ruta local donde se extrae/descarga el dataset en Colab
DATASET_LOCAL = '/content/dataset'

from google.colab import drive
drive.mount('/content/drive')

import os, shutil
os.makedirs(DRIVE_ROOT, exist_ok=True)
```

```

print(f'Carpeta raíz del proyecto: {DRIVE_ROOT}')

if DATASET_SOURCE == 'drive':
 import zipfile
 print('\nContenido de dataset_g1:')
 for f in os.listdir(DATASET_ZIP_DIR):
 print(f' {repr(f)}')

 zips = [f for f in os.listdir(DATASET_ZIP_DIR) if f.lower().endswith('.zip')]
 if not zips:
 raise FileNotFoundError(
 f'No se encontró ningún .zip en {DATASET_ZIP_DIR}\n'
 f'Descarga el dataset de Roboflow (formato YOLOv8 Pose) y colócalo ahí.'
)
 zip_path = os.path.join(DATASET_ZIP_DIR, zips[0])
 print(f'\nZIP encontrado: {zip_path}')

 os.makedirs(DATASET_LOCAL, exist_ok=True)
 with zipfile.ZipFile(zip_path, 'r') as z:
 z.extractall(DATASET_LOCAL)
 print(f'Dataset extraído en {DATASET_LOCAL}')

 # Normalizar estructura: si el ZIP anida una subcarpeta, aplanar un nivel
 subdirs = [
 d for d in os.listdir(DATASET_LOCAL)
 if os.path.isdir(os.path.join(DATASET_LOCAL, d))
 and d not in ('train', 'valid', 'test')
]
 if subdirs and not os.path.exists(os.path.join(DATASET_LOCAL, 'train')):
 inner = os.path.join(DATASET_LOCAL, subdirs[0])
 print(f'Subcarpeta anidada detectada ({subdirs[0]}) → aplanando...')
 for item in os.listdir(inner):
 shutil.move(os.path.join(inner, item), DATASET_LOCAL)
 os.rmdir(inner)
 print('Estructura aplanada.')

elif DATASET_SOURCE == 'roboflow':
 print('Modo Roboflow API seleccionado. Continúa con la Celda 3.')

else:
 raise ValueError(
 f'DATASET_SOURCE debe ser "drive" o "roboflow", no "{DATASET_SOURCE}"'
)

if DATASET_SOURCE == 'drive':
 print('\nImágenes por split:')
 for split in ['train', 'valid', 'test']:
 img_dir = os.path.join(DATASET_LOCAL, split, 'images')
 n = len(os.listdir(img_dir)) if os.path.exists(img_dir) else 0
 print(f' {split}: {n} imágenes')

```

## III.6 Celda 3 — Descargar desde Roboflow API (solo Opción B)

**SALTAR esta celda si usas DATASET\_SOURCE = "drive" en la Celda 2.**

```

CELDA 3 - Descargar dataset desde Roboflow (solo Opción B)
=====
SALTAR ESTA CELDA si usas DATASET_SOURCE = "drive" en Celda 2.
Para obtener los datos de conexión:
1. Ve a roboflow.com → tu proyecto → Versions
2. Clic en la versión → Export → Format: YOLOv8 Pose
3. Selecciona 'Show download code' y copia los valores.

import os

Rellenar solo si DATASET_SOURCE == "roboflow"
API_KEY = "PEGA_AQUI_TU_API_KEY" # ← tu clave personal de Roboflow

```

```

WORKSPACE = "PEGA_AQUI_TU_WORKSPACE" # ej: joss-workspace-7orjh
PROJECT = "PEGA_AQUI_TU_PROYECTO" # ej: giroscopio-dron
VERSION = 1 # número de versión del dataset

DATASET_LOCAL = '/content/dataset'

Ejecutar descarga solo si el dataset no existe ya
if not os.path.exists(os.path.join(DATASET_LOCAL, 'data.yaml')):
 from roboflow import Roboflow
 rf = Roboflow(api_key=API_KEY)
 project = rf.workspace(WORKSPACE).project(PROJECT)
 version = project.version(VERSION)
 dataset = version.download('yolov8', location=DATASET_LOCAL)
 print(f'\nDataset descargado en {DATASET_LOCAL}')
 for split in ['train', 'valid', 'test']:
 img_dir = os.path.join(DATASET_LOCAL, split, 'images')
 n = len(os.listdir(img_dir)) if os.path.exists(img_dir) else 0
 print(f' {split}: {n} imágenes')
else:
 print('Dataset ya disponible en', DATASET_LOCAL, '- celda omitida.')

```

### III.7 Celda 4 — Verificar y corregir el data.yaml

Roboflow genera el data.yaml con rutas relativas que no funcionan en Colab. Esta celda fuerza rutas absolutas, verifica `kpt_shape=[12,3]` y comprueba que las anotaciones tienen el formato correcto (5 + 12×3 = 41 valores por objeto).

```

CELDA 4 – Verificar y corregir el data.yaml
=====
Roboflow genera el data.yaml con rutas relativas que no
funcionan en Colab. Aquí forzamos rutas absolutas y nos
aseguramos de que kpt_shape sea [12, 3].

import yaml, os

DATASET_LOCAL = '/content/dataset'
yaml_path = os.path.join(DATASET_LOCAL, 'data.yaml')

with open(yaml_path) as f:
 data = yaml.safe_load(f)

print('=== data.yaml ORIGINAL ===')
print(yaml.dump(data, default_flow_style=False))

Forzar rutas absolutas
data['path'] = DATASET_LOCAL
data['train'] = 'train/images'
data['val'] = 'valid/images'
data['test'] = 'test/images'

12 keypoints × 3 valores (x, y, visibilidad)
data['kpt_shape'] = [12, 3]

Una sola clase
data['nc'] = 1
data['names'] = ['giroscopio']

with open(yaml_path, 'w') as f:
 yaml.dump(data, f, default_flow_style=False)

print('=== data.yaml CORREGIDO ===')
with open(yaml_path) as f:
 print(f.read())

print('=== VERIFICACIÓN DE RUTAS ===')
for split in ['train', 'valid']:
 img_path = os.path.join(DATASET_LOCAL, split, 'images')
 lbl_path = os.path.join(DATASET_LOCAL, split, 'labels')

```

```

n_img = len(os.listdir(img_path)) if os.path.exists(img_path) else 0
n_lbl = len(os.listdir(lbl_path)) if os.path.exists(lbl_path) else 0
status = '✓' if n_img > 0 and n_lbl > 0 else '⚠'
print(f' {status} {split}: {n_img} imágenes, {n_lbl} labels')

Comprobar formato de un label de muestra
Formato esperado: 5 (bbox: clase, cx, cy, w, h) + 12×3 (x, y, v) = 41 valores
print('\n=== MUESTRA DE LABEL ===')
lbl_dir = os.path.join(DATASET_LOCAL, 'train', 'labels')
sample = sorted(os.listdir(lbl_dir))[0]
with open(os.path.join(lbl_dir, sample)) as f:
 line = f.readline().strip()
values = line.split()
N_KP = 12
EXPECTED = 5 + N_KP * 3 # = 41
print(f' Archivo: {sample}')
print(f' Valores: {len(values)} (esperados: {EXPECTED})')
if len(values) == EXPECTED:
 print(f' ✓ Formato correcto')
else:
 print(f' ⚠ ERROR: se esperan {EXPECTED} valores, hay {len(values)}')
 print(f' Comprueba que el dataset exportado de Roboflow tiene 12 keypoints')

```

## III.8 Celda 5 — Entrenamiento

**RESTRICCIÓN DE AUGMENTACIÓN: NO usar fliplr ni flipud en este proyecto. Los flips horizontales y verticales invierten el orden de los keypoints simétricos, corrompiendo las anotaciones. Solo se usan aumentaciones que preservan el orden espacial: hsv\_h, hsv\_s, hsv\_v, degrees (rotación < 20°), scale.**

```

CELDA 5 - Entrenamiento
=====
YOLOv8n-pose es la variante más ligera (nano) del modelo
de pose de Ultralytics. Infiere automáticamente el número
de keypoints desde el kpt_shape del data.yaml.
#
Parámetros clave:
epochs - número de pasadas completas sobre el dataset
imgsz - resolución de entrada (640×640 px)
batch - imágenes por paso de optimización
(ajustar si hay errores de memoria, bajar a 8)
patience - early stopping: para si no mejora en N épocas
project - carpeta de resultados (apunta a Drive para
no perder nada si la sesión se reinicia)
#
AUGMENTACIÓN PERMITIDA (no incluir fliplr ni flipud):
hsv_h/s/v - variación de color (HSV)
degrees - rotación < 20° (no invierte el orden de KPs)
scale - zoom in/out
#
PROHIBIDO: fliplr, flipud, mosaic con flip.
MOTIVO: los flips invierten el orden de keypoints simétricos,
corrompiendo las anotaciones etiquetadas en Roboflow.

from ultralytics import YOLO
import torch, os

DRIVE_ROOT = '/content/drive/MyDrive/giroscopio_proyecto_g1'
DATASET_LOCAL = '/content/dataset'
RUN_NAME = 'giroscopio_v4'

print(f'GPU: {torch.cuda.get_device_name(0)}')
print(f'VRAM disponible: {torch.cuda.get_device_properties(0).total_memory/1024**3:.1f} GB')

model = YOLO('yolov8n-pose.pt')

results = model.train(

```

```

data = os.path.join(DATASET_LOCAL, 'data.yaml'),
epochs = 150,
imgsz = 640,
batch = 16,
device = 0,
name = RUN_NAME,
project = os.path.join(DRIVE_ROOT, 'runs'), # ← guardado directo en Drive
Augmentación – NO incluir flipplr ni flipud (ver nota arriba)
hsv_h = 0.015,
hsv_s = 0.7,
hsv_v = 0.4,
degrees = 15,
scale = 0.5,
patience= 30,
save = True,
verbose = True,
)

print('\n=== RESULTADO DEL ENTRENAMIENTO ===')
print(f'mAP50 (pose): {results.results_dict["metrics/mAP50(P)"]:.4f}')
print(f'mAP50-95 (pose): {results.results_dict["metrics/mAP50-95(P)"]:.4f}')
print(f'mAP50 (box): {results.results_dict["metrics/mAP50(B)"]:.4f}')
print(f'mAP50-95 (box): {results.results_dict["metrics/mAP50-95(B)"]:.4f}')
print(f'Fitness combinado: {results.fitness:.4f}')
print(f'\nPesos guardados en: {DRIVE_ROOT}/runs/{RUN_NAME}/weights/')

```

### III.9 Celda 6 — Verificar el modelo entrenado

Antes de exportar, comprueba que el modelo detecta el giroscopio: (1) tiene 12 keypoints, (2) no detecta nada en imagen negra, (3) baja confianza en imagen de ruido, (4) alta confianza en imagen real.

```

CELDA 6 – Verificar el modelo entrenado
=====
import numpy as np
from ultralytics import YOLO
import os

DRIVE_ROOT = '/content/drive/MyDrive/giroscopio_proyecto_g1'
RUN_NAME = 'giroscopio_v4'
DATASET_LOCAL = '/content/dataset'

best_pt = os.path.join(DRIVE_ROOT, 'runs', RUN_NAME, 'weights', 'best.pt')
model = YOLO(best_pt)

1. Comprobar arquitectura
kpt_shape = model.model.kpt_shape
print(f'kpt_shape del modelo: {kpt_shape}')
print(f'✓ 12 keypoints OK' if kpt_shape == [12, 3] else f'⚠ Se esperaba [12, 3]')

2. Imagen negra (no debe detectar nada)
img_black = np.zeros((640, 640, 3), dtype=np.uint8)
r1 = model(img_black, verbose=False)
print(f'\nImagen negra → detecciones: {len(r1[0].boxes)} (esperado: 0)')

3. Imagen de ruido (confianza baja)
img_noise = np.random.randint(0, 255, (640, 640, 3), dtype=np.uint8)
r2 = model(img_noise, verbose=False)
conf_noise = float(r2[0].boxes.conf.max()) if len(r2[0].boxes) else 0
print(f'Imagen ruido → detecciones: {len(r2[0].boxes)},
 conf máx: {conf_noise:.4f} (esperado: < 0.3)')

4. Imagen real de validación
import random
val_imgs = os.listdir(os.path.join(DATASET_LOCAL, 'valid', 'images'))
sample_img = os.path.join(DATASET_LOCAL, 'valid', 'images', random.choice(val_imgs))
r3 = model(sample_img, verbose=False)
conf_real = float(r3[0].boxes.conf.max()) if len(r3[0].boxes) else 0
print(f'Imagen real → detecciones: {len(r3[0].boxes)},
 conf máx: {conf_real:.4f} (esperado: > 0.3)')

```



```

if len(r3[0].boxes) > 0 and r3[0].keypoints is not None:
 n_kp = r3[0].keypoints.xy.shape[1]
 print(f'Keypoints/inst → {n_kp} (esperado: 12)')
 print('✓' if n_kp == 12 else '⚠ Número de keypoints incorrecto')

if conf_real > 0.3:
 print('✅ El modelo detecta el giroscopio correctamente. Proceder con exportación.')
else:
 print('⚠ El modelo NO detecta bien. Revisar el dataset antes de exportar.')

```

### III.10 Celda 7 — Exportar a ONNX

El pipeline Hailo DFC requiere ONNX con salidas SEPARADAS (no tensor fusionado). Opset 11, simplify=True, dynamic=False. Se verifica que el ONNX tenga ≥9 salidas.

```

CELDA 7 – Exportar a ONNX (formato para Hailo)
=====
IMPORTANTE: ejecutar solo si la Celda 6 confirmó que el
modelo funciona correctamente.
#
El pipeline Hailo DFC requiere un ONNX con salidas SEPARADAS
(no un único tensor fusionado). Para ello:
- opset 11 (máxima compatibilidad con Hailo DFC)
- simplify=True (elimina operaciones redundantes)
- dynamic=False (shapes fijas, requerido por Hailo)

from ultralytics import YOLO
import shutil, os

DRIVE_ROOT = '/content/drive/MyDrive/giroscopio_proyecto_g1'
RUN_NAME = 'giroscopio_v4'

best_pt = os.path.join(DRIVE_ROOT, 'runs', RUN_NAME, 'weights', 'best.pt')
model = YOLO(best_pt)

onnx_path = model.export(
 format = 'onnx',
 imgsz = 640,
 opset = 11,
 simplify = True,
 dynamic = False,
)
print(f'ONNX exportado en: {onnx_path}')

Verificar: con 12 KP y salidas separadas debe tener ≥ 9 tensores
(3 escalas × bbox + conf + kp). Si hay solo 1 → no sirve para Hailo.
import onnxruntime as ort
sess = ort.InferenceSession(str(onnx_path))
outputs = sess.get_outputs()
print(f'\n=== VERIFICACIÓN DEL ONNX ===')
print(f'Número de salidas: {len(outputs)}')
for o in outputs:
 print(f' {o.name} shape={o.shape}')

if len(outputs) == 1:
 print('\n⚠ ONNX con salida fusionada – NO sirve para Hailo')
 print(' Solución: !pip install ultralytics --upgrade -q y reexportar')
elif len(outputs) >= 9:
 print('\n✅ ONNX con salidas separadas – correcto para Hailo')
else:
 print(f'\n? ONNX con {len(outputs)} salidas – verificar manualmente')

Copiar archivos finales a la carpeta raíz del proyecto
dest_onnx = os.path.join(DRIVE_ROOT, f'giroscopio_pose_best_n_{RUN_NAME}.onnx')
dest_pt = os.path.join(DRIVE_ROOT, f'giroscopio_pose_best_n_{RUN_NAME}.pt')
shutil.copy(str(onnx_path), dest_onnx)
shutil.copy(best_pt, dest_pt)
print(f'\nArchivos guardados en {DRIVE_ROOT}:')

```

```
print(f' {os.path.basename(dest_onnx)}')
print(f' {os.path.basename(dest_pt)}')
print('\n✅ Proceso completo.')
```

## Anexo IV — Tutorial de compilación del modelo para Hailo-8

Este anexo documenta el proceso completo para compilar un modelo YOLOv8n-Pose exportado a ONNX al formato HEF (Hailo Executable Format) que el chip Hailo-8 puede ejecutar directamente. El proceso usa el Hailo Dataflow Compiler (DFC) v3.33.0 dentro de un contenedor Docker. Se incluyen instrucciones para Ubuntu y Windows.

✓ Estado de validación: el flujo Ubuntu está verificado en sesión real (abril 2026). El flujo Windows ha sido verificado de forma independiente por un alumno del proyecto y confirmado como correcto. Dentro del contenedor Docker, los comandos son idénticos en ambos sistemas — solo cambia la instalación de Docker y la sintaxis del volumen.

### IV.1 Compilación en Ubuntu (verificado)

#### IV.1.1 Instalar Docker en Ubuntu

```
Actualizar paquetes
sudo apt update && sudo apt upgrade -y
Instalar dependencias
sudo apt install -y ca-certificates curl gnupg lsb-release
Añadir clave GPG oficial de Docker
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
Añadir repositorio Docker
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
Instalar Docker Engine
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
Añadir usuario al grupo docker (evita usar sudo cada vez)
sudo usermod -aG docker $USER
Verificar (reiniciar sesión si es la primera vez)
docker --version
```

! Reinicio de sesión necesario: después de hacer usermod, cierra sesión y vuelve a entrar. Sin esto necesitarás poner sudo delante de cada comando docker.

#### IV.1.2 Descargar e instalar la imagen Hailo Docker

La imagen Docker no está en Docker Hub. Hay que descargarla del portal oficial: <https://hailo.ai/developer-zone/> (requiere registro gratuito). Buscar "AI Software Suite", descargar la versión 2025-10.

```
Cargar la imagen (puede tardar varios minutos, pesa ~17 GB)
docker load < hailo8_ai_sw_suite_2025-10.tar.gz
Verificar que se cargó correctamente
docker images
Salida esperada:
REPOSITORY TAG IMAGE ID SIZE
hailo8_ai_sw_suite_2025-10 1 3a94fa154926 17.5GB
```

⚠ El tag es :1, no :latest. Usar siempre hailo8\_ai\_sw\_suite\_2025-10:1 (con el tag :1). Sin el tag, Docker busca :latest en internet y falla con "pull access denied".

#### IV.1.3 Preparar el workspace

```
mkdir -p /home/nutshell/Documentos/hailo_workspace/calib
```

```
Estructura resultante:
hailo_workspace/
giroscopio_pose_best_n.onnx ← modelo exportado desde Colab
calib/ ← 100-300 imágenes JPG/PNG de calibración
imagen_001.jpg
imagen_002.jpg
...
```

**¡ Sobre las imágenes de calibración: usar las imágenes de la carpeta calibracion/ generada por grabar\_dataset\_v2.py. NUNCA usar las mismas imágenes que se usaron para entrenar el modelo (Bug 3 — calibración cruzada). Con 100-300 imágenes es suficiente.**

## IV.1.4 Arrancar el contenedor Docker

```
docker run -it --rm \
 -v /home/nutshell/Documentos/hailo_workspace:/workspace \
 hailo8_ai_sw_suite_2025-10:1 bash
El prompt cambia a:
(hailo_virtualenv) hailo@d07f0c1431f6:/local/workspace$
Ir siempre a ~ y verificar el workspace
cd ~
ls /workspace # debe mostrar el .onnx y la carpeta calib/
```

**⚠ REGLA DE ORO: trabajar siempre desde ~. /workspace es de solo lectura para el usuario hailo. Ejecutar SIEMPRE desde ~ y apuntar a /workspace solo para leer archivos.**

## IV.1.5 Convertir imágenes de calibración a .npz

```
python3 << 'EOF'
import os, numpy as np
from PIL import Image
calib_dir = '/workspace/calib/'
out_dir = os.path.expanduser('~/.calib_npy/')
os.makedirs(out_dir, exist_ok=True)
files = [f for f in os.listdir(calib_dir) if f.lower().endswith(('.jpg','.png'))]
print(f'Convirtiendo {len(files)} imagenes...')
for i, fname in enumerate(files):
 img = Image.open(os.path.join(calib_dir, fname)).convert('RGB')
 img = img.resize((640, 640))
 np.save(os.path.join(out_dir, f'calib_{i:04d}.npz'), np.array(img, dtype=np.float32))
print(f'Listo: {len(files)} archivos .npz en {out_dir}')
EOF
Salida esperada:
Convirtiendo 299 imagenes...
Listo: 299 archivos .npz en /home/hailo/.calib_npy/
```

## IV.1.6 Parsear: .onnx → .har

Es crítico especificar los 9 nodos finales correctos (capas Conv de la cabeza de detección). Sin ellos el compilador falla con "No valid partition found".

```
cd ~
hailo parser onnx /workspace/giroscopio_pose_best_n.onnx \
 --net-name giroscopio \
 --end-node-names \
 /model.22/cv2.0/cv2.0.2/Conv \
 /model.22/cv3.0/cv3.0.2/Conv \
 /model.22/cv4.0/cv4.0.2/Conv \
 /model.22/cv2.1/cv2.1.2/Conv \
 /model.22/cv3.1/cv3.1.2/Conv \
 /model.22/cv4.1/cv4.1.2/Conv \
 /model.22/cv2.2/cv2.2.2/Conv \
 /model.22/cv3.2/cv3.2.2/Conv \
 /model.22/cv4.2/cv4.2.2/Conv
Salida esperada:
[info] Translation completed on ONNX model giroscopio
[info] Saved HAR to: /home/hailo/giroscopio.har
```

```
ls -lh ~/giroscopio.har # debe mostrar ~13 MB
```

### IV.1.7 Crear el script de optimización (.alls) y optimizar

```
Crear el script .alls
cat > ~/giroscopio.alls << 'EOF'
normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])
change_output_activation(conv71, sigmoid)
change_output_activation(conv58, sigmoid)
change_output_activation(conv44, sigmoid)
pre_quantization_optimization(equalization, policy=disabled)
quantization_param(output_layer3, precision_mode=a16_w16)
quantization_param(output_layer6, precision_mode=a16_w16)
quantization_param(output_layer9, precision_mode=a16_w16)
post_quantization_optimization(finetune, policy=enabled, learning_rate=0.00015)
EOF

Lanzar optimize (~13 minutos)
hailo optimize ~/giroscopio.har \
 --hw-arch hailo8 \
 --calib-set-path ~/calib_npy/ \
 --model-script ~/giroscopio.alls
Salida al terminar:
[info] Model Optimization is done
[info] Saved HAR to: /home/hailo/giroscopio_optimized.har
```

### IV.1.8 Compilar: .har optimizado → .hef

```
Último paso. Genera el binario final para el chip Hailo-8. (~8 min)
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8
Salida al terminar:
[info] Successful Mapping (allocation time: 8m 6s)
[info] Compiling kernels of giroscopio_context_0...
[info] Compiling kernels of giroscopio_context_1...
[info] Successful Compilation (compilation time: 8s)
[info] Saved HEF to: /home/hailo/giroscopio.hef
```

### IV.1.9 Guardar archivos y salir del contenedor

**⚠ NO hagas exit hasta ver los archivos en /workspace. El comando exit con --rm es irreversible: todo lo que esté en ~ desaparece para siempre.**

```
Verificar tamaños
ls -lh ~/giroscopio.hef ~/giroscopio_optimized.har ~/giroscopio_compiled.har
Esperado:
7.1M giroscopio.hef ← ARCHIVO FINAL
62M giroscopio_optimized.har
69M giroscopio_compiled.har
Copiar a /workspace (visible en Ubuntu como hailo_workspace/)
cp ~/giroscopio.hef /workspace/
cp ~/giroscopio_optimized.har /workspace/
cp ~/giroscopio_compiled.har /workspace/
cp ~/giroscopio.alls /workspace/
Verificar que llegaron
ls -lh /workspace/*.hef /workspace/*.har
SOLO cuando veas los archivos en /workspace → salir
exit
```

### IV.1.10 Copiar el .hef a la Raspberry Pi 5

```
Sustituir <IP-RASPBERRY> por la IP real de tu RPi5
scp /home/nutshell/Documentos/hailo_workspace/giroscopio.hef \
 pi@<IP-RASPBERRY>:~/
Alternativa: copiar a pendrive USB y montar en la RPi
```

## IV.1.11 Chuleta — secuencia completa de una vez

```
— ARRANCAR DOCKER —————
docker run -it --rm \
 -v /home/nutshell/Documentos/hailo_workspace:/workspace \
 hailo8_ai_sw_suite_2025-10:1 bash

— DENTRO DEL CONTENEDOR —————
cd ~
PASO 1: Convertir imágenes de calibración a .npz
python3 << 'PYEOF'
import os, numpy as np
from PIL import Image
calib_dir='/workspace/calib/'; out_dir=os.path.expanduser('~/.calib_npy/')
os.makedirs(out_dir, exist_ok=True)
files=[f for f in os.listdir(calib_dir) if f.lower().endswith(('.jpg','.png'))]
for i,f in enumerate(files):
 img=Image.open(os.path.join(calib_dir,f)).convert('RGB').resize((640,640))
 np.save(os.path.join(out_dir,f'calib_{i:04d}.npz'),np.array(img,dtype=np.float32))
print(f'OK: {len(files)} archivos .npz')
PYEOF

PASO 2: Parsear
hailo parser onnx /workspace/giroscopio_pose_best_n.onnx \
 --net-name giroscopio --end-node-names \
 /model.22/cv2.0/cv2.0.2/Conv /model.22/cv3.0/cv3.0.2/Conv \
 /model.22/cv4.0/cv4.0.2/Conv /model.22/cv2.1/cv2.1.2/Conv \
 /model.22/cv3.1/cv3.1.2/Conv /model.22/cv4.1/cv4.1.2/Conv \
 /model.22/cv2.2/cv2.2.2/Conv /model.22/cv3.2/cv3.2.2/Conv \
 /model.22/cv4.2/cv4.2.2/Conv

PASO 3: Crear .alls
cat > ~/giroscopio.alls << 'EOF'
normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])
change_output_activation(conv71, sigmoid)
change_output_activation(conv58, sigmoid)
change_output_activation(conv44, sigmoid)
pre_quantization_optimization(equalization, policy=disabled)
quantization_param(output_layer3, precision_mode=a16_w16)
quantization_param(output_layer6, precision_mode=a16_w16)
quantization_param(output_layer9, precision_mode=a16_w16)
post_quantization_optimization(finetune, policy=enabled, learning_rate=0.00015)
EOF

PASO 4: Optimizar (~13 min)
hailo optimize ~/giroscopio.har \
 --hw-arch hailo8 --calib-set-path ~/.calib_npy/ --model-script ~/giroscopio.alls

PASO 5: Compilar (~8 min)
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8

PASO 6: Verificar y copiar (NO hacer exit hasta ver archivos en /workspace)
ls -lh ~/giroscopio.hef ~/giroscopio_optimized.har
cp ~/giroscopio.hef ~/giroscopio_optimized.har ~/giroscopio_compiled.har ~/giroscopio.alls
/workspace/
ls -lh /workspace/*.hef /workspace/*.har
— SOLO CUANDO VEAS LOS ARCHIVOS EN /workspace —————
exit

— DE VUELTA EN UBUNTU: copiar a Raspberry Pi 5 —————
scp /home/nutshell/Documentos/hailo_workspace/giroscopio.hef pi@<IP>:~/
```

## IV.1.12 Tabla de errores y soluciones (Ubuntu)

| Error              | Causa                          | Solución                                     |
|--------------------|--------------------------------|----------------------------------------------|
| pull access denied | Tag :latest no existe en local | Usar siempre<br>hailo8_ai_sw_suite_2025-10:1 |



| Error                                  | Causa                                | Solución                                               |
|----------------------------------------|--------------------------------------|--------------------------------------------------------|
| Permission denied<br>pyhailort.log     | /workspace es read-only              | Ejecutar SIEMPRE desde ~                               |
| StopIteration<br>numpy_dir_to_dataset  | calib/ contenía JPGs, no .npy        | Convertir con el script Python antes del optimize      |
| Couldn't detect<br>CalibrationDataType | Carpeta .npy vacía o mal generada    | Verificar ls ~/calib_npy/ — que hay archivos .npy      |
| --calib-path not recognized            | Parámetro incorrecto                 | Usar --calib-set-path (no --calib-path)                |
| Model requires quantized weights       | Se compiló el .har sin optimizar     | Ejecutar hailo optimize ANTES de hailo compiler        |
| No valid partition found               | Nodos finales del parser incorrectos | Usar los 9 nodos cv2/cv3/cv4 Conv con --end-node-names |
| exit prematuro, archivos perdidos      | --rm elimina todo al salir           | Verificar ls /workspace/*.hef antes de hacer exit      |

### IV.1.13 Archivos generados

| Archivo                  | Tamaño   | Descripción                                        |
|--------------------------|----------|----------------------------------------------------|
| giroscopio.har           | ~13 MB   | Modelo parseado (FP32). Salida del parser.         |
| giroscopio.all           | ~512 B   | Script de optimización. Guardar para reutilizar.   |
| calib_npy/               | variable | Arrays de calibración. Se regenera en cada sesión. |
| giroscopio_optimized.har | ~62 MB   | Modelo cuantizado INT8 con fine-tuning.            |
| giroscopio_compiled.har  | ~69 MB   | Modelo compilado con mapeo al chip.                |
| giroscopio.hef           | 7.1 MB   | ARCHIVO FINAL. El único que se carga en la RPi5.   |

## IV.2 Compilación en Windows (Docker Desktop) — verificado

✓ Proceso verificado por un alumno del proyecto de forma independiente. Los comandos dentro del contenedor Docker son idénticos a los de Ubuntu (secciones IV.1.5 a IV.1.9). Las diferencias son: instalación de Docker Desktop + WSL2, carga de imagen con -i, y sintaxis de la ruta del volumen con barras /.

### IV.2.0 Obtener el modelo .onnx desde Google Colab

```
Método A: descarga directa (disco efímero — se pierde al cerrar Colab)
En Colab: panel lateral izquierdo → carpeta → clic derecho → Descargar

Método B: guardar en Google Drive (recomendado)
from google.colab import drive
drive.mount('/content/drive')
import shutil
shutil.copy('runs/pose/train/weights/best.onnx',
 '/content/drive/MyDrive/giroscopio_pose_best_n.onnx')
Luego: drive.google.com → clic derecho → Descargar → guardar en hailo_workspace\
```

### IV.2.1 Instalar WSL2

```
Abrir PowerShell como Administrador:
wsl --install
Si WSL ya estaba instalado, actualizar a versión 2:
wsl --set-default-version 2
```

```
Reiniciar el ordenador cuando se solicite.
```

## IV.2.2 Instalar Docker Desktop

Descargar Docker Desktop desde <https://www.docker.com/products/docker-desktop/> y ejecutar el instalador. En el instalador, asegurarse de que "Use WSL 2 instead of Hyper-V" está MARCADO. Tras instalar, abrir Docker Desktop y esperar a que arranque el motor (icono de la ballena en la barra de tareas).

```
En PowerShell - verificar que Docker funciona:
docker --version
Salida esperada: Docker version 26.x.x, build ...
```

⚠ Docker Desktop debe estar abierto. A diferencia de Ubuntu donde Docker corre como servicio del sistema, en Windows Docker Desktop debe estar en ejecución. Si ves "Cannot connect to the Docker daemon", abre Docker Desktop y espera.

## IV.2.3 Cargar la imagen Hailo en Docker

```
En PowerShell o CMD (navegar primero a la carpeta de descarga)
cd C:\Users\TuUsuario\Downloads
Cargar la imagen (-i en lugar de < para Windows)
docker load -i hailo8_ai_sw_suite_2025-10.tar.gz
Verificar
docker images
```

## IV.2.4 Preparar el workspace en Windows

```
En PowerShell:
New-Item -ItemType Directory -Force -Path C:\Users\TuUsuario\hailo_workspace
New-Item -ItemType Directory -Force -Path C:\Users\TuUsuario\hailo_workspace\calib
Copiar el .onnx y las imágenes de calibración a hailo_workspace\
```

## IV.2.5 Arrancar el contenedor Docker en Windows

```
En PowerShell (carácter de continuación: acento grave `)
docker run -it --rm `
 -v C:/Users/TuUsuario/hailo_workspace:/workspace `
 hailo8_ai_sw_suite_2025-10:1 bash

En CMD (carácter de continuación: circunflejo ^)
docker run -it --rm ^
 -v C:/Users/TuUsuario/hailo_workspace:/workspace ^
 hailo8_ai_sw_suite_2025-10:1 bash

Nota: usar barras / en las rutas cuando se pasan a Docker, NO barras \
```

i Una vez dentro del contenedor, el proceso es idéntico a Ubuntu. Los pasos IV.1.5 a IV.1.9 son exactamente iguales. El entorno dentro del contenedor es Linux independientemente del sistema operativo del host.

## IV.2.6 Copiar el .hef a la Raspberry Pi 5 desde Windows

```
Método A - SCP desde PowerShell (Windows 10/11 incluyen OpenSSH)
scp C:\Users\TuUsuario\hailo_workspace\giroscopio.hef pi@<IP-RASPBERRY>:~/

Método B - WinSCP (interfaz gráfica)
Descargar desde: https://winscp.net/
Conectar: Protocol=SFTP, Host=<IP>, User=pi, Password=<password>

Método C - Pendrive USB (si no hay red)
Copiar giroscopio.hef al pendrive
En la RPi: cp /media/pi/<pendrive>/giroscopio.hef ~/
```

#### IV.2.7 Tabla de errores adicionales en Windows

| Error                               | Causa                          | Solución                                      |
|-------------------------------------|--------------------------------|-----------------------------------------------|
| pull access denied                  | Tag :latest no existe en local | Usar hailo8_ai_sw_suite_2025-10:1             |
| Cannot connect to the Docker daemon | Docker Desktop no está abierto | Abrir Docker Desktop y esperar                |
| docker: command not found           | Docker Desktop no en PATH      | Reiniciar PowerShell tras instalar            |
| invalid volume specification        | Ruta Windows mal formateada    | Usar barras /: C:/Users/... no C:\\Users\\... |
| Permission denied<br>pyhailort.log  | /workspace es read-only        | Ejecutar desde ~, no desde /workspace         |
| exit prematuro, archivos perdidos   | --rm elimina todo al salir     | Verificar ls /workspace/*.hef antes de exit   |

# Anexo V — Script de inferencia: giroscopio\_12kp\_v3h.py

Script principal de inferencia en tiempo real. Detecta el giroscopio mediante 12 keypoints YOLOv8n-Pose compilados para Hailo-8 y estima Roll, Pitch y Yaw con panel visual de actitud (arcos, péndulo, brújula). Versión v3h — 08/06/2026.

## V.1 Resumen de uso

| Modo                           | Comando                                                                                             |
|--------------------------------|-----------------------------------------------------------------------------------------------------|
| Con cámara (Raspberry Pi)      | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef /home/ai/giroscopio_v3a/p410/giroscopio.hef         |
| Con vídeo grabado              | python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --video /home/ai/resultado_p410.mp4          |
| Guardar vídeo de salida        | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --save /home/ai/resultado_v3h.mp4 |
| Sin Yaw (recomendado demos)    | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --hide-yaw                        |
| Detecciones dobles → bajar IoU | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --iou-thresh 0.25                 |
| Zona muerta visual             | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --dead-zone 10                    |
| Delta a 5 segundos             | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --delta-t 5.0                     |
| Debug con log de KPs           | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --verbose                         |
| Giroscopio girado 180° en Z    | DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef ../giroscopio.hef --flip-pitch --flip-yaw           |

## V.2 Código fuente completo

NOTA sobre los comentarios de color de KP10 y KP11 en el código: los comentarios del script dicen "KP10 rosa / KP11 marrón" pero los valores BGR son correctos y consistentes con las etiquetas físicas (KP10=marrón, KP11=rosa). Solo los nombres en los comentarios están intercambiados — el comportamiento del script es correcto.

```
#!/usr/bin/env python3
"""
giroscopio_12kp_v3h.py - Detección y estimación de ángulos
=====
Proyecto de innovación educativa - IES Politécnico Jesús Marín, Málaga
Familia Electricidad y Electrónica - Junta de Andalucía

PROPÓSITO

Detectar un giroscopio físico mediante 12 keypoints (YOLOv8n-Pose compilado
para Hailo-8) y estimar en tiempo real sus ángulos de orientación:
Roll, Pitch y Yaw.

CÓMO USAR ESTE SCRIPT

Con cámara (Raspberry Pi):
DISPLAY=:0 python3 giroscopio_12kp_v3h.py --hef /home/ai/giroscopio_v3a/p410/giroscopio.hef

Con vídeo grabado (PC o RPi sin cámara):
python3 giroscopio_12kp_v3h.py \
```

```

--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--video /home/ai/resultado_p410.mp4

Guardar vídeo de salida:
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--save /home/ai/resultado_v3h_p410.mp4

Ocultar Yaw del panel (no fiable, recomendado para demostraciones):
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--hide-yaw

Si aparecen detecciones dobles, bajar el umbral IoU:
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--iou-thresh 0.25

Recuperar más KPs en posiciones extremas (puede añadir algo de ruido):
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--kp-thresh 0.05

Con ventana de delta a 5 segundos (por defecto: 2s):
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--delta-t 5.0

Debug extra + diagnóstico de wrap-around del delta (v3g FIX 4):
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--verbose

Giroscopio montado girado 180° en eje Z:
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--flip-pitch --flip-yaw

Combinado: demo sin Yaw, umbral IoU ajustado:
DISPLAY=:0 python3 giroscopio_12kp_v3h.py \
--hef /home/ai/giroscopio_v3a/p410/giroscopio.hef \
--hide-yaw --iou-thresh 0.25

```

#### HISTORIA DE VERSIONES Y PROBLEMAS RESUELTOS

-----

- v1 – giroscopio\_12kp\_gX.py  
 BUG: tensores de keypoints (conv45/59/72) pedidos como UINT8.  
 El HEF los compila como UINT16. HailoRT aborta el hilo de inferencia.  
 Resultado: 0 detecciones siempre.
- v2 – fix UINT16  
 FIX: pedir conv45/59/72 como FormatType.UINT16.  
 NUEVO BUG: sigmoid aplicado a tensores de conf ya en formato PROB  
 (zp=0, scale=1/255). sigmoid(0) = 0.5 → ~1500 falsas detecciones.
- v3 – giroscopio\_12kp\_g1\_0det\_v3.py  
 FIX: detectar tensores de conf en formato PROB y saltar sigmoid.  
 NUEVO BUG: dataset de calibración del HEF = mismo dataset de entrenamiento  
 → compilador calibra rango optimista → tensor de confianza devuelve raw=0  
 en producción → 0 detecciones.
- v4\_det – giroscopio\_12kp\_g1\_v4\_det.py  
 FIX: CONF ALREADY PROB hardcodeado con valores reales del HEF.  
 FIX: CONF THRESH bajado a 0.01 para diagnóstico.  
 NUEVO: logging exhaustivo siempre activo.  
 NUEVO: dump automático del tensor RAW tras 60 frames sin detección.  
 NUEVO: soporte --video para pruebas sin cámara.  
 RESULTADO: confirmó raw conf = TODO CEROS → problema de compilación.  
 FIX DEFINITIVO: recompilar HEF con calibración cruzada. Resuelto 16/04/2026.
- v5 – giroscopio\_12kp\_v5.py  
 Incorpora todos los fixes anteriores.  
 NUEVO: panel lateral con Roll, Pitch, Yaw, confianza y FPS.  
 NUEVO: --hef parametrizable.  
 NUEVO: verificación automática al arranque (test de ruido).  
 NUEVO: logging a fichero log\_YYYYMMDD\_HHMMSS.txt.
- v6 – giroscopio\_12kp\_v6.py (== giroscopio\_12kp\_v3a.py – mismo código, renombrado)  
 FIX KEYPOINTS: KP\_SCALE reducido de 10.0 a 4.0 (valor estándar YOLOv8).  
 Con scale=10 las celdas del borde proyectaban KPs ±160px fuera del frame.  
 Con scale=4.0 el rango máximo es ±64px – pero resultó INSUFICIENTE para  
 este modelo (el giroscopio ocupa ~600×400px de los 640×640 de entrada).  
 FIX KEYPOINTS: clamp de coordenadas KP al rango [0, 640) antes de NMS.  
 FIX ÁNGULOS: eliminado el -90° en PITCH.

FIX ÁNGULOS: ROLL OFFSET inicial = +90.0.  
FIX LOGS: eliminados mensajes DEBUG de bajo valor.  
NUEVO: carpeta historico\_logs/ para logs históricos.  
PENDIENTE: KP\_SCALE correcto para el nuevo modelo.

v3a – giroscopio\_12kp\_v3a.py (nombre confuso – mismo código que v6)

Sin cambios funcionales respecto a v6. Solo el nombre cambió.

Se identificaron dos problemas en las pruebas del 20/05/2026:

1. KP\_SCALE=4.0 → KPs agrupados en zona de ~65×64px en lugar de dispersarse por el objeto (~600×400px). Valor insuficiente.
2. Conversión de imagen errónea antes de inferencia:  
El script hacia RGB→BGR antes de pasar al modelo (línea 777).  
El modelo fue entrenado con imágenes BGR (OpenCV estándar).  
La cámara (Picamera2) entrega RGB888. La conversión a BGR era CORRECTA.  
SIN EMBARGO: el v6 del proyecto (separado, ejecutado con el script propio del profe) NO hacía esa conversión y daba conf\_max hasta 0.8+.  
Esto sugiere que el modelo fue entrenado con imágenes RGB o que el compilador Hailo absorbe la diferencia. Se añade --bgr flag para probar.

v3b – giroscopio\_12kp\_v3b.py (20/05/2026)

FIX KEYPOINTS: KP\_SCALE aumentado de 4.0 a 8.0.

FIX COLOR: eliminada la conversión RGB→BGR antes de inferencia.

NUEVO: log de coordenadas KP por cada detección en modo --verbose.

NUEVO: log de diagnóstico al arranque mostrando KP\_SCALE y rango esperado.

FIX: nombre de ventana y logs coherentes con la versión real (v3b).

FIX: log\_dir usa directorio del script, no del proceso que lo llama.

v3c – (28/05/2026)

NUEVO: delta de ángulos – muestra la variación de Roll/Pitch/Yaw respecto a hace N segundos (parametrizable con --delta-t, default 2.0 s).

Se usa un buffer circular de (timestamp, roll, pitch, yaw). En cada frame se busca la entrada más antigua dentro de la ventana y se calcula la diferencia angular normalizada a [-180, 180].

Motivo: el giroscopio puede estar montado rotado 180° en el eje Z respecto a lo definido en la estructura de KPs – los ángulos absolutos pueden ser engañosos dependiendo de la orientación inicial. El delta es invariante a la orientación inicial y permite detectar variaciones reales.

NUEVO: colores diferenciados por eje en el panel lateral:

PITCH → verde (0, 220, 0)

ROLL → rojo (0, 60, 220) ← BGR

YAW → azul (220, 80, 0) ← BGR

MEJORA: panel lateral ampliado para mostrar δRoll, δPitch, δYaw.

NUEVO: --delta-t flag CLI para parametrizar la ventana de tiempo del delta.

FIX CÁLCULO PITCH con giroscopio girado 180° en eje Z:

Al girar 180° en Z, lo que era "frontal" pasa a ser "trasero" y viceversa.

El vector mid(KP1,KP2)→mid(KP3,KP4) se invierte → Pitch saca 180° de error.

Solución: flag --flip-pitch que invierte el signo del vector de pitch, equivalente a sumar/restar 180° tras normalizar. Aplica también a Yaw

(--flip-yaw) ya que el eje de la base también queda invertido lateralmente.

Si el dron "mira hacia atrás" con offsets a 0: usar --flip-pitch --flip-yaw.

BUGS CONOCIDOS (observados en pruebas p211 y p113, 01/06/2026):

1. Símbolos "?" en el panel – \u00b0, \u2191, \u2193, \u2194, \u0394 no son renderizables por cv2.putText con FONT\_HERSHEY (solo ASCII).
2. Roll incorrecto (~-180° en reposo). El eje correcto existe en el modelo pero los offsets y la lógica de fallback necesitan revisión.
3. Pitch funciona bien solo en ±90°. En posiciones intermedias la perspectiva lateral de la cámara acorta el vector y añade ruido.
4. Yaw confirmado como no fiable con una sola cámara lateral.

v3d – (01/06/2026)

FIX CRÍTICO – Símbolos no ASCII en panel (Bug 13): sustituidos por ASCII puro.

FIX ROLL – lógica cambiada a vectores por lado KP5-KP7 / KP6-KP8 con promedio circular (más robusto que mid(KP5,KP6)→mid(KP7,KP8)).

FIX PITCH – suavizado EMA configurable (ANGLE\_SMOOTH\_ALPHA=0.3).

MEJORA YAW – marcado como "YAW(N/D)" en panel (no fiable con cámara lateral única).

NUEVO: log de diagnóstico de roll en modo --verbose (Y de KP5-8).

BUG CONOCIDO (Bug 16): EMA puede producir valores fuera de [-180, 180] en wrap-around.

BUG CONOCIDO (Bug 14/15): ROLL\_OFFSET y PITCH\_OFFSET no calibrados para este modelo.

v3e – (01/06/2026)

FIX Bug 16 – re-normalización EMA a [-180, 180].

FIX Bug 14 – ROLL\_OFFSET = 175.0 (intento de calibración, resultó incorrecto).

FIX Bug 15 – PITCH\_OFFSET = -90.0 (correcto – verificado en pruebas v3e).

NUEVO: WINDOW\_SCALE\_FACTOR=2 restaurado.

BUG RESIDUAL: ROLL\_OFFSET=175 da -90deg en reposo en lugar de 0deg.

Diagnóstico post-pruebas: el raw del roll es ~+95deg (no -90deg como se asumía).

Eso significa que el vector KP5-KP7 apunta hacia ARRIBA en imagen (Y KP5 > Y KP7),

es decir, el modelo tiene los KPs del soporte verticalmente invertidos, igual que los del octógono. Causa: misma perspectiva de grabación por detrás (Bug 17 a continuación).

v3f – (01/06/2026)

FIX Bug 17 – ROLL\_OFFSET corregido de 175.0 a -95.0:

ANÁLISIS COMPLETO (derivado de pruebas v3e, 01/06/2026):

- Con v3d (ROLL\_OFFSET=+90): pantalla = -175deg en reposo → raw ≈ +95deg
  - Con v3e (ROLL\_OFFSET=+175): pantalla = -90deg en reposo → raw ≈ +95deg
  - Conclusión: el raw del modelo es consistentemente ~+95deg (no -90deg).
  - Por qué raw≈+95 y no -90: el vector KP5-KP7 apunta hacia ARRIBA en imagen (KP5 tiene Y mayor que KP7 en píxeles), dando arctan2(dy, dx) ≈ +90°.
- Esto ocurre porque el modelo detecta KP5 ("arriba") en posición más baja en



imagen que KP7 ("abajo") – misma inversión vertical que los KPs del octógono, causada por el vídeo de entrenamiento grabado por la parte trasera del giroscopio.

- Offset correcto:  $-95^\circ \rightarrow \text{pantalla} = \text{fix\_angle}(+95, -95) = 0\text{deg}$  ✓
- Redondeado a  $-90^\circ$  para alinearse con la geometría real ( $\arctan 2$  puro =  $\pm 90^\circ$ ).

ATENCIÓN: este offset  $-90$  es específico del modelo actual (211 imágenes, perspectiva invertida). Con el nuevo dataset grabado de frente, el vector apuntará hacia abajo ( $\text{raw} = -90\text{deg}$ ) y el offset correcto volverá a ser  $+90$  (el valor geométrico "natural").

Los comentarios en el código documentan ambos casos.

CONFIRMADO: PITCH OFFSET =  $-90.0$  es correcto (pruebas v3e muestran  $+2$  a  $+3\text{deg}$  en reposo).

SIN CAMBIOS en lógica de cálculo, EMA, panel ni ninguna otra parte.

v3g – (03/06/2026)

v3h – (03/06/2026)

v3h – ESTE SCRIPT (03/06/2026, rev.2 – optimización FPS)

FIX RENDIMIENTO: cv2.addWeighted sobre ROI en lugar de frame completo.

PROBLEMA: la versión anterior hacía frame.copy() completo ( $960 \times 540 \times 3$  px) en cada llamada a badge() y en el fondo del panel. Con 3 badges + 1 fondo = 4 copias de  $\sim 1.5\text{M}$  píxeles por frame  $\rightarrow$  caída de FPS de  $\sim 24$  a  $\sim 15$ .

CAUSA TÉCNICA: cv2.addWeighted necesita dos arrays del mismo tamaño. La implementación naive copia el frame entero para tener un array de trabajo.

SOLUCIÓN: usar vistas NumPy ( $\text{ROI} = \text{frame}[\text{y0}:\text{y1}, \text{x0}:\text{x1}]$ ).  
 $\text{frame}[\text{y0}:\text{y1}, \text{x0}:\text{x1}]$  devuelve una vista del array original, no una copia.  
 addWeighted sobre la vista escribe directamente en esa región del frame sin copiar nada. Coste: solo los píxeles del ROI.

Ahorro por frame:

- Panel ( $310 \times 340$ px): frame.copy()  $1.5\text{M}$  px  $\rightarrow$  roi.copy()  $\sim 316\text{K}$  px ( $\sim 80\%$ )
- Cada badge ( $\sim 100 \times 30$ ): frame.copy()  $1.5\text{M}$  px  $\rightarrow$  roi.copy()  $\sim 9\text{K}$  px ( $\sim 99\%$ )
- 3 badges + panel:  $6\text{M}$  px  $\rightarrow$   $\sim 343\text{K}$  px  $\rightarrow$  reducción  $\sim 94\%$

SIN CAMBIOS en: lógica de cálculo, pipeline de inferencia, visualización.

NUEVO: panel de actitud visual con arcos OpenCV.

MOTIVACIÓN: los números absolutos de ángulo (ej.  $+2.9\text{deg}$ ) tienen un ruido inherente de  $\pm 5\text{--}7\text{deg}$  medido en pruebas físicas ( $\text{stdev Roll} = 6.98^\circ$ ,  $\text{Pitch} = 3.26^\circ$  con el giroscopio en reposo). Mostrarlos con decimales da una falsa sensación de precisión. Lo que el sistema SÍ puede dar con fiabilidad es:

- ¿Cabecea hacia adelante o atrás?  $\rightarrow$  Pitch
- ¿Alabea a derecha o izquierda?  $\rightarrow$  Roll
- ¿Gira en sentido horario o antihorario?  $\rightarrow$  Yaw delta

CAMBIOS respecto a v3g:

- draw\_results() reemplazada completamente por draw\_results\_visual().
- Panel lateral reemplazado por tres indicadores visuales con arcos/péndulo:
  - PITCH: arco semicircular superior. Punto coloreado se mueve a lo largo del arco: centro=nivel, derecha=adelante, izquierda=atrás.
  - ROLL: péndulo. Aguja gira alrededor del eje central. Visualmente intuitivo como un nivel de burbuja.
  - YAW: flecha en círculo. Indica dirección del delta (no ángulo absoluto). Siempre etiquetado como (N/D) – no fiable con cámara lateral única.
- Badge de texto bajo cada arco: "NIVEL", "ADELANTE", "ATRAS", "DER", "IZQ", "HORARIO", "ANTIHOR" con fondo de color (verde/rojo/gris).
- Números siguen visibles, pequeños ( $\text{fs} = 0.38$ ), bajo el badge.
- Zona muerta configurable: DEAD\_ZONE\_DEG (default  $6^\circ$ ). Dentro de la zona muerta el indicador muestra color gris neutro y badge "NIVEL"/"QUIETO"/N/D. Este valor absorbe el ruido de  $\pm 5\text{--}7\text{deg}$  observado en pruebas físicas.
- Cabecera (Det, Conf, FPS) se mantiene en la esquina superior izquierda del panel, compacta, tamaño reducido.
- NUEVO: --dead-zone flag CLI para ajustar la zona muerta en grados.
- SIN CAMBIOS en pipeline de inferencia, keypoints, EMA, delta, logs.

AJUSTES POST-PRUEBAS FÍSICAS p309/p410 (análisis de capturas y logs del 03/06/2026):

Las pruebas confirmaron p410 como mejor modelo: confianza media  $0.888$  (vs  $0.636$  de p309), 0 frames sin detección (vs 30 consecutivos en p309),  $99.5\%$  frames  $> 0.5$  conf. Se identificaron 6 problemas y se resuelven los 4 que son de código:

FIX 1 – Detecciones dobles: IOU\_THRESH bajado de  $0.45$  a  $0.30$ .

CAUSA: la estructura metálica del giroscopio genera dos bounding boxes con solapamiento  $< 45\%$  en ciertos ángulos. Con  $0.30$  se fusionan correctamente. Observado en pruebas p410 frame 34 (Det: 2, Conf:  $0.902 + 0.047$ ).

NUEVO: --iou-thresh parametrizable en CLI para ajuste fino.

FIX 2 – Falsas detecciones débiles: CONF\_THRESH subido de  $0.01$  a  $0.05$ .

CAUSA: con  $\text{conf} = 0.01$  se aceptaban detecciones de ruido ( $0.02\text{--}0.03$ ) que nunca corresponden al giroscopio real. p410 tiene  $\text{conf\_media} = 0.888$ , así que  $0.05$  no pierde ninguna detección legítima. p309 con  $\text{conf\_media} = 0.636$  tampoco pierde. Diagnóstico: el valor  $0.01$  era necesario en las fases de debugging (v4\_det), ya no tiene sentido en producción.

FIX 3 – KPs perdidos en perspectiva extrema: KP THRESH bajado de 0.10 a 0.07.  
CAUSA: los KPs traseros del octógono (KP1 blanco, KP2 negro) tienen visibilidad 0.07-0.09 cuando están parcialmente ocultos por perspectiva lateral. Con 0.10 se pierden y el cálculo de Pitch cae al fallback de 2 puntos (menos preciso).  
Con 0.07 se recuperan sin añadir ruido significativo.

FIX 4 – Log de delta raw en modo verbose.  
NUEVO: en modo --verbose, cada frame logea los valores brutos de delta\_angle() para diagnosticar posibles problemas de wrap-around (Bug 16 residual).  
Formato: [DELTA-DIAG] d\_roll=X d\_pitch=X d\_yaw=X

NUEVO: --iou-thresh parametrizable en CLI (default: 0.30).  
NUEVO: --hide-yaw flag para ocultar la fila de Yaw del panel.  
Motivo: el Yaw no es fiable con una sola cámara lateral y su presencia confunde en demostraciones. Con --hide-yaw se oculta completamente.  
Sin el flag, se sigue mostrando como YAW(N/D) para diagnóstico.  
NUEVO: --kp-thresh parametrizable en CLI (default: 0.07).

SIN CAMBIOS en: lógica de cálculo de ángulos, EMA, pipeline de inferencia, colores de KPs, estructura de tensores, verificación del HEF.

PROBLEMAS NO RESUELTOS (requieren hardware o reentrenamiento):  
- Yaw no fiable con cámara lateral única → requiere 2ª cámara o IMU.  
- stride 8 (conv44) siempre raw conf=0 → recalibrar HEF con objetos pequeños.  
- Inestabilidad KP1/KP2 en Pitch >70° → limitación geométrica de perspectiva.

#### BUG DE CALIBRACIÓN DEL HEF – DOCUMENTADO

SÍNTOMA: raw\_conf = 0 en todos los frames para todos los strides.  
bbox y kps tienen valores normales. 0 detecciones siempre.  
CAUSA: dataset de calibración = mismo dataset de entrenamiento.  
El compilador calibra el rango de la capa de confianza con imágenes "conocidas" → rango demasiado optimista → en producción todos los valores caen en raw=0.  
SOLUCIÓN: recompilar con imágenes de OTRAS escenas como calibración.  
El nuevo modelo (220 img entrenamiento + 560 con augmentations) funcionó correctamente con calib2 y calib3.  
VERIFICACIÓN: test de ruido al arranque → conv71:max debe ser > 0.  
calib1: max=7 ✓  
calib2: max=24 ✓  
calib3: max=11 ✓  
estandar (mismo ds que train): max=0 ✗ → no usar

#### ESTRUCTURA DE 12 KEYPOINTS

| KP# | Grupo | Color          | Posición                 | Eje/Ángulo |
|-----|-------|----------------|--------------------------|------------|
| 1   | OCT   | blanco         | Esq trasera-izq octógono | PITCH      |
| 2   | OCT   | negro          | Esq trasera-der octógono | PITCH      |
| 3   | OCT   | rojo           | Esq frontal-der octógono | PITCH      |
| 4   | OCT   | amarillo       | Esq frontal-izq octógono | PITCH      |
| 5   | INT   | azul eléctrico | Punta U arriba-izq       | ROLL       |
| 6   | INT   | naranja        | Travesaño arriba-der     | ROLL       |
| 7   | INT   | verde lima     | Punta U abajo-izq        | ROLL       |
| 8   | INT   | magenta        | Travesaño abajo-der      | ROLL       |
| 9   | BASE  | cian           | Esq trasera-izq base     | YAW        |
| 10  | BASE  | rosa           | Esq trasera-der base     | YAW        |
| 11  | BASE  | marrón         | Esq frontal-der base     | YAW        |
| 12  | BASE  | morado         | Esq frontal-izq base     | YAW        |

#### CÁLCULO DE ÁNGULOS

PITCH (KP1-4): vector mid(KP1,KP2) → mid(KP3,KP4). Sin restar 90°.  
ROLL (KP5-8): vector mid(KP5,KP6) → mid(KP7,KP8). ROLL\_OFFSET=+90° compensa arctan2.  
YAW (KP9-12): vector mid(KP9,KP12) → mid(KP10,KP11).

#### TENSORES HEF (giroscopio.hef – nuevo modelo 20/05/2026)

conv43/57/70 UINT8 FCR(H×W×64) bbox DFL (strides 8, 16, 32)  
conv44/58/71 UINT8 NHWC(H×W×1) confianza – formato PROB (zp=0, scale=1/255)  
conv45/59/72 UINT16 FCR(H×W×36) keypoints (12KP × 3 = 36 valores)

Tensores de conf en formato PROB (zp=0): raw=0→0.0, raw=128→0.5, raw=255→1.0.  
NO aplicar sigmoid.

"""

```
import argparse
import datetime
import logging
import os
import re
import sys
import threading
import time
import traceback
import cv2
import numpy as np
```

```

try:
 from picamera2 import Picamera2
 HAVE_PICAMERA = True
except ImportError:
 HAVE_PICAMERA = False

try:
 from hailo_platform import (HEF, VDevice, HailoStreamInterface,
 ConfigureParams, InputVStreamParams,
 OutputVStreamParams, FormatType, InferVStreams)

 HAVE_HAILO = True
except ImportError:
 HAVE_HAILO = False

os.environ.setdefault("DISPLAY", ":0")
os.environ.pop("WAYLAND_DISPLAY", None)

=====
CONFIGURACIÓN – todos los parámetros relevantes aquí arriba
=====

VERSION = "v3h"

— Arquitectura del modelo —
PREFIX = "giroscopio/" # prefijo de tensores en el HEF (nombre de red)
NUM_KP = 12 # número de keypoints del modelo

— Cámara —
CAM_W, CAM_H = 960, 540 # resolución de captura Picamera2
MODEL_SIZE = 640 # tamaño de entrada del modelo (cuadrado)

— Umbrales de detección —
Con los HEF del nuevo modelo (220 img + calib cruzada) la conf es baja:
calib2/3 con v3c: esperamos conf max en rango 0.012-0.027 inicialmente
el script profe (v6 sin conversión BGR) llegó hasta 0.7-0.8 con calib1
Mantener CONF_THRESH bajo para no perder detecciones débiles.
CONF_THRESH = 0.05 # umbral objectness – subido de 0.01 en v3g (ver FIX 2)
KP_THRESH = 0.07 # umbral visibilidad KP – bajado de 0.10 en v3g (ver FIX 3)
IOU_THRESH = 0.30 # NMS: IoU máximo – bajado de 0.45 en v3g (ver FIX 1)

— Escala de keypoints —
HISTORIA:
10.0 → rango máximo ±160px con stride=32. KPs salían del frame en celdas
de borde. Problemático con el HEF mal calibrado (v5).
4.0 → rango máximo ±64px. Insuficiente para objeto de ~600px en pantalla.
KPs se agrupaban en zona de ~65×64px (observado 17/04/2026 y 20/05/2026).
8.0 → rango máximo ±128px. Equilibrio razonable. A verificar.
#
CÓMO DIAGNOSTICAR:
Usar --verbose y buscar en el log la línea "[KP-RANGE]".
Si rango X ≈ rango_Y ≈ 65px → subir KP_SCALE.
Si los KPs aparecen fuera del bounding box → bajar KP_SCALE.
KP_SCALE = 8.0

— Color de imagen al modelo —
El nuevo modelo (entrenado en Roboflow/Colab con PIL/albumentations) recibe
imágenes en formato RGB. Usar False. Si los resultados son malos, probar True.
En CLI: --bgr para forzar conversión RGB-BGR.
CONVERT_TO_BGR = False # False = pasar RGB tal cual; True = convertir a BGR

— Offsets de ángulos en reposo —
PENDIENTE DE CALIBRAR con el nuevo modelo.
Procedimiento: poner a 0, ejecutar con giroscopio en reposo y nivelado,
anotar valores del panel, poner offset = -(valor medido).
ROLL_OFFSET = -90.0
^ v3f: corregido tras análisis de pruebas v3e (01/06/2026).
#
HISTORIA DEL OFFSET DE ROLL:
- v3b/v3c/v3d: ROLL_OFFSET=+90.0 → pantalla=-175deg en reposo
- v3e: ROLL_OFFSET=+175.0 → pantalla=-90deg en reposo
- v3f: ROLL_OFFSET=-90.0 → pantalla=~0deg en reposo ← CORRECTO
#
ANÁLISIS: el raw del modelo para este dataset es ~+95deg (no -90deg).
El vector KP5-KP7 apunta hacia ARRIBA en imagen (KP5 tiene Y > KP7 en píxeles)
porque el modelo aprendió los KPs del soporte con inversión vertical – misma causa
que los KPs del octógono: el video de entrenamiento se grabó por detrás del giroscopio.
arctan2(-(KP7.y - KP5.y), ...) con KP7.y < KP5.y → argumento positivo → +90°
fix_angle(+90, -90) = 0° ✓
#
CON EL NUEVO DATASET (grabado de frente):
El vector KP5-KP7 apuntará hacia ABAJO (KP5 arriba, KP7 abajo, Y_KP5 < Y_KP7)
→ raw = -90deg → offset correcto = +90.0 (el valor "natural" geométrico)
Δ RECALIBRAR: poner ROLL_OFFSET=0, ejecutar en reposo, anotar valor, offset=-(valor)

PITCH_OFFSET = -90.0
^ v3e/v3f: CONFIRMADO correcto (pruebas v3e, 01/06/2026).
En reposo muestra +2 a +3deg → dentro del margen de calibración aceptable.

```

```

Causa geométrica: cámara lateral + perspectiva invertida (grabación por detrás)
→ el vector mid(KP1,KP2)-mid(KP3,KP4) da ~+90deg en reposo → offset=-90 → 0deg.
Δ RECALIBRAR con el nuevo modelo: poner 0.0, medir en reposo, offset=-(valor).

YAW_OFFSET = 0.0
^Yaw marcado N/D en el panel - no calibrar hasta tener segunda cámara.

— Escala de ventana OpenCV —————
— Zona muerta para los indicadores visuales (v3h) —————
Ángulos dentro de este rango se muestran como "quieto/nivel/N/D" en los badges.
El ruido de ±5-7° medido en pruebas físicas (03/06/2026) justifica un valor
mínimo de 6°. Subir si el indicador parpadea demasiado en reposo.
DEAD_ZONE_DEG = 6.0 # grados - ajustable con --dead-zone en CLI

v3e: restaurado desde v3b (desapareció en v3c/v3d).
1 = tamaño nativo de la fuente (960×540).
2 = 1920×1080 - recomendado para monitores FullHD.
3 = 2880×1620 - para pantallas 4K (valor original de v3b, demasiado grande en FullHD).
WINDOW_SCALE_FACTOR = 2

— Suavizado temporal de ángulos (EMA - media exponencial móvil) —————
Alpha=1.0 → sin suavizado (valor instantáneo, comportamiento v3c).
Alpha=0.3 → suavizado moderado: reduce jitter sin añadir lag excesivo.
Alpha=0.1 → suavizado fuerte: útil si los KPs son muy ruidosos.
Sólo se aplica a Roll y Pitch (el Yaw no es fiable con esta perspectiva).
ANGLE_SMOOTH_ALPHA = 0.3 # factor EMA: 0 < alpha <= 1.0

— Inversión de ejes (giroscopio montado girado 180° en eje Z) —————
Contexto (28/05/2026): el video de entrenamiento se grabó con el giroscopio
girado 180° en Z respecto a la convención de KPs (KP1/2 que deberían ser
"traseros" quedaron delante). Esto invierte los vectores de PITCH y YAW.
Síntoma típico: con el dron en reposo y offsets a 0, Pitch y Yaw dan ~180°
de error, o el dron aparece "mirando hacia atrás".
Solución: activar --flip-pitch y --flip-yaw para invertir el vector antes
del cálculo de arctan2 (equivalente a reflejar el vector, no a sumar 180°).
FLIP_PITCH = False # True = invertir sentido del vector de pitch
FLIP_YAW = False # True = invertir sentido del vector de yaw

— Delta de ángulos —————
El panel muestra δPitch, δRoll, δYaw = ángulo_actual - ángulo_hace_DELTA_T_S.
Es invariante a la orientación inicial del giroscopio - útil cuando el montaje
no sigue exactamente la convención de KPs o hay ambigüedad de 180°.
Parametrizable con --delta-t en CLI.
El buffer circular almacena (timestamp, roll, pitch, yaw). En cada frame se
busca la entrada más antigua dentro de la ventana DELTA_T_S y se hace la
diferencia angular normalizada a [-180, 180] (función delta angle).
DELTA_T_S = 2.0 # ventana temporal del delta en segundos (default: 2s)
DELTA_BUFFER_S = 10.0 # historial máximo en buffer (siempre > DELTA_T_S)

— Colores BGR del panel lateral (v3c: diferenciados por eje) —————
PITCH → verde | ROLL → rojo | YAW → azul
Nota: OpenCV usa BGR, no RGB.
COLOR_PITCH = (0, 220, 0) # verde
COLOR_ROLL = (0, 60, 220) # rojo
COLOR_YAW = (220, 80, 0) # azul

— Tensores de conf en formato PROB (NO aplicar sigmoid) —————
Confirmado en todos los HEF probados: zp=0, scale=1/255.
Se inicializa vacío y se rellena automáticamente al leer el HEF.
CONF_ALREADY_PROB = set()

— Parámetros de dequantización (fallback si quant_info no accesible) —————
Se actualizan automáticamente desde el HEF. Estos son valores de referencia
del HEF calib2 del 20/05/2026 (log 093942).
QUANT = {
 "conv43": (0.165420, 117.0),
 "conv44": (0.003922, 0.0),
 "conv45": (0.001043, 14244.0),
 "conv57": (0.105872, 120.0),
 "conv58": (0.003922, 0.0),
 "conv59": (0.000634, 15453.0),
 "conv70": (0.090270, 133.0),
 "conv71": (0.003922, 0.0),
 "conv72": (0.000564, 16619.0),
}

— Colores BGR de los 12 keypoints —————
Orden: KP1..KP12 según la tabla de estructura de arriba.
KP_COLORS = [
 (255, 255, 255), # KP1 blanco - OCT trasera-izq (PITCH)
 (30, 30, 30), # KP2 negro - OCT trasera-der (PITCH)
 (16, 16, 232), # KP3 rojo - OCT frontal-der (PITCH)
 (0, 214, 255), # KP4 amarillo - OCT frontal-izq (PITCH)
 (255, 85, 0), # KP5 azul el.- INT U arriba-izq (ROLL)
 (0, 106, 255), # KP6 naranja - INT travesaño arr (ROLL)
 (0, 192, 57), # KP7 v.lima - INT U abajo-izq (ROLL)
 (204, 0, 204), # KP8 magenta - INT travesaño abj (ROLL)
]

```

```

(204, 187, 0), # KP9 cian - BASE trasera-izq (YAW)
(153, 68, 255), # KP10 rosa - BASE trasera-der (YAW)
(0, 68, 136), # KP11 marrón - BASE frontal-der (YAW)
(204, 0, 102), # KP12 morado - BASE frontal-izq (YAW)
]

=====
LOGGING
=====

def setup_logging(hef_path="", log_dir_override=""):
 """
 Configura logging dual: fichero (INFO) + consola (WARNING).

 NOMENCLATURA DEL FICHERO (rev. 08/06/2026):
 logs_giroscopio_v3a_<modelo>_<YYYYMMDD>_HHMMSS.txt
 Ejemplo: logs_giroscopio_v3a_p410_20260608_143022.txt

 El <modelo> se extrae del directorio padre del HEF:
 /home/ai/giroscopio/giroscopio_v3a/p410/giroscopio.hef
 ^^^^ -> "p410"

 DIRECTORIO DE LOGS:
 - Si se pasa log_dir_override, se usa ese directorio.
 - Si no, se guarda en <dir_del_script>/historico_logs/
 (comportamiento original, compatibilidad con ejecucion directa).

 CAMBIO vs version anterior:
 Antes: log_gX_YYYYMMDD_HHMMSS.txt (patron_gN del nombre HEF)
 Ahora: logs_giroscopio_v3a_<modelo>_YYYYMMDD_HHMMSS.txt
 con modelo extraido del directorio padre del HEF
 """
 ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

 # Extraer nombre del modelo del directorio padre del HEF
 # ../giroscopio_v3a/p410/giroscopio.hef -> "p410"
 hef_abs = os.path.abspath(hef_path) if hef_path else ""
 modelo = os.path.basename(os.path.dirname(hef_abs)) if hef_abs else "modelo"
 if not modelo or modelo in (".", ""):
 modelo = "modelo"

 # Directorio de logs: externo (desde wrapper .sh) o por defecto junto al script
 if log_dir_override:
 log_dir = log_dir_override
 else:
 base_dir = os.path.dirname(os.path.abspath(__file__))
 log_dir = os.path.join(base_dir, "historico_logs")

 os.makedirs(log_dir, exist_ok=True)
 log_path = os.path.join(log_dir, f"logs_giroscopio_v3a_{modelo}_{ts}.txt")

 log = logging.getLogger("gyro")
 log.setLevel(logging.DEBUG)

 # Fichero: INFO y superior (sin DEBUG)
 fh = logging.FileHandler(log_path, encoding="utf-8")
 fh.setLevel(logging.INFO)
 fh.setFormatter(logging.Formatter(
 "%(asctime)s.%(msecs)03d [%(levelname)s] %(message)s",
 datefmt="%H:%M:%S"))

 # Consola: solo WARNING/ERROR (sin ruido en producción)
 ch = logging.StreamHandler(sys.stdout)
 ch.setLevel(logging.WARNING)
 ch.setFormatter(logging.Formatter("[% (levelname)s] %(message)s"))

 log.addHandler(fh)
 log.addHandler(ch)
 log.info(f"Log -> {log_path}")
 return log, log_path

=====
ARGUMENTOS
=====

def parse_args():
 p = argparse.ArgumentParser(
 description=f"Giroscopio 12KP {VERSION} - Hailo-8",
 formatter_class=argparse.RawDescriptionHelpFormatter,
 epilog="""
Ejemplos:
Cámara: DISPLAY=:0 python3 %(prog)s --hef /home/ai/giroscopio_v3a/p410/giroscopio.hef
Video: python3 %(prog)s --hef giroscopio.hef --video resultado_p410.mp4
Guardar: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --save resultado_v3h.mp4
Sin Yaw: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --hide-yaw
 """)

```

```

IoU ajust.: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --iou-thresh 0.25
KP sensible: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --kp-thresh 0.05
Debug delta: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --verbose
Delta 5s: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --delta-t 5.0
Girado 180°: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --flip-pitch --flip-yaw
Demo limpia: DISPLAY=:0 python3 %(prog)s --hef giroscopio.hef --hide-yaw --iou-thresh 0.25
 """
 p.add_argument("--hef", required=True,
 help="Ruta al archivo HEF compilado")
 p.add_argument("--video", default="",
 help="Video de entrada en lugar de cámara")
 p.add_argument("--save", default="",
 help="Ruta para guardar video de salida")
 p.add_argument("--conf", type=float, default=CONF_THRESH,
 help=f"Umbral de confianza (default: {CONF_THRESH})")
 p.add_argument("--iou-thresh", type=float, default=IOU_THRESH,
 help=f"Umbral IoU para NMS (default: {IOU_THRESH}). "
 f"Bajar si hay detecciones dobles.")
 p.add_argument("--kp-thresh", type=float, default=KP_THRESH,
 help=f"Umbral visibilidad KP individual (default: {KP_THRESH}). "
 f"Bajar para recuperar KPs en posiciones extremas.")
 p.add_argument("--hide-yaw", action="store_true",
 help="Ocultar Yaw del panel (no fiable con cámara lateral única)")
 p.add_argument("--dead-zone", type=float, default=DEAD_ZONE_DEG,
 help=f"Zona muerta en grados para los indicadores (default: {DEAD_ZONE_DEG}). "
 f"Dentro de este rango el badge muestra 'quieto/nivel'. "
 f"Subir si parpadea en reposo, bajar si no reacciona.")
 p.add_argument("--kp-scale", type=float, default=KP_SCALE,
 help=f"Factor escala keypoints (default: {KP_SCALE}). "
 f"Subir si KPs agrupados, bajar si salen del objeto.")
 p.add_argument("--bgr", action="store_true",
 help="Convertir RGB→BGR antes de inferencia (probar si los KPs fallan)")
 p.add_argument("--flip-pitch", action="store_true",
 help="Invertir sentido del vector de Pitch (usar si giroscopio girado 180° en Z)")
 p.add_argument("--flip-yaw", action="store_true",
 help="Invertir sentido del vector de Yaw (usar junto a --flip-pitch si girado 180° en Z)")
 p.add_argument("--delta-t", type=float, default=DELTA_T_S,
 help=f"Ventana temporal del delta de ángulos en segundos (default: {DELTA_T_S})")
 p.add_argument("--verbose", action="store_true",
 help="Log detallado por frame (incluye rango de coordenadas KP)")
 p.add_argument("--log-dir", default="",
 help="Directorio donde guardar los logs. "
 "Si no se especifica, se usa <dir_script>/historico_logs/. "
 "Normalmente pasado por el wrapper .sh.")
 p.add_argument("--save-auto", default="",
 help="Directorio donde guardar el video de salida con nombre automatico. "
 "Nombre: giroscopio_v3a_<modelo>_<YYYYMMDD_HHMMSS>.mp4. "
 "Incompatible con --save (que usa ruta completa manual).")
 return p.parse_args()

=====
UTILIDADES NUMÉRICAS
=====

def sigmoid(x):
 return 1.0 / (1.0 + np.exp(-np.clip(x, -88.0, 88.0)))

def softmax_dfl(x, reg_max=16):
 """DFL: Distribution Focal Loss. Decodifica bbox como combinación convexa de distancias."""
 x = x.reshape(*x.shape[:-1], 4, reg_max)
 ex = np.exp(x - x.max(axis=-1, keepdims=True))
 sm = ex / ex.sum(axis=-1, keepdims=True)
 return (sm * np.arange(reg_max, dtype=np.float32)).sum(axis=-1)

def dequant(tensor_raw, name, log):
 """Dequantización lineal: (raw - zp) * scale."""
 if name not in QUANT:
 log.error(f"dequant: '{name}' no está en QUANT")
 raise KeyError(name)
 s, zp = QUANT[name]
 return (tensor_raw.astype(np.float32) - zp) * s

def nms(boxes, scores, iou_thresh=IOU_THRESH):
 """Non-Maximum Suppression estándar por IoU."""
 if len(boxes) == 0:
 return []
 x1, y1, x2, y2 = boxes[:,0], boxes[:,1], boxes[:,2], boxes[:,3]
 areas = (x2 - x1) * (y2 - y1)
 order = scores.argsort()[::-1]
 keep = []
 while order.size > 0:
 i = order[0]; keep.append(i)
 inter = (np.maximum(0, np.minimum(x2[i], x2[order[1:]])) - np.maximum(x1[i], x1[order[1:]])) *
 np.maximum(0, np.minimum(y2[i], y2[order[1:]])) - np.maximum(y1[i], y1[order[1:]]))
 iou = inter / (areas[i] + areas[order[1:]] - inter + 1e-6)
 order = order[1:][iou < iou_thresh]
 return keep

```

```

=====
DECODIFICACIÓN
=====

def decode_stride(raw_bbox, raw_conf, raw_kps, stride,
 b_nm, c_nm, k_nm, conf_thresh, kp_scale, log, fid=0, verbose=False):
 """
 Decodifica un nivel de stride del modelo YOLOv8-Pose.
 Devuelve (boxes_xyxy, confs, keypoints_xy_score, conf_max).

 Fórmula de keypoints:
 kp_x = (sigmoid(raw_x) * kp_scale - kp_scale/2 + cell_x) * stride
 kp_y = (sigmoid(raw_y) * kp_scale - kp_scale/2 + cell_y) * stride

 Con kp_scale=8.0 y stride=32:
 rango de desplazamiento = ±4 celdas * 32px = ±128px desde el centro de celda.
 Con kp_scale=8.0 y stride=8:
 rango = ±4 celdas * 8px = ±32px desde el centro de celda.
 """
 H, W = raw_conf.shape[:2]

 bbox_f = dequant(raw_bbox, b_nm, log)
 conf_f = dequant(raw_conf, c_nm, log)
 kps_f = dequant(raw_kps, k_nm, log)

 # Mapa de confianza - NO aplicar sigmoid si ya es formato PROB (zp=0)
 raw_vals = conf_f[..., 0]
 if c_nm in CONF_ALREADY_PROB:
 conf_map = np.clip(raw_vals, 0.0, 1.0)
 else:
 conf_map = sigmoid(raw_vals)

 conf_max = float(conf_map.max())

 mask = conf_map > conf_thresh
 if not mask.any():
 return (np.empty((0, 4), np.float32),
 np.empty(0, np.float32),
 np.empty((0, NUM_KP, 3), np.float32),
 conf_max)

 ys, xs = np.where(mask)
 confs = conf_map[ys, xs]

 # BBox via DFL → coordenadas absolutas en espacio 640×640
 ltrb = softmax_dfl(bbox_f[ys, xs]) * stride
 cx, cy = (xs + 0.5) * stride, (ys + 0.5) * stride
 boxes = np.stack([cx - ltrb[:,0], cy - ltrb[:,1],
 cx + ltrb[:,2], cy + ltrb[:,3]], axis=1)

 # Keypoints
 half = kp_scale / 2.0
 kps_sel = kps_f[ys, xs].reshape(-1, NUM_KP, 3)
 kp_x = (sigmoid(kps_sel[..., 0]) * kp_scale - half + xs[:, None]) * stride
 kp_y = (sigmoid(kps_sel[..., 1]) * kp_scale - half + ys[:, None]) * stride
 kp_s = sigmoid(kps_sel[..., 2])

 # Clamp al rango válido del frame
 kp_x = np.clip(kp_x, 0.0, float(MODEL_SIZE - 1))
 kp_y = np.clip(kp_y, 0.0, float(MODEL_SIZE - 1))

 keypoints = np.concatenate([kp_x[..., None], kp_y[..., None], kp_s[..., None]], axis=-1)

 if verbose and len(kp_x) > 0:
 # Log de rango de coordenadas para diagnóstico de KP_SCALE
 log.debug(f"[KP-RANGE s={stride} f={fid}] "
 f"kp_x=[{kp_x.min():.1f}, {kp_x.max():.1f}] "
 f"kp_y=[{kp_y.min():.1f}, {kp_y.max():.1f}] "
 f"rango_X={kp_x.max()-kp_x.min():.1f} "
 f"rango_Y={kp_y.max()-kp_y.min():.1f}")

 return boxes, confs, keypoints, conf_max

=====
CÁLCULO DE ÁNGULOS
=====

def fix_angle(v, offset):
 """Aplica offset y normaliza al rango [-180, 180]."""
 if v is None: return None
 return ((v + offset + 180) % 360) - 180

def calc_angles(kps, scores, flip_pitch=False, flip_yaw=False, verbose_log=None, kp_thresh=KP_THRESH):
 """
 Calcula Roll, Pitch, Yaw a partir de los 12 keypoints visibles.
 """

```



```

kps: array (12, 2) - coordenadas X,Y en espacio 640x640
scores: array (12,) - visibilidad de cada KP
flip_pitch: invertir sentido del vector de pitch (para giroscopio girado 180° en Z)
flip_yaw: invertir sentido del vector de yaw (ídem)
verbose_log: logger para diagnóstico del vector de roll (None = silencio)

CAMBIOS v3d respecto a v3c:
- ROLL: cambiado de vector mid(KP5,KP6)-mid(KP7,KP8) a vectores por lado:
 KP5-KP7 (lado izquierdo, punta-U) y KP6-KP8 (lado derecho, travesaño).
 El ángulo es el promedio de ambos vectores (más robusto). Si solo hay
 un lado disponible se usa ese. El vector vertical en reposo apunta
 hacia abajo en imagen → arctan2 = -90° → ROLL_OFFSET=+90° da 0°.
 Diagnóstico: en modo verbose se loguea la Y de cada par para detectar
 si el modelo invierte "arriba" y "abajo".
- Roll/Pitch ahora devuelven el valor RAW (sin EMA); el suavizado se
 aplica en el bucle de inferencia con _smooth_angle().

"""
def vis(i): return float(scores[i]) > kp_thresh
def mid(i, j): return (kps[i] + kps[j]) / 2.0

def ang(p1, p2, flip=False):
 """Ángulo del vector p1-p2 en grados. Si flip=True, invierte el vector."""
 if flip:
 p1, p2 = p2, p1
 return np.degrees(np.arctan2(-(p2[1] - p1[1]), p2[0] - p1[0]))

— PITCH — KP1-4 (índices 0-3) —————
Vector mid(trasero) → mid(frontal) representa el eje de pitch.
No restar 90°. Calibrar con PITCH_OFFSET en reposo.
flip_pitch=True si el giroscopio fue grabado girado 180° en Z.
pitch = None
if vis(0) and vis(1) and vis(2) and vis(3): pitch = ang(mid(0,1), mid(2,3), flip_pitch)
elif vis(0) and vis(1) and vis(2): pitch = ang(mid(0,1), kps[2], flip_pitch)
elif vis(1) and vis(2) and vis(3): pitch = ang(kps[1], mid(2,3), flip_pitch)
elif vis(0) and vis(2): pitch = ang(kps[0], kps[2], flip_pitch)
elif vis(1) and vis(3): pitch = ang(kps[1], kps[3], flip_pitch)
elif vis(0) and vis(3): pitch = ang(kps[0], kps[3], flip_pitch)
elif vis(1) and vis(2): pitch = ang(kps[1], kps[2], flip_pitch)

— ROLL — KP5-8 (índices 4-7) —————
v3d: usar vectores por lado en lugar del vector entre puntos medios.
#
GEOMETRÍA:
KP5 (idx=4): punta U arriba-izq → lado izquierdo
KP6 (idx=5): travesaño arriba-der → lado derecho
KP7 (idx=6): punta U abajo-izq → lado izquierdo
KP8 (idx=7): travesaño abajo-der → lado derecho
#
Vector principal: KP5-KP7 (lado izquierdo, punta U - piezas reales).
Vector secundario: KP6-KP8 (lado derecho, travesaño).
En reposo ambos vectores apuntan hacia abajo en imagen (Y aumenta hacia abajo)
→ arctan2(-(dy), dx) con dy>0 → arctan2(-dy, 0) = arctan2(negativo, 0) = -90°
→ fix_angle(-90, +90) = 0° ✓
#
DIAGNÓSTICO DE INVERSIÓN (síntoma: roll ≈ -180° en reposo):
Si el modelo detecta KP5/KP6 con Y > KP7/KP8 (invertidos en imagen),
el vector apunta hacia ARRIBA → arctan2 = +90° → con offset +90 → 180°
→ fix_angle(180, +90) = -90° o similar. En ese caso subir ROLL_OFFSET
a -90.0 (o usar --flip-roll si se implementa en próxima versión).
#
FALLBACK si solo un lado está visible: usar ese lado.
FALLBACK si ningún lado izq/der: usar par mid(arriba)-mid(abajo) como v3c.
roll = None
roll_src = "none" # para log de diagnóstico

v5, v6, v7, v8 = vis(4), vis(5), vis(6), vis(7)

Calcular ángulo de cada lado disponible
roll_izq = ang(kps[4], kps[6]) if (v5 and v7) else None # KP5-KP7
roll_der = ang(kps[5], kps[7]) if (v6 and v8) else None # KP6-KP8

if roll_izq is not None and roll_der is not None:
 # Promedio angular (a través de la media de senos/cosenos para evitar wrap-around)
 rad_izq = np.radians(roll_izq)
 rad_der = np.radians(roll_der)
 avg_sin = (np.sin(rad_izq) + np.sin(rad_der)) / 2.0
 avg_cos = (np.cos(rad_izq) + np.cos(rad_der)) / 2.0
 roll = np.degrees(np.arctan2(avg_sin, avg_cos))
 roll_src = "izq+der"
elif roll_izq is not None:
 roll = roll_izq
 roll_src = "izq-solo"
elif roll_der is not None:
 roll = roll_der
 roll_src = "der-solo"
else:
 # Fallback v3c: mid(arriba)-mid(abajo)
 pa = (kps[4] + kps[5]) / 2.0 if (v5 and v6) else (kps[4] if v5 else (kps[5] if v6 else None))

```

```

 pb = (kps[6] + kps[7]) / 2.0 if (v7 and v8) else (kps[6] if v7 else (kps[7] if v8 else None))
 if pa is not None and pb is not None:
 roll = ang(pa, pb, flip=False)
 roll_src = "mid-fallback"

Log de diagnóstico de roll (visible con --verbose)
if verbose_log is not None and roll is not None:
 kp5y = f"{kps[4][1]:.1f}" if v5 else "N/V"
 kp6y = f"{kps[5][1]:.1f}" if v6 else "N/V"
 kp7y = f"{kps[6][1]:.1f}" if v7 else "N/V"
 kp8y = f"{kps[7][1]:.1f}" if v8 else "N/V"
 verbose_log.debug(
 f"[ROLL-DIAG] src={roll_src} raw={roll:.1f}deg "
 f"KP5y={kp5y} KP6y={kp6y} KP7y={kp7y} KP8y={kp8y} "
 f"(esperado: KP5y<KP7y y KP6y<KP8y para vector hacia abajo)")

— YAW — KP9-12 (índices 8-11) —————
flip_yaw=True si el giroscopio fue grabado girado 180° en Z.
v3d: el Yaw sigue calculándose pero se marca como no fiable en el panel
hasta contar con la segunda cámara.
yaw = None
has_izq = vis(8) or vis(11)
has_der = vis(9) or vis(10)
if has_izq and has_der:
 mi = mid(8, 11) if (vis(8) and vis(11)) else (kps[8] if vis(8) else kps[11])
 md = mid(9, 10) if (vis(9) and vis(10)) else (kps[9] if vis(9) else kps[10])
 yaw = ang(mi, md, flip_yaw)

return (fix_angle(roll, ROLL_OFFSET),
 fix_angle(pitch, PITCH_OFFSET),
 fix_angle(yaw, YAW_OFFSET))

=====
BUFFER DE HISTORIAL DE ÁNGULOS (para delta)
=====
Lista de (timestamp, roll, pitch, yaw). Se limpia automáticamente al superar
DELTA_BUFFER_S segundos de antigüedad.
angle_history = [] # [(timestamp float, roll_or_None, pitch_or_None, yaw_or_None)]
angle_history_lock = threading.Lock()

— Estado del suavizado EMA (Roll y Pitch) —————
Valores suavizados actuales. Inicializados a None hasta la primera detección.
_ema_roll = None
_ema_pitch = None

def _smooth_angle(ema_prev, new_val, alpha):
 """
 Media exponencial móvil para ángulos en [-180, 180].
 Maneja el wrap-around: si ema_prev=175° y new_val=-175°, interpola
 en la dirección corta (-10°) no en la larga (+350°).
 alpha=1.0 → sin suavizado (devuelve new_val directamente).

 v3e FIX Bug 16: añadida re-normalización final a [-180, 180].
 En v3d, casos de wrap-around en el límite exacto ±180° podían producir
 valores como -266.8deg porque fix_angle ya había normalizado y la EMA
 acumulaba el error. La normalización final garantiza el rango siempre.
 """
 if ema_prev is None or new_val is None:
 return new_val
 if alpha >= 1.0:
 return new_val
 # Diferencia angular normalizada a [-180, 180] — interpola por el camino corto
 diff = ((new_val - ema_prev + 180.0) % 360.0) - 180.0
 result = ema_prev + alpha * diff
 # Re-normalizar para garantizar que el resultado esté siempre en [-180, 180]
 return ((result + 180.0) % 360.0) - 180.0

def push_angles(roll, pitch, yaw):
 """
 Añade los ángulos actuales al buffer con timestamp actual.
 Limpia entradas más antiguas que DELTA_BUFFER_S.
 """
 now = time.time()
 with angle_history_lock:
 angle_history.append((now, roll, pitch, yaw))
 # Limpiar entradas demasiado antiguas
 cutoff = now - DELTA_BUFFER_S
 while angle_history and angle_history[0][0] < cutoff:
 angle_history.pop(0)

def get_delta(delta_t_s):
 """
 Devuelve (d_roll, d_pitch, d_yaw) como diferencia angular entre el ángulo
 actual y el más antiguo dentro de la ventana delta_t_s.
 Si no hay entrada suficientemente antigua, devuelve (None, None, None).
 Usa delta_angle() para manejar el wrap-around de ±180°.
 """
 now = time.time()

```

```

target_t = now - delta_t_s
with angle_history_lock:
 if len(angle_history) < 2:
 return None, None, None
 # Entrada más reciente
 _, r_now, p_now, y_now = angle_history[-1]
 # Buscar la entrada más antigua dentro de la ventana
 ref = None
 for entry in angle_history:
 if entry[0] <= target_t:
 ref = entry
 else:
 break
 if ref is None:
 return None, None, None
 _, r_ref, p_ref, y_ref = ref

d_roll = delta_angle(r_ref, r_now)
d_pitch = delta_angle(p_ref, p_now)
d_yaw = delta_angle(y_ref, y_now)
return d_roll, d_pitch, d_yaw

def delta_angle(a_ref, a_now):
 """
 Diferencia angular normalizada a [-180, 180].
 Maneja el wrap-around: si a_ref=170° y a_now=-170°, el delta es -20° (no +340°).
 Devuelve None si alguno de los argumentos es None.
 """
 if a_ref is None or a_now is None:
 return None
 d = a_now - a_ref
 return ((d + 180.0) % 360.0) - 180.0

=====
DIBUJO - panel lateral + keypoints + bboxes
=====

def _attitude_color(val, dead):
 """
 Devuelve color BGR según magnitud y signo de un ángulo respecto a zona muerta.
 - Dentro de dead: gris neutro
 - Positivo moderado (<45°): verde claro
 - Positivo fuerte (>=45°): verde vivo
 - Negativo moderado: rojo claro
 - Negativo fuerte: rojo vivo
 """
 if val is None:
 return (120, 120, 120)
 a = abs(val)
 if a <= dead:
 return (140, 140, 140) # gris - zona muerta
 if val > 0:
 return (20, 220, 20) if a < 45 else (0, 255, 0) # verde claro → vivo
 return (60, 60, 220) if a < 45 else (0, 0, 255) # rojo claro → vivo

def _badge(frame, cx, y, text, color_bgr, font, fs):
 """
 Dibuja una pildora de texto centrada en (cx, y).
 Fondo semitransparente del color del indicador, texto blanco.
 """
 (tw, th), _ = cv2.getTextSize(text, font, fs, 1)
 pad_x, pad_y = 8, 4
 x0 = cx - tw // 2 - pad_x
 y0 = y - th - pad_y
 x1 = cx + tw // 2 + pad_x
 y1 = y + pad_y

 # Fondo semitransparente - solo sobre el ROI del badge (no el frame completo)
 # frame[y0:y1, x0:x1] es una VISTA del array, no una copia.
 # addWeighted sobre la vista escribe directamente en el frame.
 h_f, w_f = frame.shape[:2]
 x0c = max(0, x0); y0c = max(0, y0)
 x1c = min(w_f, x1); y1c = min(h_f, y1)
 roi = frame[y0c:y1c, x0c:x1c]
 bg = roi.copy() # copia solo del ROI (~9K px vs 1.5M)
 cv2.rectangle(bg, (0, 0), (x1c - x0c, y1c - y0c), color_bgr, -1)
 cv2.addWeighted(bg, 0.55, roi, 0.45, 0, roi) # escribe en roi → en frame
 cv2.rectangle(frame, (x0, y0), (x1, y1), color_bgr, 1)
 cv2.putText(frame, text, (cx - tw // 2, y), font, fs, (240, 240, 240), 1)

def _badge_text_pitch(val, dead):
 if val is None:
 return "SIN KP"
 if abs(val) <= dead:
 return "NIVEL"
 return "ADELANTE" if val > 0 else "ATRAS"

```

```

def _badge_text_roll(val, dead):
 if val is None: return "SIN KP"
 if abs(val) <= dead: return "NIVELADO"
 return "DER" if val > 0 else "IZQ"

def _badge_text_yaw(val, dead):
 if val is None: return "N/D"
 if abs(val) <= dead: return "N/D"
 return "HORARIO" if val > 0 else "ANTIHOR"

def _draw_pitch_arc(frame, cx, cy, r, val, dead):
 """
 Arco semicircular superior (abierto hacia abajo).
 El punto de referencia (val=0) está en el centro del arco (parte superior).
 val>0 → punto a la derecha, val<0 → izquierda.
 Eje: de -90° a +90° mapeado al arco de 180° a 360° (en coords imagen).
 """
 # Dibujar arco de fondo (gris oscuro)
 cv2.ellipse(frame, (cx, cy), (r, r), 0, 180, 360, (60, 60, 60), 3)
 # Marcas de referencia: 0° (arriba), ±45°, ±90°
 for mark_deg, label in [(-90, ""), (-45, ""), (0, ""), (45, ""), (90, "")]:
 angle_cv = 270 + mark_deg # 270 = arriba en coords OpenCV
 rad = np.radians(angle_cv)
 mx = int(cx + r * np.cos(rad))
 my = int(cy + r * np.sin(rad))
 cv2.circle(frame, (mx, my), 2, (80, 80, 80), -1)

 # Punto 0° (arriba del arco)
 cv2.circle(frame, (cx, cy - r), 3, (80, 80, 80), -1)

 if val is None:
 return

 # Posición del indicador: val=0→arriba, val=+90→derecha, val=-90→izquierda
 val_clamped = max(-90.0, min(90.0, val))
 angle_cv = 270 + val_clamped # 270° = arriba en OpenCV
 rad = np.radians(angle_cv)
 px = int(cx + r * np.cos(rad))
 py = int(cy + r * np.sin(rad))

 color = _attitude_color(val, dead)

 # Línea desde centro hasta el punto en el arco
 cv2.line(frame, (cx, cy), (px, py), color, 2)
 # Punto grueso en el arco
 cv2.circle(frame, (px, py), 7, color, -1)
 cv2.circle(frame, (px, py), 8, (0, 0, 0), 1)

 # Línea vertical de referencia (nivel cero)
 cv2.line(frame, (cx, cy), (cx, cy - r), (60, 60, 60), 1)

def _draw_roll_pendulum(frame, cx, cy, r, val, dead):
 """
 Péndulo de Roll: la aguja cuelga del centro y se inclina con el eje.
 val=0 → aguja vertical (arriba)
 val>0 → inclinada a la derecha
 val<0 → inclinada a la izquierda
 Elipse horizontal como "horizonte" de referencia visual.
 """
 # Elipse de referencia (horizonte del eje)
 cv2.ellipse(frame, (cx, cy), (r, r // 3), 0, 0, 360, (50, 50, 50), 1)

 # Línea vertical de referencia
 cv2.line(frame, (cx, cy - r), (cx, cy + r // 4), (60, 60, 60), 1)

 # Marcas ±30° y ±60°
 for mark in [-60, -30, 30, 60]:
 mr = np.radians(-90 + mark)
 mx = int(cx + r * np.cos(mr))
 my = int(cy + r * np.sin(mr))
 cv2.circle(frame, (mx, my), 2, (70, 70, 70), -1)

 if val is None:
 return

 val_clamped = max(-90.0, min(90.0, val))
 angle_cv = -90 + val_clamped # -90 = apunta arriba en OpenCV
 rad = np.radians(angle_cv)
 px = int(cx + r * np.cos(rad))
 py = int(cy + r * np.sin(rad))

```

```

color = _attitude_color(val, dead)

Aguja del péndulo
cv2.line(frame, (cx, cy), (px, py), color, 3)
Punto en el extremo
cv2.circle(frame, (px, py), 7, color, -1)
cv2.circle(frame, (px, py), 8, (0, 0, 0), 1)
Pivote central
cv2.circle(frame, (cx, cy), 4, (180, 180, 180), -1)

def _draw_yaw_compass(frame, cx, cy, r, delta_val, dead):
 """
 Brújula de Yaw: siempre muestra el DELTA, no el ángulo absoluto.
 La flecha indica la dirección del giro respecto a los últimos N segundos.
 Color azul para diferenciar de pitch/roll y remarcar que es N/D absoluto.
 """
 # Círculo base
 cv2.circle(frame, (cx, cy), r, (50, 50, 50), 1)

 # Marcas cardinales
 for angle_deg, lbl in [(270, "^"), (0, ">"), (90, "v"), (180, "<")]:
 rad = np.radians(angle_deg)
 mx = int(cx + (r + 8) * np.cos(rad))
 my = int(cy + (r + 8) * np.sin(rad))
 cv2.putText(frame, lbl, (mx - 4, my + 4),
 cv2.FONT_HERSHEY_SIMPLEX, 0.3, (70, 70, 70), 1)

 # Flecha norte (referencia fija)
 cv2.line(frame, (cx, cy), (cx, cy - r), (60, 60, 60), 1)

 if delta_val is None:
 return

 delta_clamped = max(-180.0, min(180.0, delta_val))
 angle_cv = -90 + delta_clamped
 rad = np.radians(angle_cv)
 px = int(cx + r * np.cos(rad))
 py = int(cy + r * np.sin(rad))

 color = _attitude_color(delta_val, dead)
 # Yaw siempre azul (diferenciador visual) si hay movimiento
 if abs(delta_val) > dead:
 color = (220, 120, 20) if delta_val > 0 else (180, 80, 20)

 cv2.line(frame, (cx, cy), (px, py), color, 3)
 cv2.circle(frame, (px, py), 6, color, -1)
 cv2.circle(frame, (px, py), 7, (0, 0, 0), 1)
 cv2.circle(frame, (cx, cy), 3, (180, 180, 180), -1)

def draw_results(frame, boxes, confs, kps_list, angles_list,
 fps, scale, pad_x, pad_y, frame_count, delta_t_s,
 hide_yaw=False, kp_thresh=KP_THRESH, log=None,
 dead_zone=DEAD_ZONE_DEG):
 """
 v3h: panel de actitud visual con arcos OpenCV.

 Panel derecho dividido en dos zonas:
 1. Cabecera compacta: Det, Conf, FPS, Frame
 2. Tres indicadores visuales: Pitch (arco), Roll (péndulo), Yaw delta (brújula)
 Cada indicador tiene: nombre, gráfico, badge de texto, número pequeño.

 Los indicadores usan zona muerta (dead_zone) para no oscilar con el ruido
 inherente del modelo (~±5-7°). Dentro de la zona muerta todo es gris.
 Los números absolutos se mantienen pequeños bajo cada badge.

 Los bounding boxes y keypoints se dibujan igual que en v3g.
 """
 h, w = frame.shape[:2]
 n_det = len(boxes)

 # Obtener ángulos actuales
 roll_v = angles_list[0][0] if (angles_list and angles_list[0][0] is not None) else None
 pitch_v = angles_list[0][1] if (angles_list and angles_list[0][1] is not None) else None
 yaw_v = angles_list[0][2] if (angles_list and angles_list[0][2] is not None) else None

 push_angles(roll_v, pitch_v, yaw_v)
 d_roll, d_pitch, d_yaw = get_delta(delta_t_s)

 # Log verbose FIX 4 (v3g)
 if log is not None and log.isEnabledFor(10):
 dr_s = f"{d_roll:.2f}" if d_roll is not None else "None"

```

```

dp_s = f"{d_pitch:.2f}" if d_pitch is not None else "None"
dy_s = f"{d_yaw:.2f}" if d_yaw is not None else "None"
log.debug(f"[DELTA-DIAG f={frame_count}] d_roll={dr_s}deg "
 f"d_pitch={dp_s}deg d_yaw={dy_s}deg (ventana={delta_t_s}s)")

font = cv2.FONT_HERSHEY_SIMPLEX
fs_hdr = 0.42 # cabecera
fs_lbl = 0.38 # etiquetas pequeñas bajo los indicadores
fs_nam = 0.40 # nombre del eje encima del indicador

--- PANEL DERECHO ---
Geometría: panel de 300x350 en la esquina superior derecha
Dividido en: cabecera (60px) + zona de indicadores (290px, 3 columnas)
pw = 310
ph = 340
px0 = w - pw - 6
py0 = 6
Fondo del panel - solo sobre el ROI del panel (310x340px vs 960x540)
Misma técnica que _badge(): vista NumPy, sin copia del frame completo.
rx0 = max(0, px0 - 2); ry0 = max(0, py0 - 2)
rx1 = min(w, px0 + pw + 2); ry1 = min(h, py0 + ph + 2)
roi_panel = frame[ry0:ry1, rx0:rx1]
bg_panel = roi_panel.copy() # ~316K px vs 1.5M del frame
cv2.rectangle(bg_panel, (0, 0), (rx1 - rx0, ry1 - ry0), (25, 25, 25), -1)
cv2.addWeighted(bg_panel, 0.75, roi_panel, 0.25, 0, roi_panel)
cv2.rectangle(frame, (px0, py0), (px0 + pw, py0 + ph), (70, 70, 70), 1)

--- Cabecera compacta ---
hdr_h = 58
cv2.line(frame, (px0, py0 + hdr_h), (px0 + pw, py0 + hdr_h), (70, 70, 70), 1)

det_color = (0, 220, 0) if n_det > 0 else (0, 0, 220)
conf_s = f"{confs[0]:.3f}" if n_det > 0 else "--"
cv2.putText(frame, f"Det:{n_det} Conf:{conf_s}",
 (px0 + 8, py0 + 18), font, fs_hdr, det_color, 1)
cv2.putText(frame, f"FPS:{fps:.1f} Frame:{frame_count}",
 (px0 + 8, py0 + 38), font, fs_hdr, (180, 180, 180), 1)
cv2.putText(frame, f"dz={-dead_zone:.0f}deg dt={delta_t_s:.0f}s",
 (px0 + 8, py0 + 55), font, 0.32, (100, 100, 100), 1)

if n_det == 0:
 cv2.putText(frame, "SIN DETECCION", (10, 35),
 cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 0, 255), 2)
 # Dibujar indicadores en gris si no hay detección
 roll_v = pitch_v = yaw_v = d_roll = d_pitch = d_yaw = None

--- Zona de indicadores: 3 columnas de ancho igual ---
Cada indicador ocupa pw/3 de ancho.
Dentro de cada columna: etiqueta arriba, gráfico en medio, badge+num abajo.
ind_y0 = py0 + hdr_h + 6 # tope de los indicadores
col_w = pw // 3 # ~103px por columna
arc_r = 38 # radio del arco/péndulo/brújula

Centros X de cada columna
cx_p = px0 + col_w // 2 # Pitch
cx_r = px0 + col_w + col_w // 2 # Roll
cx_y = px0 + 2 * col_w + col_w // 2 # Yaw

--- PITCH ---
arc_cy_p = ind_y0 + arc_r + 20 # centro del arco de pitch
Etiqueta
cv2.putText(frame, "PITCH", (cx_p - 20, ind_y0 + 12),
 font, fs_nam, COLOR_PITCH, 1)
Arco
_draw_pitch_arc(frame, cx_p, arc_cy_p, arc_r, pitch_v, dead_zone)
Badge
badge_y_p = arc_cy_p + arc_r + 22
badge_txt_p = _badge_text_pitch(pitch_v, dead_zone)
badge_col_p = _attitude_color(pitch_v, dead_zone)
_badge(frame, cx_p, badge_y_p, badge_txt_p, badge_col_p, font, 0.36)
Número pequeño
num_p = f"{pitch_v:.1f}d" if pitch_v is not None else "--"
cv2.putText(frame, num_p, (cx_p - 22, badge_y_p + 18),
 font, fs_lbl, (130, 130, 130), 1)

--- Separador vertical ---
cv2.line(frame, (px0 + col_w, ind_y0), (px0 + col_w, py0 + ph - 4),
 (55, 55, 55), 1)

--- ROLL ---
pend_cy_r = ind_y0 + arc_r + 20
cv2.putText(frame, "ROLL", (cx_r - 16, ind_y0 + 12),
 font, fs_nam, COLOR_ROLL, 1)
_draw_roll_pendulum(frame, cx_r, pend_cy_r, arc_r, roll_v, dead_zone)
badge_y_r = pend_cy_r + arc_r + 22
badge_txt_r = _badge_text_roll(roll_v, dead_zone)
badge_col_r = _attitude_color(roll_v, dead_zone)

```

```

_badge(frame, cx_r, badge_y_r, badge_txt_r, badge_col_r, font, 0.36)
num_r = f"{roll_v:.1f}d" if roll_v is not None else "--"
cv2.putText(frame, num_r, (cx_r - 22, badge_y_r + 18),
 font, fs_lbl, (130, 130, 130), 1)

— Separador vertical —————
cv2.line(frame, (px0 + 2 * col_w, ind_y0), (px0 + 2 * col_w, py0 + ph - 4),
 (55, 55, 55), 1)

— YAW —————
comp_cy_y = ind_y0 + arc_r + 20
yaw_label = "YAW" if hide_yaw else "YAW(N/D)"
cv2.putText(frame, yaw_label, (cx_y - 28, ind_y0 + 12),
 font, 0.34, COLOR_YAW, 1)
Siempre dibujamos la brújula con el delta de Yaw (no el absoluto)
_draw_yaw_compass(frame, cx_y, comp_cy_y, arc_r, d_yaw, dead_zone)
badge_y_y = comp_cy_y + arc_r + 22
badge_txt_y = _badge_text_yaw(d_yaw, dead_zone)
badge_col_y = COLOR_YAW if (d_yaw is not None and abs(d_yaw) > dead_zone) else (100, 100, 100)
_badge(frame, cx_y, badge_y_y, badge_txt_y, badge_col_y, font, 0.36)
if not hide_yaw:
 num_y = f"d={d_yaw:.1f}d" if d_yaw is not None else "--"
 cv2.putText(frame, num_y, (cx_y - 26, badge_y_y + 18),
 font, fs_lbl, (100, 100, 100), 1)

— Bounding boxes y keypoints —————
Sin cambios respecto a v3g
if n_det == 0:
 return

for idx, (box, conf) in enumerate(zip(boxes, confs)):
 x1 = int(np.clip((box[0] - pad_x) / scale, 0, w - 1))
 y1 = int(np.clip((box[1] - pad_y) / scale, 0, h - 1))
 x2 = int(np.clip((box[2] - pad_x) / scale, 0, w - 1))
 y2 = int(np.clip((box[3] - pad_y) / scale, 0, h - 1))
 cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
 cv2.putText(frame, f"Giro {conf:.3f}", (x1, max(y1 - 6, 12)),
 cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 0), 1)

 if idx < len(kps_list):
 kps = kps_list[idx]
 for ki in range(NUM_KP):
 kx, ky, ks = kps[ki]
 if ks < kp_thresh: continue
 px = int(np.clip((kx - pad_x) / scale, 0, w - 1))
 py = int(np.clip((ky - pad_y) / scale, 0, h - 1))
 r = max(4, int(ks * 10))
 cv2.circle(frame, (px, py), r, KP_COLORS[ki], -1)
 cv2.circle(frame, (px, py), r + 1, (0, 0, 0), 1)
 cv2.putText(frame, str(ki + 1), (px + r + 2, py + 4),
 cv2.FONT_HERSHEY_SIMPLEX, 0.32, KP_COLORS[ki], 1)

=====
VERIFICACIÓN AL ARRANQUE
=====

def verificar_hef(pipe, in_name, log):
 """
 Test de ruido: inyecta imagen aleatoria y comprueba que los tensores de
 confianza tengan algún valor > 0. Si todo es 0, el HEF fue compilado con
 el mismo dataset de entrenamiento y no detectará nada en producción.

 Valores observados en HEFs buenos vs malos:
 calib1: conv71=7 → ✓ OK
 calib2: conv71=24 → ✓ OK
 calib3: conv71=11 → ✓ OK
 estandar: conv71=0 → ✗ MAL — recompilar
 """
 log.info("Verificando HEF con imagen de ruido aleatorio...")
 img = np.random.randint(0, 255, (MODEL_SIZE, MODEL_SIZE, 3), dtype=np.uint8)
 raw = pipe.infer({in_name: np.expand_dims(img, 0)})
 maxes = {s: int(raw[PREFIX + s][0].max()) for s in ("conv44", "conv58", "conv71")}
 best = max(maxes.values())
 for k, v in maxes.items():
 log.info(f" {k}: max={v}")
 if best == 0:
 log.error("⚠ PROBLEMA DE CALIBRACIÓN DEL HEF: raw_conf=0 con imagen aleatoria.")
 log.error(" El HEF fue compilado con el mismo dataset de entrenamiento.")
 log.error(" Recompilar con imágenes de calibración de otra escena/dataset.")
 log.error(" Ver documentación: bug_calibracion_hef.md")
 return False
 log.info(f"✓ HEF OK — conf max={best} con ruido aleatorio")
 return True

=====
ESTADO COMPARTIDO (hilo de inferencia ↔ bucle principal)
=====

```



```

latest_frame = None # último frame preparado para inferencia
latest_result = None # último resultado decodificado
frame_lock = threading.Lock()
result_lock = threading.Lock()

=====
HILO DE INFERENCIA
=====

def infer_thread_fn(ng, ng_params, in_name, conf_thresh, kp_scale, args, log, iou_thresh=IOU_THRESH,
kp_thresh_val=KP_THRESH):
 """
 Hilo dedicado a inferencia. Lee latest_frame, ejecuta la red Hailo,
 decodifica los 3 strides y escribe en latest_result.

 Separado del bucle principal para que la UI no bloquee la inferencia
 y viceversa.
 """
 global latest_result
 infer_count = 0
 frames_sin_det = 0

 # Leer configuración de flip desde args (pasados en el arranque)
 flip_pitch = getattr(args, 'flip_pitch', False)
 flip_yaw = getattr(args, 'flip_yaw', False)
 log.info(f"flip_pitch={flip_pitch} flip_yaw={flip_yaw}")
 log.info("Hilo de inferencia iniciado.")
 try:
 with ng.activate(ng_params):
 in_p = InputVStreamParams.make(ng, format_type=FormatType.UINT8)
 out_u8 = OutputVStreamParams.make(ng, format_type=FormatType.UINT8)
 out_ul6 = OutputVStreamParams.make(ng, format_type=FormatType.UINT16)

 # Los tensores de KP son UINT16; el resto UINT8
 kps_names = {PREFIX + n for n in ("conv45", "conv59", "conv72")}
 out_p = {n: p for n, p in out_u8.items() if n not in kps_names}
 out_p.update({n: p for n, p in out_ul6.items() if n in kps_names})
 log.info(f"OutputVStreamParams: {len(out_p)} tensores "
 f"(6 UINT8 bbox/conf + 3 UINT16 kps)")

 with InferVStreams(ng, in_p, out_p) as pipe:
 verificar_hef(pipe, in_name, log)

 log.info("Bucle de inferencia activo.")
 s8_zeros_warned = False

 while True:
 with frame_lock:
 frame_snap = latest_frame
 if frame_snap is None:
 time.sleep(0.005)
 continue

 img = frame_snap.copy()
 infer_count += 1

 try:
 raw = pipe.infer({in_name: np.expand_dims(img, 0)})
 except Exception as e:
 log.error(f"[INFER f={infer_count}] pipe.infer falló: {e}")
 time.sleep(0.1)
 continue

 # Aviso único si s=8 siempre da ceros (bug calibración conocido)
 if not s8_zeros_warned:
 if int(raw[PREFIX + "conv44"][0].max()) == 0:
 log.warning("s=8 raw_conf=0 siempre (bug calibración HEF conocido "
 "- ver bug_calibracion_hef.md). Este aviso no se repetirá.")
 s8_zeros_warned = True

 # Decodificar los 3 strides
 all_boxes, all_confs, all_kps = [], [], []
 conf_maxes = []

 for stride, b, c, k in [
 (8, "conv43", "conv44", "conv45"),
 (16, "conv57", "conv58", "conv59"),
 (32, "conv70", "conv71", "conv72"),
]:
 key_b = PREFIX + b
 if key_b not in raw:
 log.error(f"Tensor {key_b} no encontrado en salida del HEF")
 continue
 try:
 boxes, confs, kps, cm = decode_stride(

```

```

 raw[PREFIX + b][0], raw[PREFIX + c][0], raw[PREFIX + k][0],
 stride, b, c, k, conf_thresh, kp_scale,
 log, infer_count, args.verbose)
 conf_maxes.append(cm)
 if len(boxes):
 all_boxes.append(boxes)
 all_confs.append(confs)
 all_kps.append(kps)
 except Exception as e:
 log.error(f"decode stride s={stride} f={infer_count}: {e}")
 log.error(traceback.format_exc())

if all_boxes:
 boxes = np.concatenate(all_boxes)
 confs = np.concatenate(all_confs)
 kps_all = np.concatenate(all_kps)
 keep = nms(boxes, confs, iou_thresh)
 boxes, confs, kps_all = boxes[keep], confs[keep], kps_all[keep]

Calcular ángulos. En verbose, pasar el logger para diagnóstico
de roll (incluye Y de KP5-8 para detectar inversiones).
vlog = log if args.verbose else None
angles_raw = [calc_angles(k[:, :2], k[:, 2],
 flip_pitch=flip_pitch,
 flip_yaw=flip_yaw,
 verbose_log=vlog,
 kp_thresh=kp_thresh_val) for k in kps_all]

Aplicar suavizado EMA a Roll y Pitch de la primera detección.
El Yaw no se suaviza: no es fiable con esta perspectiva.
global _ema_roll, _ema_pitch
angles = []
for i, (r_raw, p_raw, y_raw) in enumerate(angles_raw):
 if i == 0:
 # Solo suavizamos la detección principal
 _ema_roll = _smooth_angle(_ema_roll, r_raw, ANGLE_SMOOTH_ALPHA)
 _ema_pitch = _smooth_angle(_ema_pitch, p_raw, ANGLE_SMOOTH_ALPHA)
 angles.append((_ema_roll, _ema_pitch, y_raw))
 else:
 angles.append((r_raw, p_raw, y_raw))

r_str = f"{angles[0][0]:.1f}" if angles[0][0] is not None else "N/A"
p_str = f"{angles[0][1]:.1f}" if angles[0][1] is not None else "N/A"
log.info(f"[f={infer_count}] OK {len(boxes)} det(s) "
 f"conf_max={confs.max():.3f} "
 f"roll={r_str}deg pitch={p_str}deg")
if args.verbose:
 # Log detallado de KPs para diagnóstico de escala y posición
 for di, kps in enumerate(kps_all):
 vis_kps = [(i+1, kps[i,0], kps[i,1], kps[i,2])
 for i in range(NUM_KP) if kps[i,2] >= kp_thresh_val]
 if vis_kps:
 xs = [v[1] for v in vis_kps]
 ys = [v[2] for v in vis_kps]
 log.debug(f" det[{di}] KPs visibles={len(vis_kps)} "
 f"X=[{min(xs):.1f},{max(xs):.1f}] "
 f"Y=[{min(ys):.1f},{max(ys):.1f}] "
 f"rangos=({max(xs)-min(xs):.1f}, {max(ys)-min(ys):.1f})")
 frames_sin_det = 0
 else:
 # Sin detección: resetear EMA para que no arrastre el
 # último valor conocido cuando el objeto vuelva a aparecer
 _ema_roll = None
 _ema_pitch = None
 frames_sin_det += 1
 if frames_sin_det % 30 == 0:
 best = max(conf_maxes) if conf_maxes else 0.0
 log.info(f"[f={infer_count}] 0 detecciones "
 f"frames_sin_det={frames_sin_det} "
 f"mejor_conf={best:.4f}")
 boxes = np.empty((0, 4), np.float32)
 confs = np.empty(0, np.float32)
 kps_all = np.empty((0, NUM_KP, 3), np.float32)
 angles = []

with result_lock:
 latest_result = (boxes, confs, kps_all, angles)

except Exception as e:
 log.error(f"Excepción fatal en hilo de inferencia: {e}")
 log.error(traceback.format_exc())

=====
MAIN
=====

def main():
 global latest_frame, CONVERT_TO_BGR, FLIP_PITCH, FLIP_YAW, DELTA_T_S

```

```

args = parse_args()
log, log_path = setup_logging(args.hef, log_dir_override=args.log_dir)
kp_scale = args.kp_scale
iou_thresh = args.iou_thresh
kp_thresh_val = args.kp_thresh
hide_yaw = args.hide_yaw
dead_zone = args.dead_zone

Si se pasa --bgr en CLI, sobrescribe la constante del script
if args.bgr:
 CONVERT_TO_BGR = True

Flags de inversión de eje para giroscopio girado 180° en Z
if args.flip_pitch:
 FLIP_PITCH = True
if args.flip_yaw:
 FLIP_YAW = True

Ventana temporal del delta (puede ser distinta a la constante por defecto)
delta_t_s = args.delta_t

log.info("=" * 60)
log.info(f"giroscopio_12kp_{VERSION}.py - inicio")
log.info(f"HEF: {args.hef}")
log.info(f"Fuente: {'video: ' + args.video if args.video else 'cámara'}")
log.info(f"CONF_THRESH={args.conf} KP_THRESH={KP_THRESH}")
log.info(f"KP_SCALE={kp_scale} CONVERT_TO_BGR={CONVERT_TO_BGR}")
log.info(f"FLIP_PITCH={FLIP_PITCH} FLIP_YAW={FLIP_YAW}")
log.info(f"DELTA_T_S={delta_t_s}s DELTA_BUFFER_S={DELTA_BUFFER_S}s")
log.info(f"IOU_THRESH={iou_thresh} HIDE_YAW={hide_yaw} DEAD_ZONE={dead_zone}deg")
log.info(f"KP_THRESH={kp_thresh_val} (parametrizado)")

Log de diagnóstico de rangos esperados de KP por stride
for st in [8, 16, 32]:
 rango = (kp_scale / 2) * st
 log.info(f"KP_SCALE={kp_scale} stride={st} → rango máximo KP = ±{rango:.0f}px")

log.info("=" * 60)

if not HAVE_HAILO:
 log.error("hailo_platform no disponible.")
 sys.exit(1)

if not os.path.exists(args.hef):
 log.error(f"HEF no encontrado: {args.hef}")
 sys.exit(1)

— Inicializar Hailo —————
log.info("Cargando HEF...")
hef = HEF(args.hef)
target = VDevice()
cfg = ConfigureParams.create_from_hef(hef, interface=HailoStreamInterface.PCIe)
ng = target.configure(hef, cfg)[0]
ng_p = ng.create_params()
in_name = hef.get_input_vstream_infos()[0].name
log.info(f"Input: {in_name}")

Leer parámetros de cuantización del HEF y actualizar QUANT + CONF_ALREADY_PROB
for info in hef.get_output_vstream_infos():
 short = info.name.replace(PREFIX, "")
 if short not in QUANT: continue
 try:
 s = float(info.quant_info.qp_scale)
 zp = float(info.quant_info.qp_zp)
 QUANT[short] = (s, zp)
 # Formato PROB: zp=0 y scale=1/255 (≈0.003922)
 if short in {"conv44", "conv58", "conv71"} and zp < 100:
 CONF_ALREADY_PROB.add(short)
 log.info(f" {short}: scale={s:.6f} zp={zp:.1f}"
 + (" [PROB]" if short in CONF_ALREADY_PROB else ""))
 except AttributeError:
 log.warning(f" {short}: quant_info no accesible, usando valor de fallback")

log.info(f"CONF_ALREADY_PROB: {CONF_ALREADY_PROB}")

— Hilo de inferencia —————
t_infer = threading.Thread(
 target=infer_thread_fn,
 args=(ng, ng_p, in_name, args.conf, kp_scale, args, log, iou_thresh, kp_thresh_val),
 daemon=True)
t_infer.start()

— Fuente de imagen —————
use_video = bool(args.video)
if use_video:

```

```

cap = cv2.VideoCapture(args.video)
if not cap.isOpened():
 log.error(f"No se puede abrir: {args.video}")
 sys.exit(1)
src_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
src_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
vid_fps = cap.get(cv2.CAP_PROP_FPS) or 25.0
log.info(f"Video: {src_w}x{src_h} {vid_fps:.1f}fps")
cam = None
else:
 if not HAVE_PICAMERA:
 log.error("Picamera2 no disponible y no se especificó --video")
 sys.exit(1)
 cam = Picamera2()
 cam.configure(cam.create_preview_configuration(
 main={"size": (CAM_W, CAM_H), "format": "RGB888"}))
 cam.start()
 time.sleep(1.0)
 src_w, src_h = CAM_W, CAM_H
 cap = None
 log.info(f"Cámara lista: {src_w}x{src_h}")

--- Letterbox ---
Escala y padding para convertir src_wxsrc_h → 640x640 con bordes negros
scale = MODEL_SIZE / max(src_w, src_h)
nw = int(src_w * scale)
nh = int(src_h * scale)
pad_x = (MODEL_SIZE - nw) // 2
pad_y = (MODEL_SIZE - nh) // 2
log.info(f"Letterbox: scale={scale:.4f} nw={nw} nh={nh} pad_x={pad_x} pad_y={pad_y}")

--- VideoWriter ---
Dos modos de grabacion:
--save <ruta.mp4> : ruta completa manual (uso desde terminal)
--save-auto <dir/> : nombre automatico en directorio dado (uso desde wrapper)
Nombre: giroscopio_v3a_<modelo>_<YYYYMMDD_HHMMSS>.mp4
El <modelo> es el mismo que se extrae para el log (dir padre del HEF)
writer = None
save_path_efectivo = args.save # puede ser "" si no se pide grabacion

if args.save_auto and not args.save:
 # Calcular nombre automatico igual que en setup_logging
 ts_vid = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
 hef_abs = os.path.abspath(args.hef) if args.hef else ""
 modelo_vid = os.path.basename(os.path.dirname(hef_abs)) if hef_abs else "modelo"
 if not modelo_vid or modelo_vid in (".", ""):
 modelo_vid = "modelo"
 video_dir = args.save_auto
 os.makedirs(video_dir, exist_ok=True)
 save_path_efectivo = os.path.join(
 video_dir, f"giroscopio_v3a_{modelo_vid}_{ts_vid}.mp4")
 log.info(f"Grabacion automatica: {save_path_efectivo}")
elif args.save and args.save_auto:
 log.warning("--save y --save-auto especificados simultaneamente; "
 "se usa --save y se ignora --save-auto")

if save_path_efectivo:
 fourcc = cv2.VideoWriter_fourcc(*"mp4v")
 writer = cv2.VideoWriter(save_path_efectivo, fourcc, 25, (src_w, src_h))
 log.info(f"Grabando salida en: {save_path_efectivo}" if writer.isOpened()
 else f"No se pudo abrir VideoWriter: {save_path_efectivo}")

win_title = f"Giroscopio 12KP {VERSION} - Hailo-8"
cv2.namedWindow(win_title, cv2.WINDOW_NORMAL)
Escalar ventana según WINDOW_SCALE_FACTOR (restaurado en v3e desde v3b)
Factor 1 = nativo (960x540), 2 = FullHD (1920x1080), 3 = 4K (2880x1620)
cv2.resizeWindow(win_title, src_w * WINDOW_SCALE_FACTOR, src_h * WINDOW_SCALE_FACTOR)
log.info(f"Ventana: {src_w * WINDOW_SCALE_FACTOR}x{src_h * WINDOW_SCALE_FACTOR} "
 f"(factor={WINDOW_SCALE_FACTOR})")

fps_t = time.time(); fps_c = 0; fps = 0.0
frame_count = 0; last_status = time.time()
log.info("Bucle principal. Pulsa 'q' para salir.")

try:
 while True:
 # Capturar frame
 if use_video:
 ret, frame_bgr = cap.read()
 if not ret:
 log.info("Fin del vídeo.")
 break
 frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
 time.sleep(1.0 / max(vid_fps, 1.0))
 else:
 frame_rgb = cam.capture_array() # entrega RGB888

 frame_count += 1; fps_c += 1

```

```

Preparar imagen para inferencia cada 2 frames
(el hilo de inferencia procesa en background)
if frame_count % 2 == 0:
 resized = cv2.resize(frame_rgb, (nw, nh))
 lb = np.zeros((MODEL_SIZE, MODEL_SIZE, 3), dtype=np.uint8)
 lb[pad_y:pad_y + nh, pad_x:pad_x + nw] = resized

 if CONVERT_TO_BGR:
 # Conversión explícita si el modelo fue entrenado con BGR
 img_inf = cv2.cvtColor(lb, cv2.COLOR_RGB2BGR)
 if frame_count == 2:
 log.info("Modo BGR activado: convirtiendo RGB-BGR antes de inferencia")
 else:
 # Por defecto: pasar RGB tal cual (nuevo modelo entrenado con RGB)
 img_inf = lb

 with frame_lock:
 latest_frame = img_inf

Dibujar resultado (en frame_rgb original para la ventana)
frame_display = frame_rgb.copy()
with result_lock:
 result = latest_result

if result is not None:
 boxes, confs, kps_list, angles_list = result
 draw_results(frame_display, boxes, confs, kps_list, angles_list,
 fps, scale, pad_x, pad_y, frame_count, delta_t_s,
 hide_yaw=hide_yaw, kp_thresh=kp_thresh_val, log=log,
 dead_zone=dead_zone)
else:
 cv2.putText(frame_display, "Esperando inferencia...",
 (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

Calcular FPS de la UI (no del hilo de inferencia)
now = time.time()
if now - fps_t >= 1.0:
 fps = fps_c / (now - fps_t); fps_c = 0; fps_t = now

Status periódico al log (cada 10 s)
if now - last_status >= 10.0:
 n = len(result[0]) if result is not None else 0
 log.info(f"[STATUS] FPS={fps:.1f} frames={frame_count} "
 f"detecciones={n} log={log_path}")
 last_status = now

Mostrar y guardar
cv2.imshow(win_title, frame_display)
if writer:
 # VideoWriter espera BGR - frame_display es RGB
 writer.write(cv2.cvtColor(frame_display, cv2.COLOR_RGB2BGR))
if cv2.waitKey(1) & 0xFF == ord("q"):
 log.info("Saliendo por tecla 'q'.")
 break

except KeyboardInterrupt:
 log.info("Ctrl+C.")
except Exception as e:
 log.error(f"Excepción en bucle principal: {e}")
 log.error(traceback.format_exc())
finally:
 if cam:
 try: cam.stop()
 except: pass
 if cap:
 cap.release()
 cv2.destroyAllWindows()
 if writer:
 writer.release()
 log.info(f"Video guardado: {args.save}")
 try: target.release()
 except: pass
 log.info(f"Fin. Log en: {log_path}")

if __name__ == "__main__":
 main()

```